FORMATION – PARTIE 2

Jenkins

Projets complexes · Jenkins en cluster · Administration

Partie 4 · Construction des projets complexes | Partie 5 · Utilisation de Jenkins en cluster | Partie 6 · Administration de Jenkins

Utopios – MA-IT Consulting · Support v1.5 · Suite du support « Jenkins – Intégration continue » (parties 1 à 3)

Programme et fil rouge

4 – Construction des projets complexes

- Enchaînements de projets Maven
- Construire une application J2EE / Jakarta EE complète
- Construire un projet avec Ant
- Conjuguer plusieurs outils
- Déployer dans les référentiels Maven
- Piloter le déploiement d'applications

5 – Utilisation de Jenkins en cluster

- Configuration des agents (ex-« esclaves »)
- Modes de démarrage Unix, Windows
- Répartition des jobs entre agents
- Bonnes pratiques de déploiement

6 – Administration de Jenkins

- Configuration des vues
- Considérations multi plates-formes
- Visite guidée de la `JENKINS_HOME`
- Monitorer et sauvegarder Jenkins
- Scripts Jenkins en Groovy
- Ligne de commande d'administration

Fil rouge étendu : la suite « Library »

- `library-core` : bibliothèque Java partagée (entités, règles métier) – JAR
- `library-api` : API REST Spring Boot (parties 1-3) – JAR, dépend de `library-core`
- `library-portal` : application Jakarta EE (EJB + WAR + EAR) déployée sur WildFly
- `library-web` : front Angular (npm) servi par Nginx
- `library-legacy` : ancien module construit avec Ant

Prérequis

- Avoir suivi les parties 1 à 3 : Jenkins LTS (JDK 21) opérationnel, Tools `jdk21` / `maven-3.9`, credentials Git et Nexus, pipeline `library-api` fonctionnel
- Nexus et SonarQube démarrés (Docker Compose de la partie 2)
- Pour la partie 5 : une VM Linux et une VM Windows (ou deux conteneurs) joignables depuis le controller

04

Construction des projets complexes

Enchaînements de projets Maven · Application J2EE complète · Projet Ant · Conjuguer plusieurs outils · Référentiels Maven · Piloter le déploiement

Enchaînements de projets (1/3) : upstream / downstream

Le problème

- `library-api` dépend de `library-core` : quand `core` change, `api` doit être reconstruit et retesté
- Après un build réussi, on veut lancer un job de déploiement, de tests E2E, de packaging…
- Plusieurs dépôts Git = plusieurs jobs : il faut les **orchestrer**

Vocabulaire

- **Upstream project** : le job amont (ex. `library-core`)
- **Downstream project** : le job aval déclenché (ex. `library-api`)
- Jenkins affiche les relations sur la page du job : *Upstream Projects* / *Downstream Projects*

Mécanismes disponibles

Mécanisme	Où	Remarque
Build after other projects are built	Trigger du job aval	Simple, condition : stable / unstable / failed
Build other projects (post-build)	Job amont	Déclenche sans attendre
Parameterized Trigger (plugin)	Amont, build step ou post-build	Passe des paramètres, peut **bloquer** jusqu'à la fin
Step `build job:`	Pipeline	Le standard : attend, récupère le résultat
Dépendances Maven (SNAPSHOT)	Maven project / `withMaven`	Déclenchement automatique par le graphe de dépendances

Freestyle : configurer l'enchaînement

1. Job `library-core` : *Post-build Actions* → *Build other projects* → `library-api`, *Trigger only if build is stable*
2. Ou côté `library-api` : *Build Triggers* → *Build after other projects are built* → `library-core`
3. Vérifier sur la page de `library-api` : *Upstream Projects : library-core*
4. Lancer `library-core` ; observer dans la Build Queue puis dans l'historique de `library-api` la cause « Started by upstream project library-core build number 12 »

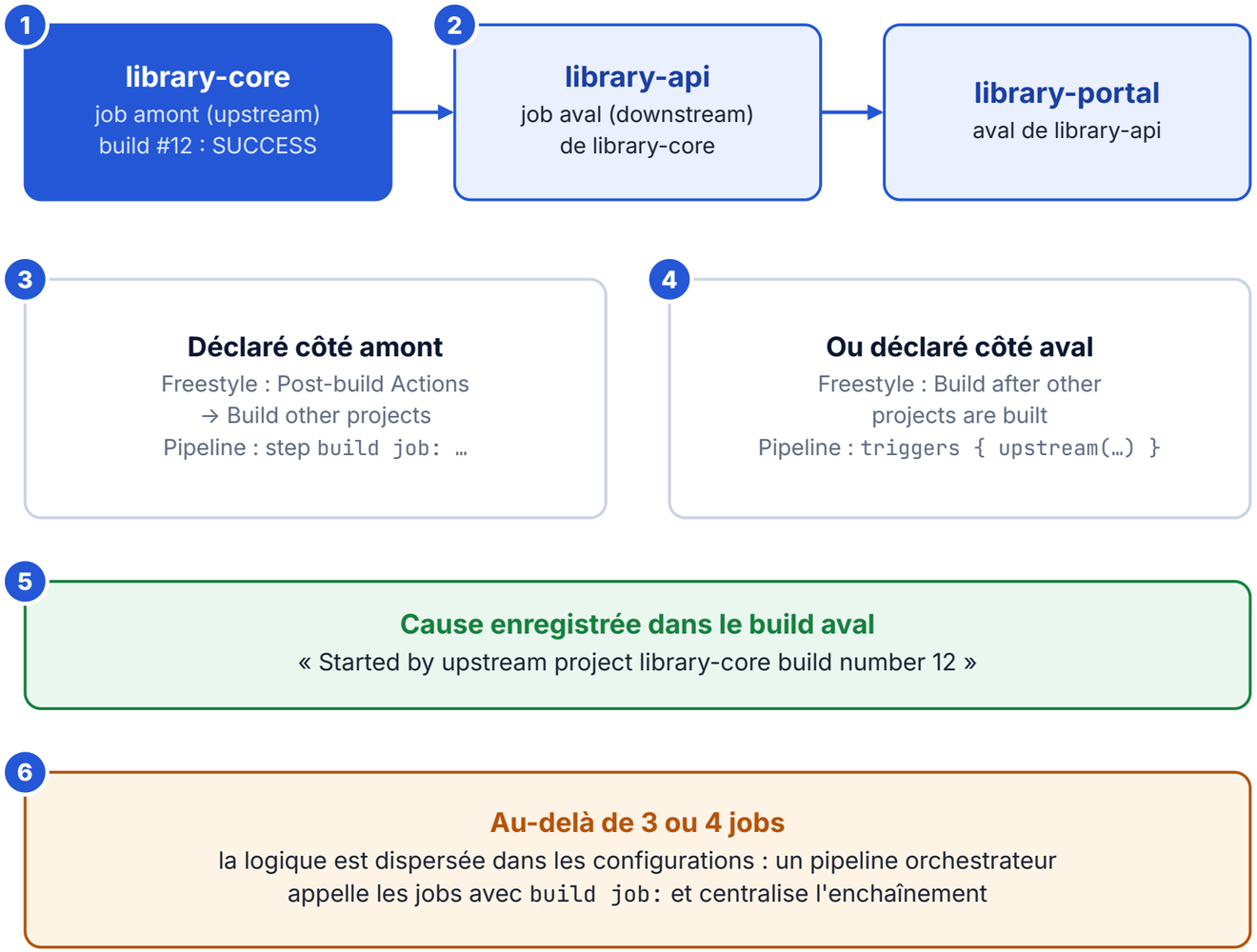
Dans un Jenkinsfile (côté aval)

```
pipeline {
  agent any
  triggers {
    upstream(upstreamProjects: 'library/library-core/main',
             threshold: hudson.model.Result.SUCCESS)
  }
  stages { /* … */ }
}
```

Limite

Ce couplage « job A → job B » dispersé dans les configurations devient vite illisible au-delà de 3 ou 4 jobs. Un pipeline orchestrateur centralise la logique.

Schéma : un enchaînement upstream / downstream



Comment le lire

1. `library-core` se termine avec succès : c'est le job **amont**.
2. `library-api`, qui en dépend, est relancé : c'est le job **aval**. Il peut lui-même être l'amont d'un autre job.
3. Le lien peut être déclaré **côté amont** (« je lance les suivants »)…
4. … ou **côté aval** (« je démarre après celui-ci ») ; Jenkins affiche les deux sens sur la page du job.
5. La **cause** du build aval garde la trace de l'amont et de son numéro.
6. Quand la chaîne s'allonge, on la remplace par un pipeline orchestrateur.

À retenir

La condition (stable, instable, échec) se règle sur le lien : par défaut, on ne propage qu'un amont **stable**.

Enchaînements de projets (2/3) : orchestrer depuis un pipeline

Le step `build`

```
stage('Build dependencies') {
  steps {
    script {
      // Lance library-core et ATTEND son résultat
      def core = build job: 'library/library-core/main',
                       wait: true,
                       propagate: true,        // échec amont ⇒ échec ici
                       parameters: [
                         string(name: 'RELEASE_VERSION', value: '2.3.0')
                       ]
      echo "library-core #${core.number} : ${core.result}"
      env.CORE_BUILD = core.number
    }
  }
}

stage('Trigger E2E') {
  steps {
    // Fire-and-forget : ne bloque pas ce pipeline
    build job: 'library/e2e-tests', wait: false,
          parameters: [string(name: 'API_VERSION', value: env.APP_VERSION)]
  }
}
```

- `wait: true` + `propagate: false` : récupérer le résultat et décider soi-même (`core.result == 'UNSTABLE'`)
- Le job appelé peut être un Multibranch : chemin `folder/job/branche`

Récupérer les artefacts d'un autre job

```
// plugin Copy Artifact
stage('Fetch core jar') {
  steps {
    copyArtifacts projectName: 'library/library-core/main',
                  selector: specific("${env.CORE_BUILD}"),   // ou lastSuccessful()
                  filter: 'target/library-core-*.jar',
                  target: 'libs/', flatten: true,
                  fingerprintArtifacts: true
  }
}
```

Le job source doit autoriser la copie : `options { copyArtifactPermission('library/*') }` .

Préférer le dépôt d'artefacts

> **Règle**
>
> Entre projets Maven, **ne pas** copier des JAR entre workspaces : `library-core` publie dans Nexus (`mvn deploy`), `library-api` le consomme comme dépendance normale. Jenkins n'orchestre que l'ordre et le feedback ; Maven gère le contrat.

Pipeline orchestrateur

```
stage('Suite Library') {
  parallel {
    stage('core')   { steps { build 'library/library-core/main' } }
    stage('legacy') { steps { build 'library/library-legacy' } }
  }
}
stage('API')    { steps { build 'library/library-api/main' } }
stage('Portal') { steps { build 'library/library-portal/main' } }
```

Enchaînements de projets (3/3) : dépendances Maven et multi-modules

Déclenchement par le graphe de dépendances

- Le plugin **Pipeline Maven Integration** (`withMaven`) enregistre, pour chaque build, les artefacts **produits** et les dépendances **consommées**
- Quand un build publie `fr.utopios:library-core:2.3.0-SNAPSHOT`, tout pipeline qui a consommé ce SNAPSHOT est déclenché automatiquement
- Activation : *Manage Jenkins → Tools → Pipeline Maven Configuration*, avec une base PostgreSQL ou MySQL (H2 n'est plus préconfiguré et tourne dans la JVM de Jenkins), puis dans le Jenkinsfile aval :

```
triggers {
  snapshotDependencies()      // déclenché quand un SNAPSHOT consommé est rebuildé
}
```

- Équivalent legacy sur un « Maven project » : *Build whenever a SNAPSHOT dependency is built*
- Fonctionne uniquement pour les **SNAPSHOT** : une release est immuable, la changer est un commit dans le POM aval

Visualiser

Page du job → *Dependency graph* (plugin Dependency Graph Viewer) : vue des relations upstream/downstream calculées.

Alternative : un seul dépôt multi-modules

```
library-suite/
├── pom.xml            <packaging>pom</packaging> + <modules>
├── library-core/pom.xml
├── library-api/pom.xml      dépend de library-core (même ${project.version})
└── library-portal/pom.xml
```

- Le **reactor** Maven ordonne les modules selon leurs dépendances et construit tout en une commande
- Un seul Jenkinsfile, un seul numéro de build pour toute la suite

```
# Tout construire
mvn -B -T 1C clean verify

# Construire seulement library-api et ce dont il dépend (also-make)
mvn -B -pl library-api -am clean verify

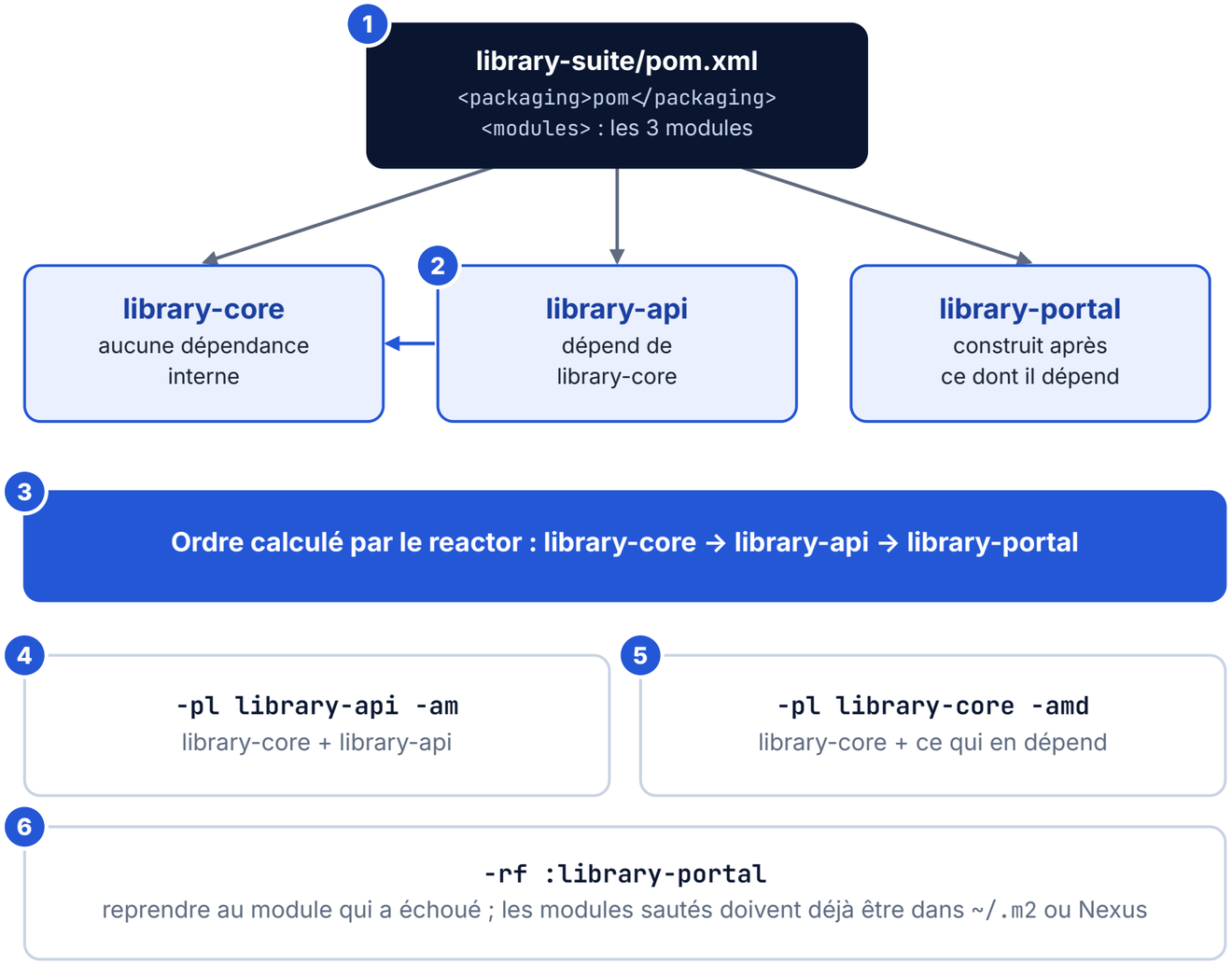
# Construire ce qui dépend de library-core (also-make-dependents)
mvn -B -pl library-core -amd clean verify

# Reprendre après l'échec d'un module
mvn -B -rf :library-portal clean verify
```

Choisir

Multi-dépôts + enchaînement	Mono-dépôt multi-modules
Équipes et cycles de release indépendants	Composants versionnés ensemble
Builds plus courts, plus de jobs à orchestrer	Un build plus long, zéro orchestration

Schéma : le reactor Maven d'un dépôt multi-modules



Comment le lire

1. Le POM parent liste les modules : une seule commande construit toute la suite.
2. Les dépendances entre modules (`library-api` → `library-core`) sont lues dans leurs POM.
3. Le **reactor** en déduit l'ordre de construction : un module passe toujours après ceux dont il dépend.
4. `-am` (also-make) : construire un module **et ce dont il dépend**.
5. `-amd` (also-make-dependents) : construire un module **et ce qui dépend de lui**.
6. `-rf` (resume-from) : reprendre après un échec, à condition que les modules sautés aient été installés.

À retenir
Dans un mono-dépôt, l'enchaînement upstream/downstream disparaît : c'est Maven qui ordonne, dans un seul build Jenkins.

Application J2EE / Jakarta EE complète (1/3) : structure

Rappels

- **J2EE** (1999) → **Java EE** (2006) → **Jakarta EE** 8 (2019, Eclipse Foundation, encore `javax.*`) → Jakarta EE 9 (2020, bascule vers `jakarta.*`). Jakarta EE 10/11 aujourd'hui.
- Serveurs d'applications : **WildFly** / JBoss EAP, **Payara** / GlassFish, Open Liberty, TomEE ; Tomcat/Jetty sont des servlet containers : WAR uniquement, sans les API du Web Profile (CDI, JPA, EJB Lite…) – TomEE, lui, implémente le Web Profile
- Archives :
  - **JAR** : bibliothèques, modules EJB (`packaging ejb`)
  - **WAR** : module web (servlets, JAX-RS, JSF)
  - **EAR** : *Enterprise Archive* qui agrège EJB + WAR + libs ; `META-INF/application.xml` optionnel depuis Java EE 5 (généré par maven-ear-plugin)

Pourquoi c'est « complexe » pour la CI

- Plusieurs modules avec ordre de construction imposé
- Dépendances `provided` (fournies par le serveur) : ne doivent pas être embarquées
- Tests d'intégration qui nécessitent un vrai serveur d'applications
- Déploiement spécifique au serveur (CLI, management API, hot deploy)

Fil rouge : `library-portal`

```
library-portal/                      packaging pom (parent + agrégateur)
├── pom.xml
├── portal-ejb/                      packaging ejb  - services métier, JPA
│   └── src/main/java/…/LoanServiceBean.java
├── portal-web/                      packaging war  - JAX-RS, JSF, servlets
│   └── src/main/webapp/WEB-INF/web.xml
├── portal-ear/                      packaging ear  - assemblage
│   └── pom.xml  (maven-ear-plugin)
└── Jenkinsfile
```



Application J2EE / Jakarta EE complète (2/3) : construire l'EAR avec Maven

POM parent (`library-portal/pom.xml`)

```
<packaging>pom</packaging>
<modules>
  <module>portal-ejb</module>
  <module>portal-web</module>
  <module>portal-ear</module>
</modules>
<dependencyManagement>
  <dependencies>
    <dependency>                <!— API Jakarta EE fournie par le serveur —>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>10.0.0</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>fr.utopios</groupId>
      <artifactId>library-core</artifactId>
      <version>2.3.0</version>      <!— release depuis Nexus —>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Module EJB

```
<packaging>ejb</packaging>
<build><plugins><plugin>
  <artifactId>maven-ejb-plugin</artifactId>
  <version>3.2.1</version>
  <configuration><ejbVersion>4.0</ejbVersion></configuration>
</plugin></plugins></build>
```

Module EAR (`portal-ear/pom.xml`)

```
<packaging>ear</packaging>
<dependencies>
  <dependency>
    <groupId>fr.utopios</groupId><artifactId>portal-ejb</artifactId>
    <version>${project.version}</version><type>ejb</type>
  </dependency>
  <dependency>
    <groupId>fr.utopios</groupId><artifactId>portal-web</artifactId>
    <version>${project.version}</version><type>war</type>
  </dependency>
</dependencies>
<build><plugins><plugin>
  <artifactId>maven-ear-plugin</artifactId>
  <version>3.3.0</version>
  <configuration>
    <version>10</version>
    <defaultLibBundleDir>lib</defaultLibBundleDir>
    <skinnyWars>true</skinnyWars>        <!— libs partagées dans lib/, pas dans le WAR —>
    <modules>
      <ejbModule><groupId>fr.utopios</groupId><artifactId>portal-ejb</artifactId></ejbModule>
      <webModule>
        <groupId>fr.utopios</groupId><artifactId>portal-web</artifactId>
        <contextRoot>/library</contextRoot>
      </webModule>
    </modules>
  </configuration>
</plugin></plugins></build>
```

Build

```
mvn -B clean verify         # reactor : ejb → web → ear
ls portal-ear/target/*.ear  # portal-ear-1.0.0-SNAPSHOT.ear (artifactId-version)
```

Le reactor garantit l'ordre ; dans Jenkins, un seul stage Maven suffit pour les trois modules.

Application J2EE / Jakarta EE complète (3/3) : tester et déployer sur WildFly

Tests d'intégration sur un vrai serveur

- **Arquillian** : exécute les tests *dans* le conteneur (WildFly managed/embedded) ; référence historique Java EE
- **Testcontainers** : démarre `quay.io/wildfly/wildfly` en Docker, déploie l'EAR, teste via HTTP – plus simple en CI, nécessite Docker sur l'agent
- Rapports Failsafe → step `junit 'portal-*/target/failsafe-reports/*.xml'`

Déploiement : plugin Maven WildFly

```
<plugin>
  <groupId>org.wildfly.plugins</groupId>
  <artifactId>wildfly-maven-plugin</artifactId>
  <version>6.2.0.Final</version>
  <configuration>
    <hostname>${wildfly.host}</hostname>
    <port>9990</port>
    <username>${env.WF_USR}</username>
    <password>${env.WF_PSW}</password>
    <name>library-portal.ear</name>   <!— nom stable : redeploy écrase —>
  </configuration>
</plugin>
```

```
# après un mvn install : le goal relance package sur portal-ear
# et résout portal-ejb / portal-web dans le dépôt local
mvn -B -pl portal-ear wildfly:deploy -Dwildfly.host=recette.example.com
```

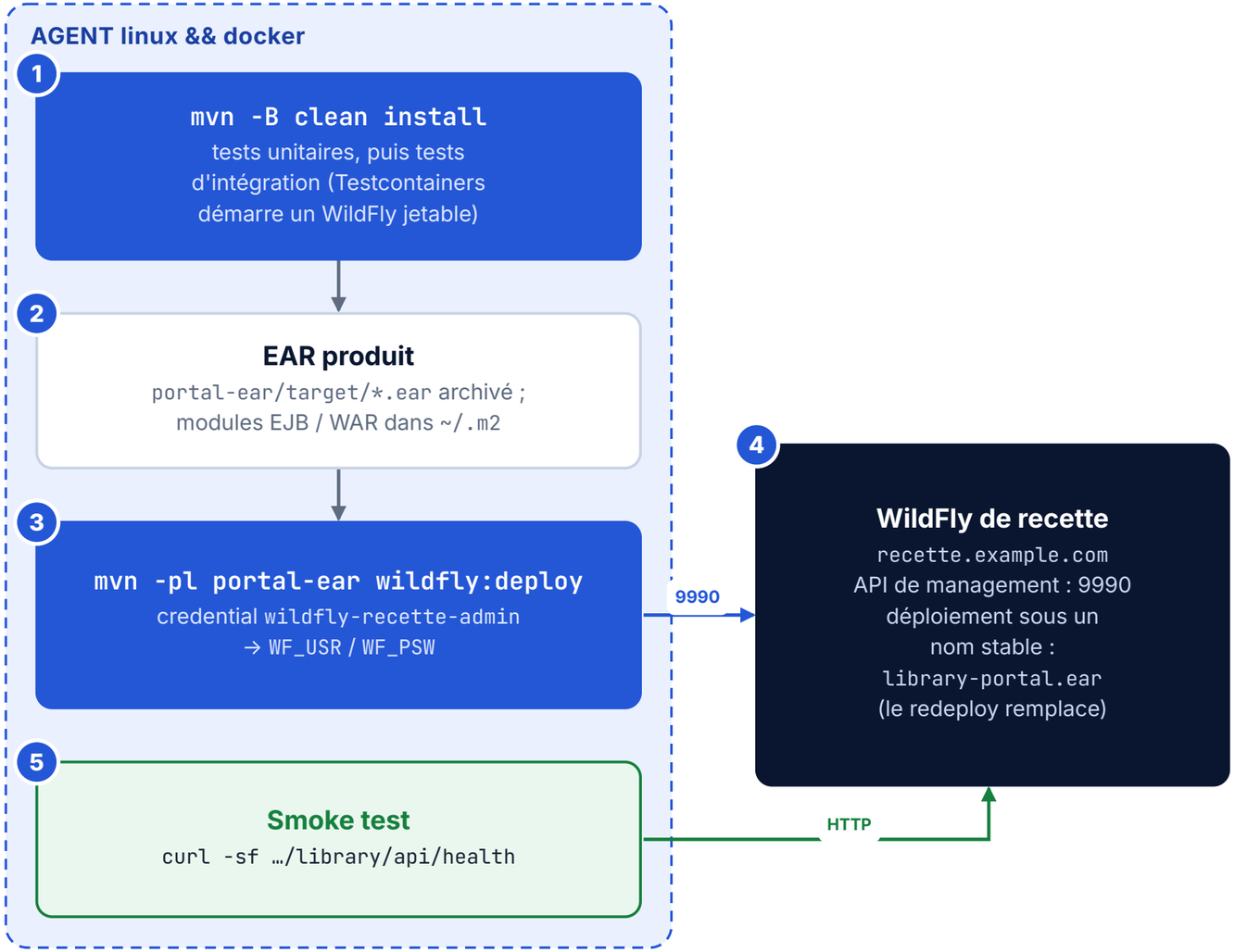
Jenkinsfile de `library-portal`

```
pipeline {
  agent { label 'linux && docker' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }
  stages {
    stage('Build EAR') {
      steps { sh 'mvn -B clean install' }    // unit + Testcontainers ; install pour le deploy
      post { always {
        junit '**/target/surefire-reports/*.xml, **/target/failsafe-reports/*.xml'
        archiveArtifacts 'portal-ear/target/*.ear'
      } }
    }
    stage('Deploy recette') {
      when { branch 'main' }
      environment { WF = credentials('wildfly-recette-admin') }  // WF_USR / WF_PSW
      steps {
        sh 'mvn -B -pl portal-ear wildfly:deploy -Dwildfly.host=recette.example.com'
        sh 'curl -sf https://recette.example.com/library/api/health'
      }
    }
  }
}
```

Autres voies de déploiement

Moyen	Usage
`jboss-cli.sh --connect --command="deploy …"`	Script shell, scriptable à distance
Plugin **Deploy to container**	Tomcat/JBoss/GlassFish via UI, Freestyle
Copie dans `standalone/deployments/`	Hot deploy, dev uniquement
Image Docker `FROM quay.io/wildfly/wildfly` + EAR	Approche conteneur : immutable, Kubernetes

Schéma : de l'EAR testé au WildFly de recette



Comment le lire

1. Sur un agent qui a Docker, `mvn install` exécute les tests unitaires puis les tests d'intégration contre un WildFly démarré par Testcontainers.
2. L'EAR est archivé ; `install` met les modules EJB et WAR dans `~/.m2`, d'où `-pl portal-ear` les résout quand `wildfly:deploy` relance `package`.
3. `wildfly:deploy` envoie l'EAR à l'**API de management** du serveur, avec les identifiants injectés par le credential.
4. WildFly déploie sous un nom stable : un nouveau déploiement remplace le précédent.
5. Un appel HTTP de santé vérifie que l'application répond avant de déclarer le déploiement réussi.

À retenir
Le port 9990 (management) ne doit être ouvert qu'aux agents de déploiement, jamais au public.

Construire un projet avec Ant (1/2) : rappels

Apache Ant

- Outil de build Java historique (2000), **procédural** : on décrit *comment* construire, tâche par tâche
- Fichier `build.xml` : **project** → **targets** (avec dépendances `depends`) → **tasks** (`javac`, `jar`, `copy`, `junit`, `exec`…)
- Pas de gestion de dépendances native : JAR dans `lib/` versionnés avec le code, ou **Apache Ivy** pour les résoudre depuis Maven Central / Nexus
- Toujours présent dans les projets legacy, les bases de code anciennes, certains outils (Android historique, NetBeans)

Ant vs Maven

Ant	Maven
Procédural, flexible, verbeux	Déclaratif, conventions, cycle de vie
Chaque projet est différent	Tous les projets se ressemblent
Dépendances manuelles (ou Ivy)	Dépendances transitives, dépôts
Pas de format de rapport imposé	Surefire → JUnit XML standard

`library-legacy/build.xml`

```
<project name="library-legacy" default="dist" basedir=".">
  <property name="src" location="src"/>
  <property name="build" location="build"/>
  <property name="dist" location="dist"/>
  <property name="version" value="1.0.0"/>        <!— surchargé par -Dversion —>
  <path id="classpath"><fileset dir="lib" includes="*.jar"/></path>

  <target name="clean"><delete dir="${build}"/><delete dir="${dist}"/></target>

  <target name="compile">
    <mkdir dir="${build}/classes"/>
    <javac srcdir="${src}/main" destdir="${build}/classes" classpathref="classpath"
           release="21" includeantruntime="false" debug="true"/>
  </target>

  <target name="test" depends="compile">
    <mkdir dir="${build}/test-classes"/><mkdir dir="${build}/test-reports"/>
    <javac srcdir="${src}/test" destdir="${build}/test-classes" includeantruntime="false">
      <classpath><path refid="classpath"/><pathelement location="${build}/classes"/></classpath>
    </javac>
    <junit printsummary="yes" haltonfailure="no" failureproperty="tests.failed">
      <classpath><path refid="classpath"/>
        <pathelement location="${build}/classes"/><pathelement location="${build}/test-classes"/></classpath>
      <formatter type="xml"/>                    <!— format JUnit XML lisible par Jenkins —>
      <batchtest todir="${build}/test-reports">
        <fileset dir="${src}/test" includes="**/*Test.java"/>
      </batchtest>
    </junit>
  </target>

  <target name="dist" depends="test">
    <mkdir dir="${dist}"/>
    <jar destfile="${dist}/library-legacy-${version}.jar" basedir="${build}/classes"/>
    <fail if="tests.failed" message="Tests en échec"/>
  </target>
</project>
```

Construire un projet avec Ant (2/2) : dans Jenkins

Prérequis

1. Plugin **Ant** installé (*Manage Jenkins → Plugins*)
2. *Manage Jenkins → Tools → Ant installations* : nom `ant-1.10`, *Install automatically* (Apache) ou chemin sur l'agent
3. Si Ivy : plugin **Ivy** ou simplement `ivy.jar` dans `~/.ant/lib` de l'agent

Freestyle

- *Build Steps → Invoke Ant* : Ant Version `ant-1.10`, Targets `clean dist`, Build File `build.xml`, Properties `version=1.0.${BUILD_NUMBER}`
- *Post-build → Publish JUnit test result report* : `build/test-reports/*.xml`
- *Archive the artifacts* : `dist/*.jar`

Point d'attention

Ant ne fait pas échouer la commande sur un test rouge sauf si on le lui demande (`haltonfailure` / `fail if`). Sans cela, Jenkins verrait un build SUCCESS avec des tests en échec : c'est le step `junit` qui rétablit UNSTABLE.

Pipeline

```
pipeline {
  agent { label 'linux' }
  tools { jdk 'jdk21'; ant 'ant-1.10' }         // ANT_HOME positionné, ant dans le PATH
  environment { LEGACY_VERSION = "1.0.${BUILD_NUMBER}" }
  stages {
    stage('Build legacy') {
      steps {
        sh 'ant -version'
        sh 'ant -Dversion=$LEGACY_VERSION clean dist'
      }
      post {
        always  { junit 'build/test-reports/*.xml' }
        success { archiveArtifacts 'dist/*.jar' }
      }
    }
    stage('Publish to Nexus') {
      when { branch 'main' }
      steps {
        // Pas de pom : upload direct via le plugin Nexus Artifact Uploader
        nexusArtifactUploader nexusVersion: 'nexus3', protocol: 'http', nexusUrl: 'nexus:8081',
          repository: 'maven-releases', credentialsId: 'nexus-deploy',
          groupId: 'fr.utopios', version: env.LEGACY_VERSION,
          artifacts: [[artifactId: 'library-legacy', type: 'jar', classifier: '',
                       file: "dist/library-legacy-${env.LEGACY_VERSION}.jar"]]
      }
    }
  }
}
```

Moderniser progressivement

- **Étape 1** : Ant + Ivy pour sortir les JAR de `lib/` du dépôt Git
- **Étape 2** : `maven-antrun-plugin` pour appeler des targets Ant depuis un POM pendant la migration
- **Étape 3** : POM complet, suppression de `build.xml`

Conjuguer plusieurs outils (1/3) : front npm + back Maven + Docker

La réalité d'une application

- Backend Java (Maven ou Gradle), frontend JavaScript (npm / pnpm / yarn), base de données (migrations Flyway/Liquibase), image Docker, manifestes Kubernetes/Helm, tests E2E (Playwright)
- Chaque outil a ses prérequis (Node 22, JDK 21, Docker), son cache, son format de rapport

Trois approches pour outiller les agents

Approche	Avantages	Limites
Tools Jenkins (`tools { jdk; maven; nodejs }`)	Simple, versions choisies par job	Installation sur chaque agent, pas de Docker
Un conteneur par stage (`agent { docker { image } }`)	Image officielle, zéro installation, isolation	Docker requis sur l'agent, workspace partagé par volume
Image d'agent « tout-en-un »	Rapide, prévisible	Image lourde à maintenir

Recommandation

Un conteneur par stage avec les images officielles (`maven:3.9-eclipse-temurin-21` , `node:22-alpine`) : la version de l'outil est dans le Jenkinsfile, donc versionnée avec le code.

Plugin NodeJS (approche Tools)

- *Manage Jenkins* → *Tools* → *NodeJS installations* : nom `node-22` , *Install automatically* (nodejs.org), *Global npm packages to install* : `@angular/cli`

```
stage('Front') {
  tools { nodejs 'node-22' }
  steps {
    dir('library-web') {
      sh 'node -v && npm ci'
      sh 'npm run lint'
      sh 'npm test -- --watch=false'   // reporter JUnit configuré dans le projet
      sh 'npm run build -- --configuration production'
    }
  }
  post { always { junit 'library-web/test-results/*.xml' } }
}
```

Rapports non-Java

- Vitest (défaut d'Angular depuis la v21), Jest, Karma : reporter **junit** configuré dans le projet → même step `junit` ; Karma exige un navigateur (Chrome) sur l'agent
- ESLint 9 : `npm i -D eslint-formatter-checkstyle` , puis `eslint -f checkstyle -o eslint.xml` → `recordIssues tools: [esLint(pattern: 'eslint.xml')]`
- Couverture : reporter `cobertura` ou `lcov` de Vitest/Jest/Istanbul → `recordCoverage tools: [[parser: 'COBERTURA', pattern: 'coverage/cobertura-coverage.xml']]` (ou `parser: 'LCOV', pattern: 'coverage/lcov.info'`)

Conjuguer plusieurs outils (2/3) : pipeline multi-outils avec conteneurs — le code

```groovy
pipeline {
  agent none                                  // chaque stage choisit son agent
  options { timestamps(); skipDefaultCheckout() }
  environment { APP_VERSION = "2.0.${BUILD_NUMBER}" }

  stages {
    stage('Checkout') {
      agent { label 'linux && docker' }
      steps { checkout scm; stash name: 'src', includes: '**', excludes: '**/node_modules/**' }
    }

    stage('Build') {
      parallel {
        stage('Backend (Maven)') {
          // le conteneur tourne avec l'uid de l'agent, pas root : cache sur un
          // répertoire de l'hôte appartenant à cet utilisateur
          agent { docker { image 'maven:3.9-eclipse-temurin-21'; label 'docker'
                           args '-v /var/cache/m2:/m2' } }
          steps {
            unstash 'src'
            sh 'mvn -B -ntp -Dmaven.repo.local=/m2 -Drevision=$APP_VERSION clean verify'
            stash name: 'jar', includes: 'library-api/target/*.jar'
          }
          post { always { junit 'library-api/target/surefire-reports/*.xml' } }
        }
        stage('Frontend (npm)') {
          agent { docker { image 'node:22-alpine'; label 'docker' } }
          steps {
            unstash 'src'
            dir('library-web') {
              sh 'npm ci && npm run lint && npm test -- --watch=false'  // Vitest : pas de navigateur
              sh 'npm run build -- --configuration production'
            }
            stash name: 'dist', includes: 'library-web/dist/**'
          }
          post { always { junit 'library-web/test-results/*.xml' } }
        }
      }
    }

    stage('Images Docker') {
      agent { label 'linux && docker' }
      steps {
        unstash 'src'; unstash 'jar'; unstash 'dist'
        script {
          docker.withRegistry('https://registry.example.com', 'docker-registry') {
            docker.build("utopios/library-api:${APP_VERSION}", 'library-api').push()
            docker.build("utopios/library-web:${APP_VERSION}", 'library-web').push()
          }
        }
      }
    }

    stage('E2E (Playwright)') {
      agent { label 'linux && docker' }           // docker compose tourne sur l'hôte
      steps {
        unstash 'src'
        sh 'docker compose -f e2e/compose.yml up -d'    // api + web + db
        script {
          // @playwright/test dans e2e/package.json à la même version que l'image
          docker.image('mcr.microsoft.com/playwright:v1.49.0-noble').inside('--network host') {
            sh 'cd e2e && npm ci && PLAYWRIGHT_JUNIT_OUTPUT_NAME=results.xml npx playwright test --reporter=junit'
          }
        }
      }
      post { always { junit 'e2e/results.xml'; sh 'docker compose -f e2e/compose.yml down -v' } }
    }
  }
}
```

Conjuguer plusieurs outils (2/3) : pipeline multi-outils avec conteneurs — lecture

Lecture

- `agent none` au niveau pipeline : chaque stage déclare son agent ; pas d'executor bloqué pendant les attentes
- `parallel` : backend et frontend construits en même temps sur des agents différents – gain de temps direct
- `stash` / `unstash` : transférer des fichiers entre stages qui s'exécutent sur des agents différents (passe par le controller : réserver aux petits fichiers ; au-delà, Nexus/Artifactory)
- Images officielles pinnées : la version de Node, Maven, Playwright est dans le Jenkinsfile
- Chaque outil produit du JUnit XML → un seul step `junit` par stage, un seul onglet Test Result

Variante : un stage, plusieurs conteneurs

```
// Kubernetes : un pod multi-conteneurs
agent {
  kubernetes { yaml '''
    spec:
      containers:
      - name: maven
        image: maven:3.9-eclipse-temurin-21
        command: ["sleep"]
        args: ["infinity"]
      - name: node
        image: node:22-alpine
        command: ["sleep"]
        args: ["infinity"]
  ''' }
}
steps {
  container('maven') { sh 'mvn -B verify' }
  container('node')  { sh 'npm ci && npm run build' }
}
```

Même workspace partagé par les conteneurs du pod : pas de stash nécessaire.

Conjuguer plusieurs outils (3/3) : factoriser avec les Shared Libraries

Le problème

- 15 dépôts, 15 Jenkinsfile quasi identiques (Maven + Sonar + Nexus + Docker)
- Une correction (nouvelle version d'image, nouveau credential) = 15 PR

Shared Library

- Un dépôt Git `jenkins-shared-lib` avec une structure imposée :

```
jenkins-shared-lib/
├── vars/                       # steps globaux (un fichier = un step)
│   ├── mavenBuild.groovy       # → mavenBuild(goals: 'verify')
│   ├── sonarAnalysis.groovy
│   ├── dockerPublish.groovy
│   ├── notifyTeam.groovy
│   └── javaServicePipeline.groovy   # pipeline complet paramétré
├── src/fr/utopios/ci/          # classes Groovy (helpers)
│   └── Versioning.groovy
└── resources/                  # fichiers (templates, scripts) via libraryResource
```

- Déclaration : *Manage Jenkins → System → Global Pipeline Libraries* : nom `utopios-ci`, SCM Git, version par défaut `main`, *Load implicitly* ou `@Library('utopios-ci') _`
- Les libs globales s'exécutent **hors sandbox** : dépôt protégé, revue obligatoire. Les libs déclarées au niveau Folder restent en sandbox.

`vars/mavenBuild.groovy`

```groovy
def call(Map cfg = [:]) {
  def goals  = cfg.goals  ?: 'clean verify'
  def image  = cfg.image  ?: 'maven:3.9-eclipse-temurin-21'
  docker.image(image).inside('-v /var/cache/m2:/m2') {   // uid de l'agent, pas root
    sh "mvn -B -ntp -Dmaven.repo.local=/m2 ${goals}"
  }
  junit allowEmptyResults: true, testResults: '**/target/surefire-reports/*.xml'
}
```

`vars/javaServicePipeline.groovy` **(pipeline complet)**

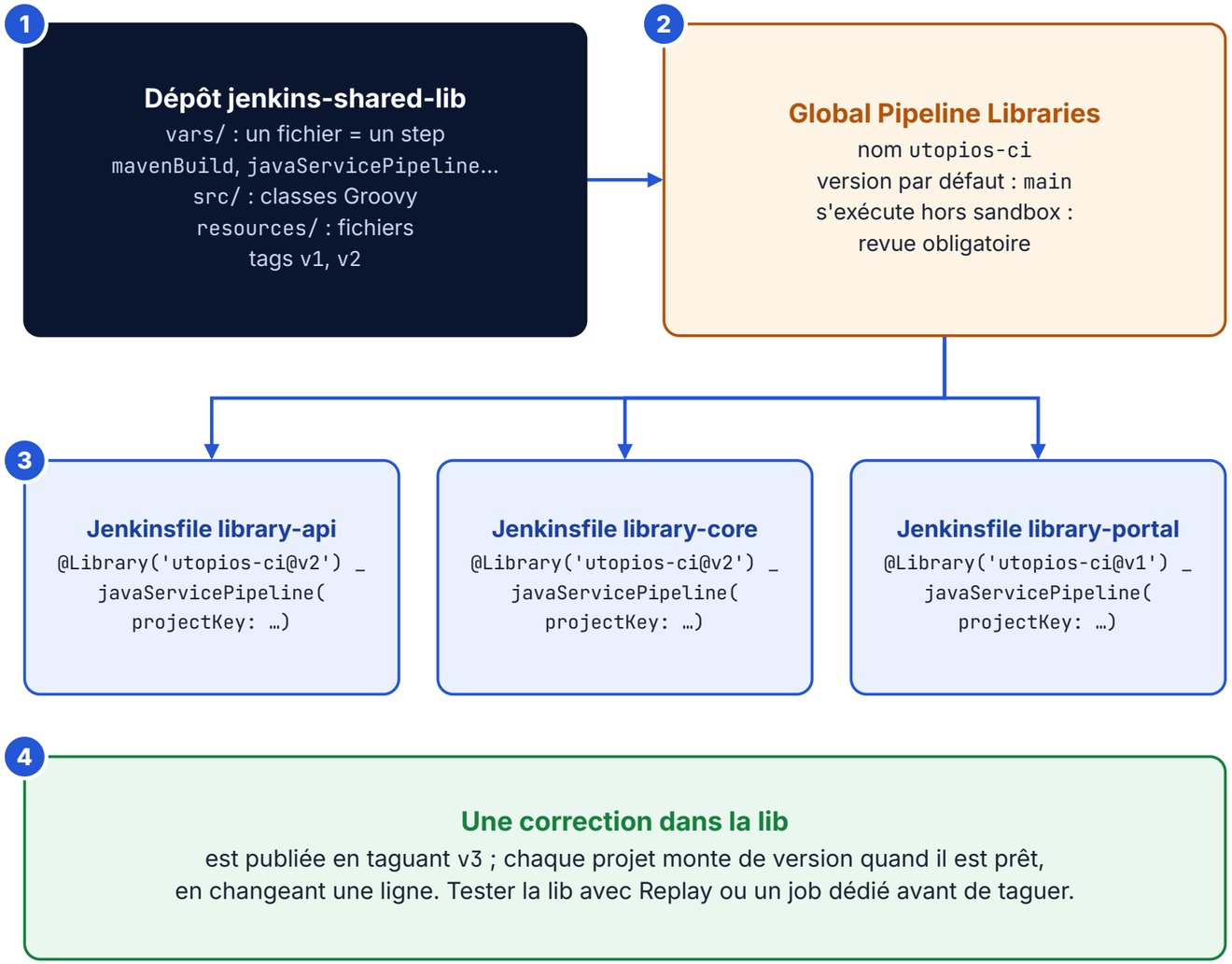
```groovy
def call(Map cfg) {
  pipeline {
    agent { label 'linux && docker' }
    stages {
      stage('Build')   { steps { mavenBuild() } }
      stage('Quality') { steps { sonarAnalysis(projectKey: cfg.projectKey) } }
      stage('Publish') { when { branch 'main' }
                         steps { dockerPublish(image: cfg.image) } }
    }
    post { failure { notifyTeam(cfg.channel) } }
  }
}
```

Jenkinsfile d'un service : 4 lignes

```groovy
@Library('utopios-ci@v2') _
javaServicePipeline(
  projectKey: 'fr.utopios:library-api',
  image: 'utopios/library-api', channel: '#ci-library')
```

Versionner la lib par tags (`@v2`) : les projets choisissent quand monter de version. Tester la lib avec *Replay* ou un job dédié avant de taguer.

Schéma : une Shared Library pour N pipelines



Comment le lire

1. La logique commune vit dans **un dépôt Git** : chaque fichier de `vars/` devient un step utilisable dans les Jenkinsfile.
2. Jenkins connaît la lib par son nom (Global Pipeline Libraries). Une lib globale s'exécute **hors sandbox** : son dépôt doit être protégé et revu.
3. Chaque projet importe la lib à une **version** (tag) et n'écrit plus que quelques lignes de paramètres.
4. Une évolution est publiée sous un nouveau tag : les projets choisissent quand l'adopter.

À retenir
15 Jenkinsfile identiques = 15 PR pour une correction ; avec la lib, une PR et un tag.

Déployer dans les référentiels Maven (1/3) : SNAPSHOT, release, versioning

Deux types de dépôts, deux politiques

	`maven-snapshots`	`maven-releases`
Versions	`2.3.0-SNAPSHOT`	`2.3.0`
Redéploiement	Autorisé (horodaté : `2.3.0-20261024.091500-7`)	**Interdit** (Deployment policy : Disable redeploy)
Rétention	Nettoyage automatique (garder N derniers)	Conservé
Usage	Intégration continue entre équipes	Livraison, production, traçabilité

Stratégies de version

- **maven-release-plugin** : classique. `release:prepare` (retire -SNAPSHOT, tag Git, incrémente) puis `release:perform` (checkout du tag, deploy). Deux commits dans Git, lent, fragile en CI.
- **CI-friendly versions** (`${revision}`) : la version est calculée par Jenkins (`1.4.${BUILD_NUMBER}` , ou depuis un tag Git) et injectée avec `-Drevision` . Aucun commit, un seul build. **Recommandé**, avec le *flatten-maven-plugin*, obligatoire dès qu'on fait `install` ou `deploy` (sinon le POM publié contient `${revision}` non résolu).
- **Semantic versioning** : `MAJEUR.MINEUR.CORRECTIF` ; le tag Git `v2.3.0` est la source de vérité

Flux recommandé

Branche feature / PR

Build + tests + Sonar. Aucune publication.

Branche main

`mvn deploy -Drevision=2.3.0-SNAPSHOT` → `maven-snapshots` . Les projets aval consomment le SNAPSHOT et sont déclenchés.

Tag v2.3.0

Build du tag : `mvn deploy -Drevision=2.3.0` → `maven-releases` . Immuable.

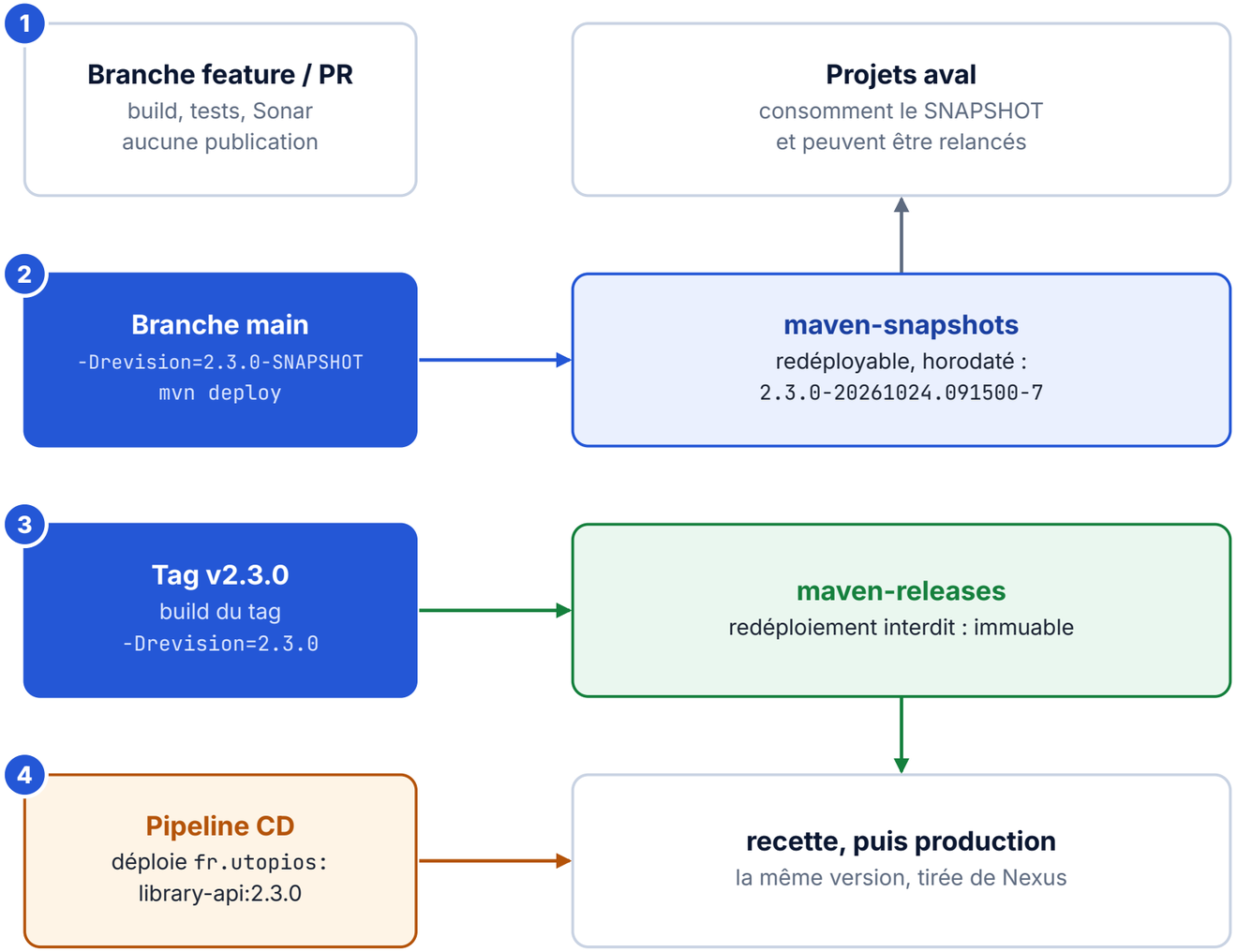
Pipeline CD

Déploie `fr.utopios:library-api:2.3.0` depuis Nexus vers recette, puis production.

```
// Jenkinsfile : version selon le contexte
script {
  if (env.TAG_NAME) {
    env.REVISION = env.TAG_NAME.replaceFirst(/^v/, '')      // v2.3.0 → 2.3.0
  } else {
    env.REVISION = "2.3.0-SNAPSHOT"
  }
}
sh 'mvn -B -ntp -Drevision=$REVISION clean deploy'
```

Le Multibranch Pipeline doit découvrir les tags (*Behaviours → Discover tags*) et les **construire** : un tag découvert n'est pas buildé automatiquement sans *Build strategies → Tags* (plugin Basic Branch Build Strategies, en déclarant aussi les stratégies branches et PR). Puis `when { buildingTag() }` sur le stage de release.

Schéma : le parcours d'une version, du commit à la production



Comment le lire

1. Sur une branche de travail ou une PR, on construit et on teste, mais on ne publie rien.
2. Sur `main`, chaque build publie un **SNAPSHOT** : redéployable, horodaté, consommé par les projets aval pour l'intégration continue entre équipes.
3. Poser le tag `v2.3.0` déclenche (avec *Discover tags* et la build strategy *Tags*) le build qui publie la **release** : version fixe, immuable dans `maven-releases`.
4. Le pipeline CD ne déploie que des releases, tirées de Nexus, d'abord en recette puis en production.

À retenir
Le tag Git est la source de vérité de la version : la même release pointe toujours sur le même commit.

Déployer dans les référentiels Maven (2/3) : maven-release-plugin dans Jenkins

Quand l'utiliser

- Projets existants qui l'utilisent déjà ; bibliothèques publiées vers Maven Central avec ce workflow historique
- Release lancée **manuellement** par un job paramétré, pas à chaque commit

Prérequis

- `<scm>` renseigné dans le POM (`connection` , `developerConnection` , `tag`)
- Jenkins doit pouvoir **pousser** dans Git : credential SSH ou token avec droit d'écriture ; identité Git configurée sur l'agent
- Branche protégée : autoriser le compte technique Jenkins à pousser directement, ou utiliser une branche de release

```
<scm>
  <connection>scm:git:https://git.example.com/utopios/library-core.git</connection>
  <developerConnection>scm:git:git@git.example.com:utopios/library-core.git</developerConnection>
  <tag>HEAD</tag>
</scm>
<plugin>
  <artifactId>maven-release-plugin</artifactId>
  <version>3.1.1</version>
  <configuration>
    <tagNameFormat>v@{project.version}</tagNameFormat>
    <autoVersionSubmodules>true</autoVersionSubmodules>
    <releaseProfiles>release</releaseProfiles>
  </configuration>
</plugin>
```

Job de release paramétré

```
pipeline {
  agent { label 'linux' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }
  parameters {
    string(name: 'RELEASE_VERSION', defaultValue: '2.3.0')
    string(name: 'NEXT_VERSION',    defaultValue: '2.4.0-SNAPSHOT')
  }
  options { disableConcurrentBuilds() }
  stages {
    stage('Checkout') {
      steps {
        // checkout complet d'une branche nommée (pas de HEAD détaché)
        checkout scmGit(branches: [[name: '*/main']],
          extensions: [localBranch('main')],
          userRemoteConfigs: [[url: 'git@git.example.com:utopios/library-core.git',
                               credentialsId: 'git-ssh-key']])
        sh 'git config user.email jenkins@example.com && git config user.name "Jenkins"'
      }
    }
    stage('Prepare') {
      steps {
        sshagent(['git-ssh-key']) {
          configFileProvider([configFile(fileId: 'nexus-settings', variable: 'SETTINGS')]) {
            sh 'mvn -B -s $SETTINGS release:clean release:prepare -DreleaseVersion=$RELEASE_VERSION -DdevelopmentVersion=$NEXT_VERSION'
          }
        }
      }
      // rollback (pousse dans Git : restaure les POM, supprime le tag) : seulement si prepare
      // échoue ; après perform, la version est déjà publiée (et immuable) dans Nexus
      post { failure { sshagent(['git-ssh-key']) { sh 'mvn -B release:rollback' } } }
    }
    stage('Perform') {
      environment { NEXUS = credentials('nexus-deploy') }
      steps {
        sshagent(['git-ssh-key']) {
          configFileProvider([configFile(fileId: 'nexus-settings', variable: 'SETTINGS')]) {
            sh 'mvn -B -s $SETTINGS release:perform'
          }
        }
      }
    }
  }
}
```

Ce qui se passe

1. `prepare` : POM → 2.3.0, commit, tag `v2.3.0` , POM → 2.4.0-SNAPSHOT, commit, push
2. `perform` : checkout de `v2.3.0` dans `target/checkout` , `mvn deploy` (profil `release` : sources, javadoc, signature GPG si Central)

Déployer dans les référentiels Maven (3/3) : Nexus / Artifactory, politiques, promotion

Configuration côté Nexus Repository 3

Élément	Réglage
`maven-central` (proxy)	Cache de Central ; *Maximum component age* ; blocage possible des composants vulnérables (Nexus Firewall / IQ)
`maven-releases` (hosted)	Version policy **Release**, Deployment policy **Disable redeploy**
`maven-snapshots` (hosted)	Version policy **Snapshot**, Allow redeploy
`maven-public` (group)	releases + snapshots + central : URL unique pour le `<mirror>`
Rôle `nx-jenkins`	Privileges `nx-repository-view-maven2-maven-releases-add/edit/read` , `…-maven-snapshots-add/edit/read` , `…-maven-public-read` (Maven met aussi à jour `maven-metadata.xml`)
Cleanup policy	Snapshots : supprimer au-delà de 30 jours (« garder les N dernières versions » : Nexus Pro ; en Community Edition, tâche *Maven - Delete SNAPSHOT*, *Minimum snapshot count*) ; appliquée au dépôt snapshots + tâche planifiée *Admin – Cleanup repositories* + *Compact blob store*
Docker hosted	Registry Docker sur port dédié (8082) pour les images

Vérifier une publication

```
curl -s -u jenkins:*** \
 "http://nexus:8081/service/rest/v1/search?repository=maven-releases&name=library-core&version=2.3.0" \
 | jq '.items[].assets[].downloadUrl'
```

Promotion d'artefacts

- **Principe** : l'artefact ne change pas, c'est son *statut* qui évolue (candidat → validé → production)
- **Nexus 3** : pas de staging dans l'édition gratuite (Community Edition, ex-OSS, avec limites d'usage) ; promotion = **tags** (Pro) ou dépôts distincts (`maven-candidates` → `maven-releases` via API `/v1/staging/move` en Pro) ; en Community Edition, convention : release seulement depuis un tag Git validé
- **Artifactory** : propriétés sur l'artefact (`qa.status=passed`), dépôts par maturité et promotion de build (`jf rt build-promote`)
- **Jenkins** : le pipeline CD enregistre la promotion (`currentBuild.description` , fingerprint, *Keep this build forever*) et met à jour les propriétés côté dépôt

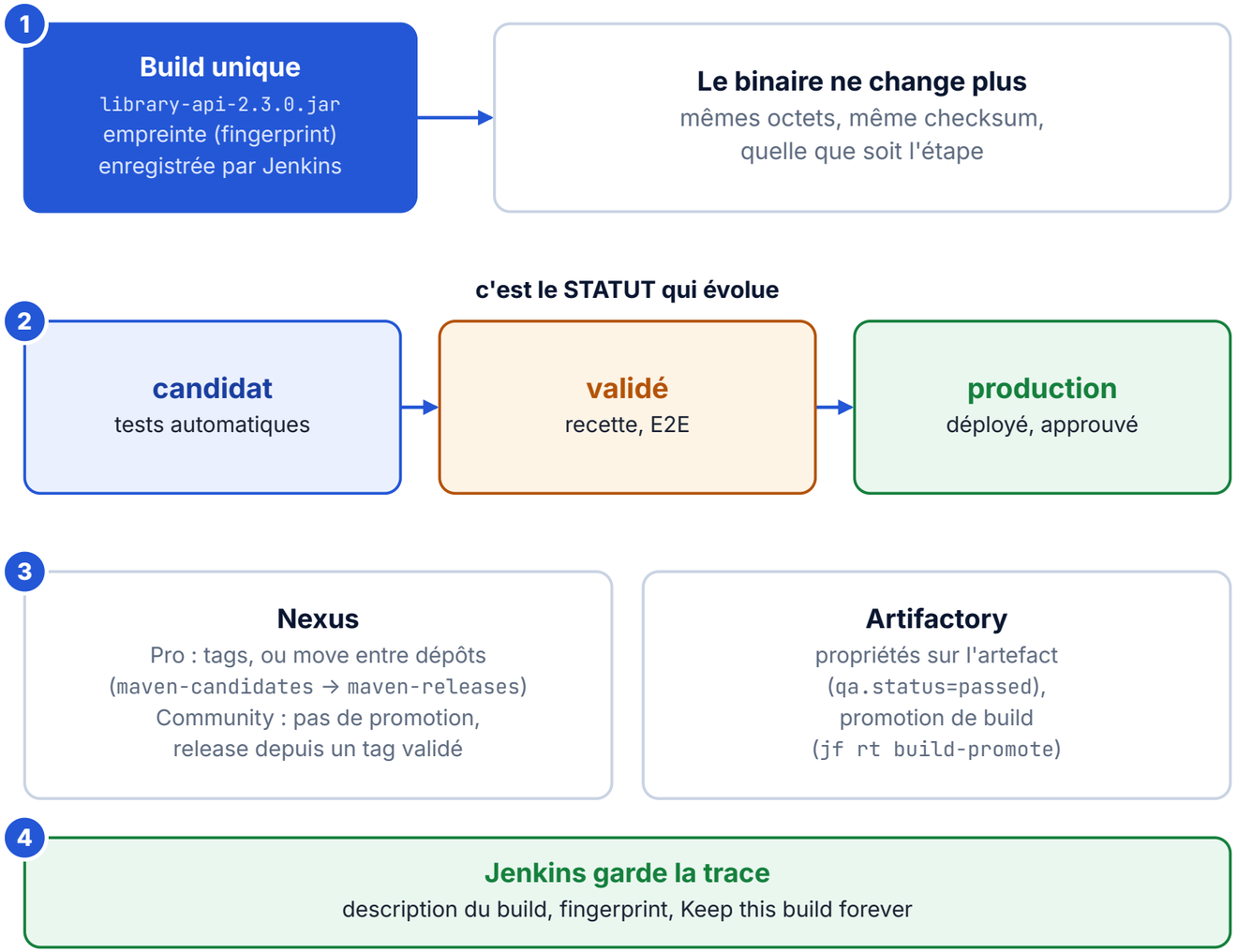
Artifactory depuis Jenkins (plugin + JFrog CLI)

```
stage('Publish') {
  steps {
    jf 'c add utopios --url=https://artifactory.example.com --access-token=$JF_TOKEN'
    jf 'mvn-config --repo-resolve-releases libs-release --repo-deploy-releases libs-release-local'
    jf 'mvn clean deploy -B'
    jf 'rt build-publish'                // build-info : traçabilité complète
  }
}
```

Traçabilité attendue

Depuis un artefact en production, retrouver : le build Jenkins, le commit Git, les tests exécutés, le Quality Gate, l'approbateur. C'est le fingerprint + le build-info qui l'assurent.

Schéma : la promotion d'un artefact



Comment le lire

1. L'artefact est construit **une seule fois** ; Jenkins enregistre son empreinte.
2. Il passe de *candidat* à *validé* puis à *production* sans être reconstruit : seul son statut change.
3. Le statut se matérialise selon l'outil : tags ou move entre dépôts dans Nexus Pro (rien en Community : release depuis un tag validé), propriétés et promotion de build dans Artifactory.
4. Jenkins garde la preuve : quel build, quel commit, qui a approuvé.

À retenir
Reconstruire pour la production, c'est livrer un binaire qui n'a pas été testé.

Piloter le déploiement d'applications (1/3) : le pipeline CD

Séparer CI et CD

- **Pipeline CI** (par dépôt) : construit, teste, publie **une version**
- **Pipeline CD** (par application) : prend une version **déjà publiée** et la déploie dans **un environnement**
- Le CD est **paramétré**, **idempotent**, **rejouable** (rollback = relancer avec la version précédente)
- Le CD ne construit jamais : pas de `mvn package`, uniquement du téléchargement et de l'installation

Entrées du pipeline CD

Paramètre	Exemple
`APP`	`library-api`, `library-portal`
`VERSION`	`2.3.0` (release) – jamais un SNAPSHOT en production
`TARGET_ENV`	`dev` / `recette` / `preprod` / `prod`
`CHANGE_REF`	Ticket ITSM / change, obligatoire en prod

Qui déclenche ?

- Le pipeline CI, en fin de build sur `main` → déploiement `dev` / `recette` automatique
- Un humain (*Build with Parameters*) ou un `input` → `preprod` / `prod`
- Un outil externe (ITSM, ChatOps) via l'API REST

Pipeline CD `library/deploy`

```
pipeline {
  agent { label 'linux && deploy' }          // agent dédié, outils de déploiement
  parameters {
    choice(name: 'APP', choices: ['library-api', 'library-portal'])
    string(name: 'VERSION', description: 'Version release publiée dans Nexus')
    choice(name: 'TARGET_ENV', choices: ['dev', 'recette', 'preprod', 'prod'])
    string(name: 'CHANGE_REF', defaultValue: '', description: 'Obligatoire pour prod')
  }
  options { disableConcurrentBuilds(); timestamps() }
  stages {
    stage('Validate') {
      steps { script {
        def apps = [
          'library-api'   : [gav: 'fr.utopios:library-api', type: 'jar', health: '/library-api/actuator/health'],
          'library-portal': [gav: 'fr.utopios:portal-ear',  type: 'ear', health: '/library/api/health']]
        env.GAV = apps[params.APP].gav; env.TYPE = apps[params.APP].type
        env.HEALTH = apps[params.APP].health
        if (params.VERSION.endsWith('-SNAPSHOT') && params.TARGET_ENV in ['preprod','prod'])
          error('Pas de SNAPSHOT en preprod/prod')
        if (params.TARGET_ENV == 'prod' && !params.CHANGE_REF)
          error('CHANGE_REF obligatoire en production')
        currentBuild.displayName = "#${BUILD_NUMBER} ${params.APP} ${params.VERSION} → ${params.TARGET_ENV}"
      } }
    }
    stage('Fetch artifact') {
      environment { NEXUS = credentials('nexus-deploy') }   // lu par le settings.xml
      steps {
        configFileProvider([configFile(fileId: 'nexus-settings', variable: 'SETTINGS')]) {
          sh '''
            mvn -B -q -s $SETTINGS dependency:copy \
              -Dartifact=${GAV}:${VERSION}:${TYPE} \
              -DoutputDirectory=dist -Dmdep.stripVersion=true
          '''
        }
      }
    }
    stage('Deploy') {
      options { lock(resource: "env-${params.TARGET_ENV}") }
      steps { sh 'ansible-playbook -i inventories/${TARGET_ENV} deploy.yml -e app=${APP} -e version=${VERSION}' }
    }
    stage('Smoke test') {
      steps { sh 'curl -sf https://${TARGET_ENV}.example.com${HEALTH}' }
    }
  }
  post {
    success { echo "Déployé ${params.APP} ${params.VERSION} sur ${params.TARGET_ENV}" }
    failure { emailext subject: "[CD] ÉCHEC ${params.APP} ${params.VERSION} → ${params.TARGET_ENV}",
                       body: env.BUILD_URL, to: 'ops@example.com' }
  }
}
```

Piloter le déploiement d'applications (2/3) : stratégies de déploiement

Recreate



interruption de service

Rolling update



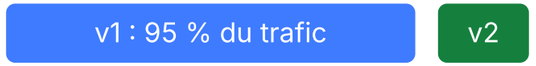
instance par instance, sans coupure

Blue / Green



bascule instantanée, rollback = rebasculer

Canary



5 % → 25 % → 100 % selon métriques

Feature flags : déployer le code sans l'activer ; activer par configuration, sans redéploiement.

Stratégie	Coupure	Rollback	Coût infra	Mise en œuvre
Recreate	Oui	Redéployer v1	×1	systemd, Tomcat, `strategy: Recreate`
Rolling	Non	Rolling inverse	×1 (+1)	K8s Deployment par défaut, Ansible `serial`
Blue/Green	Non	Instantané	×2	2 environnements + bascule LB/Ingress/DNS
Canary	Non	Couper le canari	×1 (+ε)	Gateway API (HTTPRoute pondérée), service mesh, Argo Rollouts

Ce que Jenkins pilote

- Rolling : `kubectl set image` + `rollout status` ; `kubectl rollout undo` en `post { failure }`
- Blue/Green : déployer sur l'environnement inactif → smoke tests → `input` → bascule (HTTPRoute de la Gateway API, cible du LB) → garder l'ancien 24 h
- Canary : boucle `for (pct in [5, 25, 50, 100])` → pondération → attente → lecture des métriques (Prometheus API) → `error()` si taux d'erreur > seuil

Prérequis applicatifs

- Compatibilité **N-1** du schéma de base de données (migrations Flyway additives, *expand/contract*)
- Endpoints `/health` (liveness) et `/ready` (readiness)
- Sessions externalisées ou stateless (sinon rolling et canary dégradent l'expérience)

Piloter le déploiement d'applications (3/3) : chaîne de promotion complète — le code

```
// library/promote - orchestrateur de promotion d'une release
pipeline {
  agent none
  parameters { string(name: 'VERSION', description: 'Release validée, ex. 2.3.0') }
  options { timestamps() }      // pas de disableConcurrentBuilds : le milestone ordonne les promotions

  stages {
    stage('Recette') {
      steps {
        build job: 'library/deploy', parameters: [
          string(name: 'APP', value: 'library-api'), string(name: 'VERSION', value: params.VERSION),
          string(name: 'TARGET_ENV', value: 'recette')]
        build job: 'library/e2e-tests', parameters: [string(name: 'BASE_URL', value: 'https://recette.example.com')]
      }
    }

    stage('Préproduction') {
      steps {
        build job: 'library/deploy', parameters: [
          string(name: 'APP', value: 'library-api'), string(name: 'VERSION', value: params.VERSION),
          string(name: 'TARGET_ENV', value: 'preprod')]
        build job: 'library/perf-tests', parameters: [string(name: 'BASE_URL', value: 'https://preprod.example.com')]
      }
    }

    stage('Go / No-Go production') {
      options { timeout(time: 3, unit: 'DAYS') }
      steps {
        script {
          def ok = input(message: "Déployer library-api ${params.VERSION} en production ?",
                         ok: 'GO', submitter: 'release-managers',
                         submitterParameter: 'APPROVER',
                         parameters: [string(name: 'CHANGE_REF', description: 'Référence du change ITSM')])
          env.CHANGE_REF = ok.CHANGE_REF; env.APPROVER = ok.APPROVER
        }
      }
    }

    stage('Production') {
      steps {
        milestone 300
        build job: 'library/deploy', parameters: [
          string(name: 'APP', value: 'library-api'), string(name: 'VERSION', value: params.VERSION),
          string(name: 'TARGET_ENV', value: 'prod'), string(name: 'CHANGE_REF', value: env.CHANGE_REF)]
      }
    }
  }

  post {
    success {
      script { currentBuild.description = "${params.VERSION} en prod - approuvé par ${env.APPROVER} (${env.CHANGE_REF})"
               currentBuild.keepLog = true }
      slackSend channel: '#releases', color: 'good', message: "library-api ${params.VERSION} déployé en production"
    }
    failure { slackSend channel: '#releases', color: 'danger', message: "Promotion ${params.VERSION} échouée : ${env.BUILD_URL}" }
  }
}
```

Piloter le déploiement d'applications (3/3) : chaîne de promotion complète — lecture

Lecture

- **Un seul artefact** traverse tous les environnements : `VERSION` est fixé à l'entrée
- Chaque environnement = déploiement + **tests adaptés** (E2E en recette, performance en préprod)
- `agent none` : l'orchestrateur ne consomme pas d'executor pendant 3 jours d'attente d'approbation
- `milestone` : si une promotion de 2.3.1 démarre pendant que 2.3.0 attend, l'ancienne est abandonnée quand la nouvelle passe le milestone (pas de déploiement d'une version obsolète). C'est pourquoi le job n'a pas `disableConcurrentBuilds()` : la 2.3.1 resterait sinon en file d'attente ; le `lock` du job `deploy` protège déjà chaque environnement
- La trace de la décision (approbateur, change) est conservée dans la description du build, gardé indéfiniment

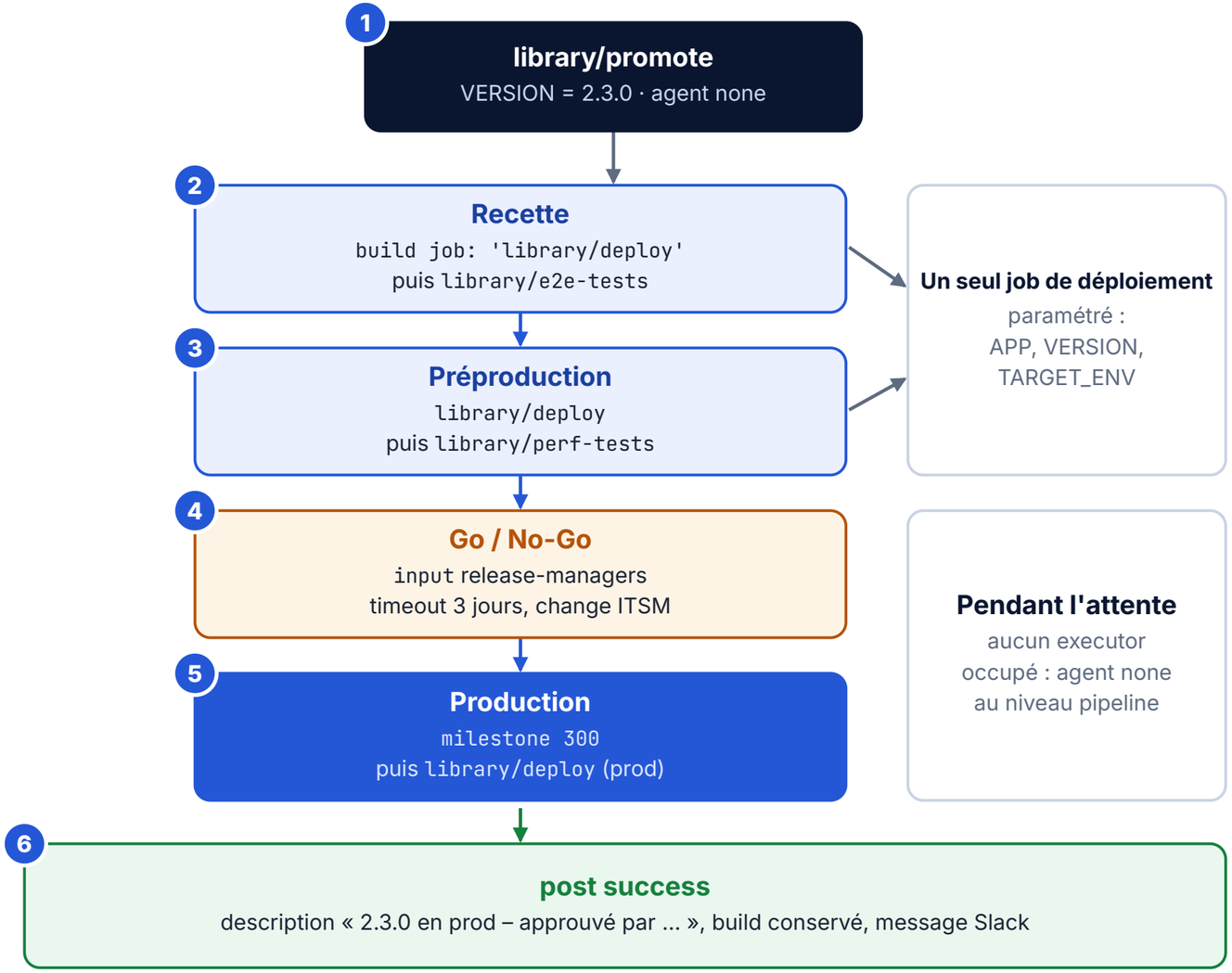
Variante GitOps

- Jenkins ne touche pas aux clusters : il **met à jour un dépôt Git de configuration** (`values-prod.yaml : image.tag: 2.3.0`) via une PR
- Argo CD / Flux appliquent le changement, Jenkins vérifie la synchronisation (`argocd app wait`)
- Avantage : le dépôt Git est l'historique des déploiements ; rollback = revert

Tableau de bord de promotion

Plugin **Delivery Pipeline** ou **Pipeline Graph View** : visualiser pour chaque version quel environnement elle a atteint.

Schéma : la chaîne de promotion d'une release



Comment le lire

1. L'orchestrateur reçoit **une** version validée : elle ne changera plus jusqu'à la production.
2. Recette : déploiement par le job générique, puis tests de bout en bout.
3. Préproduction : même déploiement, puis tests de performance.
4. Go / No-Go : un release manager approuve et saisit la référence du changement ; sans réponse en 3 jours, la promotion s'arrête.
5. Production : le `milestone` abandonne une promotion plus ancienne qui arriverait après une plus récente, puis le déploiement part.
6. La décision est tracée dans le build, conservé indéfiniment, et annoncée.

À retenir

Tous les environnements sont déployés par **le même job** : la seule différence est le paramètre `TARGET_ENV`.

05

Utilisation de Jenkins en cluster

Configuration des agents · Modes de démarrage Unix et Windows · Répartition des jobs entre agents · Bonnes pratiques de déploiement

Pourquoi distribuer les builds ?

Terminologie

- Historiquement **master** / **slaves** : *slave* → *agent* depuis Jenkins 2.0 (2016), *master* → *controller* depuis 2020, node « master » → **built-in node** en 2.307 (2021). Certains programmes de formation et identifiants techniques de plugins (`ssh-slaves`) gardent l'ancien terme : même notion.
- Le **built-in node** est le controller lui-même vu comme un node

Trois raisons de sortir les builds du controller

Capacité

Un controller a une JVM, un disque, un CPU. 50 builds Maven simultanés le saturent. Les agents se multiplient horizontalement.

Hétérogénéité

Des builds Windows (.NET, installeurs MSI), macOS (iOS, signature), Linux ARM, GPU : un agent par plate-forme, choisi par **label**.

Sécurité

Un build exécute du code arbitraire (celui du dépôt). Sur le controller, il a accès à `JENKINS_HOME` : credentials, configuration, tous les jobs. Sur un agent, il n'atteint que ce que le compte de l'agent peut atteindre sur cette machine (dont les workspaces et caches des autres jobs) : d'où les agents éphémères et la séparation des agents par niveau de confiance.

Ce qui reste sur le controller

- Interface web et API, ordonnancement (queue), configuration, plugins
- Stockage des résultats : logs, rapports JUnit, artefacts archivés, fingerprints
- Exécution du **Groovy des pipelines** (le CPS) – les steps `sh` / `bat` s'exécutent sur l'agent, mais la logique Groovy tourne sur le controller : éviter les boucles lourdes et les traitements de données en Groovy

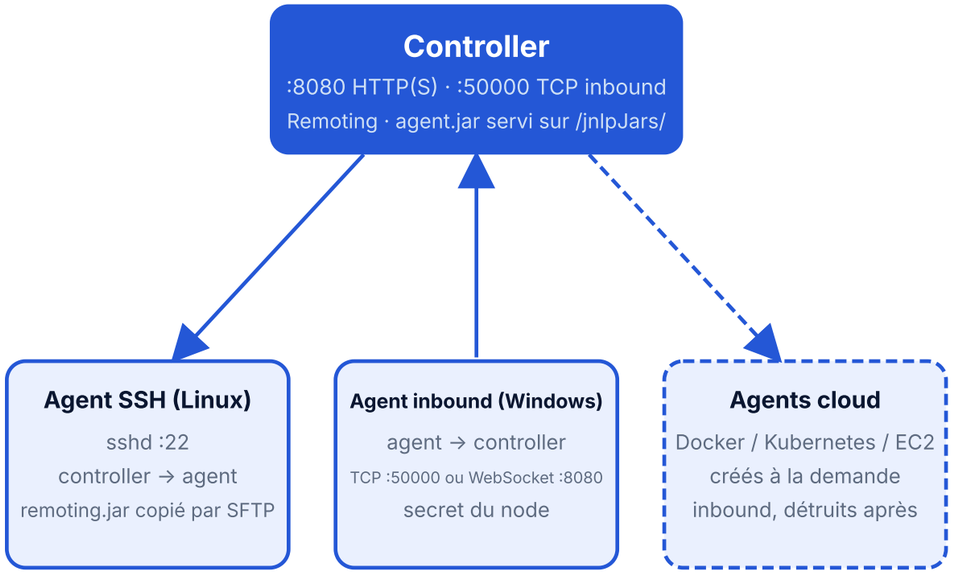
Ce qui va sur les agents

- Checkout des sources, compilation, tests, packaging, Docker, déploiement
- Les outils (JDK, Maven, Node, Docker, kubectl…) et leurs caches

Objectif de cette partie

- Monter un cluster : controller (0 executor) + agent Linux (SSH) + agent Windows (inbound) + agents éphémères Docker
- Router chaque job vers le bon agent et vérifier le comportement (file d'attente, agent hors ligne, labels inexistants)

Architecture et communication controller ↔ agents



Dans tous les cas : une JVM **`agent.jar`** tourne sur l'agent et exécute les commandes demandées par le controller

Le protocole Remoting

- Canal bidirectionnel chiffré entre le controller et `agent.jar` ; le controller envoie des *commandes* (lancer un process, lire/écrire un fichier), l'agent renvoie sorties et résultats
- **Outbound (SSH)** : le controller initie ; nécessite d'atteindre l'agent (port 22 ouvert vers lui)
- **Inbound** (ex-JNLP) : l'agent initie vers le controller ; traverse NAT et pare-feu sortants ; deux transports : **TCP** (port à activer : désactivé par défaut dans les paquets, 50000 dans l'image Docker officielle) ou **WebSocket** (réutilise le port HTTP(S) : recommandé derrière un reverse proxy)

Réglages côté controller

- *Manage Jenkins → Security → Agents* : *TCP port for inbound agents* : **Fixed** (50000) ou **Disable** si uniquement SSH/WebSocket
- *Agent → Controller* : protection **toujours active** depuis 2.326 (plus un réglage) – l'agent ne peut pas demander n'importe quel fichier au controller
- Version de Java : l'agent doit exécuter une version supportée par Jenkins (21 ou 25 en 2026), idéalement la même que le controller

Propriétés communes à tout node

Champ	Rôle
Name	Identifiant (`linux-1`) – apparaît dans `NODE_NAME`
Remote root directory	Dossier de travail : `workspace/` , `tools/` , `remoting/`
# of executors	Builds simultanés sur ce node
Labels	Liste séparée par des espaces
Usage	*Use this node as much as possible* | *Only build jobs with label expressions matching this node*
Availability	Toujours en ligne / à la demande / planning
Node Properties	Variables d'environnement, **Tool Locations** propres au node

Configuration des agents : préparer la machine

Prérequis sur tout agent

1. **Java** 21 ou 25 (JRE suffit pour `agent.jar` ; un JDK si on compile sans Tools)
2. Un **utilisateur dédié** non privilégié : `jenkins` (Linux) / `svc-jenkins` (Windows)
3. Un **répertoire racine** appartenant à cet utilisateur : `/home/jenkins/agent` ou `C:\jenkins`, sur un disque suffisant (workspaces, caches Maven/npm, images Docker)
4. **Git** installé (et dans le PATH)
5. Les outils non gérés par *Tools* : Docker, kubectl, Ansible…
6. Résolution DNS et accès réseau : vers le dépôt Git, Nexus, le registry ; vers le controller (inbound) ou depuis lui (SSH)
7. Horloge synchronisée (NTP) : Jenkins signale les décalages (*Clock Difference*)

Linux (Debian/Ubuntu)

```
sudo apt-get update && sudo apt-get install -y openjdk-21-jre-headless git
sudo useradd -m -s /bin/bash jenkins
sudo mkdir -p /home/jenkins/agent && sudo chown jenkins:jenkins /home/jenkins/agent
# Docker (optionnel) : le groupe docker donne des droits équivalents à root
# → réservé aux agents dédiés et jetables (ou Docker rootless)
sudo usermod -aG docker jenkins
```

Windows Server / 10 / 11

```
# PowerShell administrateur
winget install --id EclipseAdoptium.Temurin.21.JRE -e
# le MSI Temurin ne crée pas JAVA_HOME par défaut (requis par le service WinSW)
[Environment]::SetEnvironmentVariable('JAVA_HOME', (Resolve-Path 'C:\Program Files\Eclipse Adoptium\jre-21*').Path, 'Machine')
winget install --id Git.Git -e
New-Item -ItemType Directory -Path C:\jenkins
# Compte de service local ou de domaine
net user svc-jenkins "MotDePasseFort!" /add
icacls C:\jenkins /grant "svc-jenkins:(OI)(CI)F"
```

Vérifications avant d'enregistrer le node

```
# Sur l'agent
java -version                # 21.x ou 25.x
git --version
df -h /home/jenkins          # espace disque
# Depuis le controller (mode SSH)
ssh -i jenkins_agent_key jenkins@linux-1 'echo ok'     # clé créée au mode SSH
# Depuis l'agent (mode inbound)
curl -sI https://jenkins.example.com/login | head -1
```

Ne pas réutiliser le compte du controller

L'utilisateur de l'agent ne doit avoir aucun accès au `JENKINS_HOME` du controller, ni de droits sudo généraux. Les droits ponctuels (redémarrer un service lors d'un déploiement) passent par des règles `sudoers` ciblées.

Mode de démarrage Unix (1/2) : agent lancé par SSH

Principe

- Le controller se connecte en SSH, copie `remoting.jar` dans le *remote root* (SFTP, repli SCP), le lance avec `java -jar remoting.jar` et garde la session ouverte comme canal
- Plugin **SSH Build Agents** (installé par défaut)
- L'agent n'a rien à installer côté Jenkins : seulement Java et sshd

Étape 1 – clé SSH dédiée

```
# Sur le controller (ou n'importe où) : paire de clés sans passphrase (ou avec)
ssh-keygen -t ed25519 -C "jenkins-controller" -f jenkins_agent_key

# Sur chaque agent : autoriser la clé publique pour l'utilisateur jenkins
sudo -u jenkins mkdir -p /home/jenkins/.ssh
sudo -u jenkins tee -a /home/jenkins/.ssh/authorized_keys < jenkins_agent_key.pub
sudo chmod 700 /home/jenkins/.ssh && sudo chmod 600 /home/jenkins/.ssh/authorized_keys
```

Étape 2 – credential dans Jenkins

- *Manage Jenkins* → *Credentials* → *System* → *Global* → *Add* : type **SSH Username with private key**, ID `agent-ssh-key`, username `jenkins`, clé privée collée (*Enter directly*)
- Scope **System** suffit : ce credential ne sert qu'au controller

Étape 3 – créer le node

Champ	Valeur
Name	`linux-1`
Remote root directory	`/home/jenkins/agent`
Labels	`linux java21 maven docker`
Usage	Only build jobs with label expressions matching this node
Launch method	**Launch agents via SSH**
Host	`linux-1.example.com`
Credentials	`agent-ssh-key`
Host Key Verification Strategy	**Known hosts file** (production) ou *Manually trusted key* (première connexion) – jamais *Non verifying* en prod
Advanced → JavaPath	`/usr/bin/java` si plusieurs JVM
Advanced → JVM Options	`-Xmx512m` (pour agent.jar, pas pour les builds)
Availability	Keep this agent online as much as possible

Étape 4 – vérifier

- Page du node → **Log** : `[SSH] Starting agent process… Agent successfully connected and online`
- Dashboard → *Build Executor Status* : `linux-1` avec ses executors
- Job de test : `agent { label 'linux-1' }` + `sh 'hostname; whoami; pwd'`

Erreurs fréquentes

Permission denied → droits de `.ssh` ; *java: not found* → PATH de la session non interactive, renseigner JavaPath ; *Host key verification failed* → stratégie de vérification.

Mode de démarrage Unix (2/2) : agent inbound en service systemd

Quand préférer l'inbound sous Linux

- L'agent est derrière un NAT ou un pare-feu qui bloque le SSH entrant (poste de dev, site distant, cloud sans IP publique)
- Politique de sécurité : seules les connexions *sortantes* depuis la zone de l'agent sont autorisées
- L'agent doit démarrer tout seul au boot et se reconnecter après une coupure

Côté Jenkins

- Node `linux-2`, Launch method **Launch agent by connecting it to the controller**
- Options : *Use WebSocket* (recommandé derrière un reverse proxy HTTPS) ; sinon activer le port TCP inbound (*Security → Agents → Fixed : 50000*)
- Après Save, la page du node affiche la commande de lancement et le **secret** ; le récupérer dans un fichier protégé sur l'agent

```
# Sur l'agent linux-2
sudo -u jenkins mkdir -p /home/jenkins/agent
sudo -u jenkins curl -sSLo /home/jenkins/agent/agent.jar \
     https://jenkins.example.com/jnlpJars/agent.jar
echo "<secret affiché par Jenkins>" | sudo -u jenkins tee /home/jenkins/agent/secret-file >/dev/null
sudo chmod 600 /home/jenkins/agent/secret-file

# Test manuel
sudo -u jenkins java -jar /home/jenkins/agent/agent.jar \
   -url https://jenkins.example.com/ -name linux-2 \
   -secret @/home/jenkins/agent/secret-file -webSocket -workDir /home/jenkins/agent
```

Unité systemd

```
# /etc/systemd/system/jenkins-agent.service
[Unit]
Description=Jenkins inbound agent (linux-2)
After=network-online.target
Wants=network-online.target

[Service]
User=jenkins
Group=jenkins
WorkingDirectory=/home/jenkins/agent
ExecStart=/usr/bin/java -Xmx512m -jar /home/jenkins/agent/agent.jar \
  -url https://jenkins.example.com/ \
  -name linux-2 \
  -secret @/home/jenkins/agent/secret-file \
  -webSocket \
  -workDir /home/jenkins/agent
Restart=always
RestartSec=10
# Durcissement
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now jenkins-agent
sudo systemctl status jenkins-agent
journalctl -u jenkins-agent -f        # INFO: Connected
```

Comportement

`Restart=always` : si le controller redémarre (mise à jour), l'agent retente toutes les 10 s et revient en ligne sans intervention. `agent.jar` est à re-télécharger à chaque mise à jour du controller (ajouter un `ExecStartPre` avec le `curl`).

Schéma : SSH ou inbound, qui ouvre la connexion ?



Comment le lire

1. En **SSH**, le controller se connecte à l'agent avec un credential de clé privée.
2. Il copie `remoting.jar` sur l'agent et le lance : l'agent n'a besoin que de sshd et de Java.
3. En **inbound**, c'est l'agent qui se connecte : un service lance `agent.jar` avec le nom et le secret du node, et se reconnecte seul après une coupure.
4. Il joint le controller en WebSocket sur le port HTTPS (ou en TCP sur 50000) : seule une connexion **sortante** est nécessaire côté agent.

À retenir
Une fois connectés, les deux modes sont identiques pour les builds : le choix dépend du réseau.

Mode de démarrage Windows (1/2) : agent inbound en service Windows

Pourquoi inbound sous Windows

- Pas de sshd par défaut (OpenSSH Server est installable, mais le lancement SSH d'agents Windows reste fragile : chemins, droits, PowerShell vs cmd)
- Le mode inbound + service Windows est le mode **standard** et supporté

Côté Jenkins

- Node `windows-1`, Remote root `C:\jenkins`, labels `windows dotnet msbuild`, Launch method *Launch agent by connecting it to the controller*, *Use WebSocket*
- Copier le secret affiché

Installation en service avec WinSW

1. Télécharger `WinSW-x64.exe` (github.com/winsw/winsw, releases) → `C:\jenkins\jenkins-agent.exe`
2. Télécharger `agent.jar` : `https://jenkins.example.com/jnlpJars/agent.jar` → `C:\jenkins\agent.jar`
3. Créer `C:\jenkins\jenkins-agent.xml` (ci-contre)
4. Installer et démarrer le service

```
# PowerShell administrateur
cd C:\jenkins
.\jenkins-agent.exe install
.\jenkins-agent.exe start
.\jenkins-agent.exe status      # Started
Get-Content .\jenkins-agent.err.log -Tail 20   # INFO: Connected (java.util.logging → stderr)
```

`C:\jenkins\jenkins-agent.xml`

```
<service>
  <id>jenkins-agent</id>
  <name>Jenkins Agent (windows-1)</name>
  <description>Agent Jenkins inbound</description>
  <executable>%JAVA_HOME%\bin\java.exe</executable>   <!— JAVA_HOME système —>
  <arguments>-Xmx512m -jar "C:\jenkins\agent.jar"
    -url https://jenkins.example.com/
    -name windows-1
    -secret @C:\jenkins\secret-file
    -webSocket
    -workDir "C:\jenkins"</arguments>
  <workingdirectory>C:\jenkins</workingdirectory>
  <logmode>rotate</logmode>
  <onfailure action="restart" delay="10 sec"/>
  <serviceaccount>
    <username>.\svc-jenkins</username>
    <password>MotDePasseFort!</password>
    <allowservicelogon>true</allowservicelogon>
  </serviceaccount>
</service>
```

Alternatives

- **Installeur Windows** : depuis la page du node, lancer l'agent une première fois en session interactive (`java -jar agent.jar …`) puis menu *File* → *Install as a service* (si disponible dans la version) – génère la même configuration WinSW
- **Tâche planifiée** au démarrage (*Task Scheduler*, « Run whether user is logged on or not ») si l'installation d'un service est interdite
- **Agent Docker Windows** : image `jenkins/inbound-agent:windowsservercore-ltsc2022`

Session 0
Un service Windows tourne sans bureau : les tests UI (Selenium avec navigateur visible, WPF) nécessitent une session interactive ou un mode headless.

Mode de démarrage Windows (2/2) : particularités des builds Windows

Steps et shells

Step	Interpréteur	Remarque
`bat 'dir'`	cmd.exe	Standard Windows ; `%VAR%` , `^` pour continuer une ligne
`powershell 'Get-ChildItem'`	Windows PowerShell 5.1	Préférer pour la logique ; `$env:VAR`
`pwsh '…'`	PowerShell 7+	Multi-plateforme
`sh '…'`	sh (Git Bash si dans le PATH)	Fonctionne si `sh.exe` trouvé ; sinon erreur

```
pipeline {
  agent { label 'windows' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }     // Tool Locations spécifiques au node si besoin
  stages {
    stage('Build') {
      steps {
        bat 'mvn -B clean verify'
        powershell '''
          $v = Get-Content VERSION.txt
          Write-Host "Version $v"
          Compress-Archive -Path target\\*.jar -DestinationPath dist\\app-$v.zip -Force
        '''
      }
    }
  }
}
```

Pièges classiques

- **Chemins** : `\` dans les chaînes Groovy doit être doublé (`'C:\\jenkins'`) ; longueur max 260 caractères (*LongPathsEnabled* + `git config --system core.longpaths true` , insuffisant pour cmd et certains outils ; le plus fiable : un remote root court, `C:\j`) ; `node_modules` et Maven multi-modules dépassent vite
- **Fins de ligne** : `git config --global core.autocrlf false` sur l'agent pour que les scripts Bash restent exécutables ; `.gitattributes` dans le dépôt
- **Verrous de fichiers** : un processus qui garde un fichier ouvert fait échouer `cleanWs()` → tuer les processus résiduels du build (filtrés sur leur ligne de commande, jamais tous les `java.exe` : l'agent en est un) ou agents éphémères
- **Encodage** de la console : `chcp 65001` et/ou `-Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8` (JDK 19+ ; `file.encoding` vaut déjà UTF-8 depuis le JDK 18)
- **Antivirus** : exclure le remote root du scan temps réel (gain notable sur npm/Maven, à mesurer)
- **Profil** : sous LocalSystem, `%USERPROFILE%` = `C:\Windows\system32\config\systemprofile` ; sous un compte dédié (`svc-jenkins`), son profil est chargé. Dans les deux cas, placer les caches près du remote root : `MAVEN_OPTS=-Dmaven.repo.local=C:\jenkins\.m2` , `npm config set cache C:\jenkins\.npm`

Tool Locations par node

Node → Configure → *Node Properties* → *Tool Locations* : indiquer que `maven-3.9` se trouve en `C:\tools\apache-maven-3.9.9` sur ce node. Le Jenkinsfile reste identique entre Linux et Windows.

Agents éphémères (1/2) : Docker plugin

Principe

- Un hôte Docker (ou plusieurs) est déclaré comme **Cloud** ; pour chaque build demandant un label géré, Jenkins crée un conteneur agent, y exécute le build, puis le détruit
- Workspace neuf à chaque fois ; version d'outils figée par l'image ; montée en charge limitée uniquement par l'hôte
- Image de base : `jenkins/inbound-agent:jdk21` (ou `jenkins/agent`) – ou une image maison `FROM jenkins/inbound-agent` + Maven + Docker CLI

Configuration (*Manage Jenkins → Clouds → New cloud → Docker*)

Champ	Valeur
Docker Host URI	`tcp://docker-host.example.com:2376` (TLS, credential certificat) ou `unix:///var/run/docker.sock` si local
Enabled	✔ ; *Container Cap* : 10
Docker Agent template → Labels	`docker-maven`
Docker Image	`registry.example.com/ci/agent-maven:21`
Remote File System Root	`/home/jenkins/agent`
Connect method	**Connect with JNLP** (inbound) – recommandé ; ou *Attach Docker container*
Volumes / Mounts	`type=volume,src=m2cache,dst=/home/jenkins/.m2`
Pull strategy	*Pull once and update latest*
Remove volumes / Instance cap	✔ / 5

Image d'agent maison

```
# ci/agent-maven/Dockerfile
FROM jenkins/inbound-agent:jdk21
USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
      maven docker.io curl jq && rm -rf /var/lib/apt/lists/*
USER jenkins
ENV MAVEN_OPTS="-Dmaven.repo.local=/home/jenkins/.m2/repository"
```

Utilisation

```
pipeline {
  agent { label 'docker-maven' }        // Jenkins provisionne un conteneur
  stages {
    stage('Build') { steps { sh 'mvn -B -ntp clean verify' } }
  }
}
```

Alternative : `agent { docker }` (plugin Docker Pipeline)

- Ici c'est un agent **persistant** doté de Docker qui lance un conteneur *pour les steps*, en montant son workspace ; pas de provisioning de node
- Plus simple à mettre en place, moins isolé (le conteneur tourne sur un agent qui existe déjà)

Observer

Build Executor Status : un node `docker-maven-xxxxx` apparaît pendant le build puis disparaît. Logs du cloud : *Manage Jenkins → System Log*, logger `com.nirima.jenkins.plugins.docker`.

Agents éphémères (2/2) : Kubernetes plugin

Principe

- Chaque build = un **pod** créé dans un namespace dédié (`jenkins-agents`), détruit à la fin
- Le pod contient toujours un conteneur `jnlp` (l'agent inbound) + les conteneurs d'outils déclarés
- Le controller peut lui-même tourner dans le cluster (Helm chart `jenkinsci/jenkins`) ou à l'extérieur avec un kubeconfig

Configuration (*Clouds → New cloud → Kubernetes*)

Champ	Valeur
Kubernetes URL	Vide si le controller est dans le cluster ; sinon URL de l'API + credential kubeconfig
Kubernetes Namespace	`jenkins-agents`
Jenkins URL	`http://jenkins.jenkins.svc.cluster.local:8080`
Jenkins tunnel	`jenkins-agent.jenkins.svc.cluster.local:50000` (si TCP) ; vide avec WebSocket
Concurrency Limit	20
Pod Templates	Optionnel : templates nommés avec label ; sinon YAML dans le Jenkinsfile

RBAC (Role de référence du plugin), limité au namespace des agents : `pods` et `pods/exec` : create, delete, get, list, patch, update, watch ; `pods/log` : get, list, watch ; `events` : watch ; `secrets` : get.

Jenkinsfile

```
pipeline {
  agent {
    kubernetes {
      yaml '''
        apiVersion: v1
        kind: Pod
        spec:
          containers:
          - name: maven
            image: maven:3.9-eclipse-temurin-21
            command: ["sleep"]
            args: ["infinity"]
            resources:
              requests: { cpu: "1", memory: "2Gi" }
              limits:   { cpu: "2", memory: "3Gi" }
            volumeMounts:
            - { name: m2, mountPath: /root/.m2 }
          - name: kaniko                  # projet Google archivé (2025) : fork maintenu
            image: gcr.io/kaniko-project/executor:debug   # ou BuildKit rootless
            command: ["sleep"]
            args: ["infinity"]
          volumes:
          - name: m2
            persistentVolumeClaim: { claimName: maven-cache }
      '''
      defaultContainer 'maven'
    }
  }
  stages {
    stage('Build') { steps { sh 'mvn -B -ntp clean verify' } }
    stage('Image') {
      steps {
        container('kaniko') {       // build d'image sans démon Docker
          sh '/kaniko/executor --context $WORKSPACE --destination registry.example.com/utopios/library-api:$BUILD_NUMBER'
        }
      }
    }
  }
}
```

Bonnes pratiques

Toujours fixer `resources` (sinon le scheduler place mal les pods et les builds sont OOM-killed) ; un PVC ou un dépôt proxy proche pour les caches ; **Buildah** ou **BuildKit** rootless (Kaniko : projet Google archivé, à remplacer ou prendre via un fork maintenu) plutôt que de monter la socket Docker.

Schéma : la vie d'un agent Kubernetes



Comment le lire

1. Le build demande un agent décrit par un pod (YAML dans le Jenkinsfile ou pod template) et attend en queue.
2. Le plugin crée le pod dans le namespace des agents, dans la limite de concurrence configurée.
3. Le conteneur `jnlp` est l'agent : il se connecte au controller comme un agent inbound.
4. Les steps s'exécutent dans le conteneur par défaut (`maven`) ou dans celui désigné par `container('…')` ; tous voient le même workspace.
5. À la fin du build, le pod est supprimé : chaque build repart d'un environnement neuf.

À retenir

Toujours fixer `resources` pour chaque conteneur : sans elles, le scheduler place mal les pods et les builds sont tués faute de mémoire.

Répartition des jobs entre agents (1/3) : labels et expressions

Labels

- Chaque node porte des labels (son nom en est un implicitement) : `linux`, `windows`, `java21`, `docker`, `gpu`, `deploy`, `prod-access`
- Un job demande une **expression de label** ; Jenkins ne l'exécute que sur un node qui la satisfait
- Convention : des labels de **capacité** (`docker`, `maven`), de **plate-forme** (`linux`, `windows`, `arm64`) et de **zone** (`dmz`, `prod-access`) ; éviter de cibler un node par son nom

Expressions

Expression	Signification
`linux`	Tout node ayant le label linux
`linux && docker`	Les deux labels
`linux || windows`	L'un ou l'autre
`linux && !gpu`	Linux sans GPU
`(linux || macos) && java21`	Parenthèses
`linux -> docker`	Implication (rare)
vide (Freestyle) / `agent any` (Declarative)	N'importe quel node en mode *Use this node as much as possible*

Dans les jobs

- **Freestyle** : *General → Restrict where this project can be run → Label Expression*
- **Pipeline** : `agent { label 'linux && docker' }` au niveau pipeline ou stage ; `node('windows') { }` en scripté

Le mode « Usage » du node

- **Use this node as much as possible** : accepte aussi les jobs sans label → un job `agent any` peut atterrir n'importe où (y compris sur Windows !)
- **Only build jobs with label expressions matching this node** : n'accepte que les jobs qui le demandent explicitement → recommandé pour tous les nodes spécialisés et pour le controller

Comportements à montrer

Label inexistant

`agent { label 'macos' }` sans node macos → le build reste dans la *Build Queue* avec « There are no nodes with the label 'macos' ». Il ne tombe pas en erreur : il attend. Avec un agent au niveau pipeline, le `timeout` global ne démarre qu'une fois l'agent obtenu et ne couvre pas cette attente ; pour la couvrir : `timeout` dans les options du stage, ou `agent none` + agents par stage.

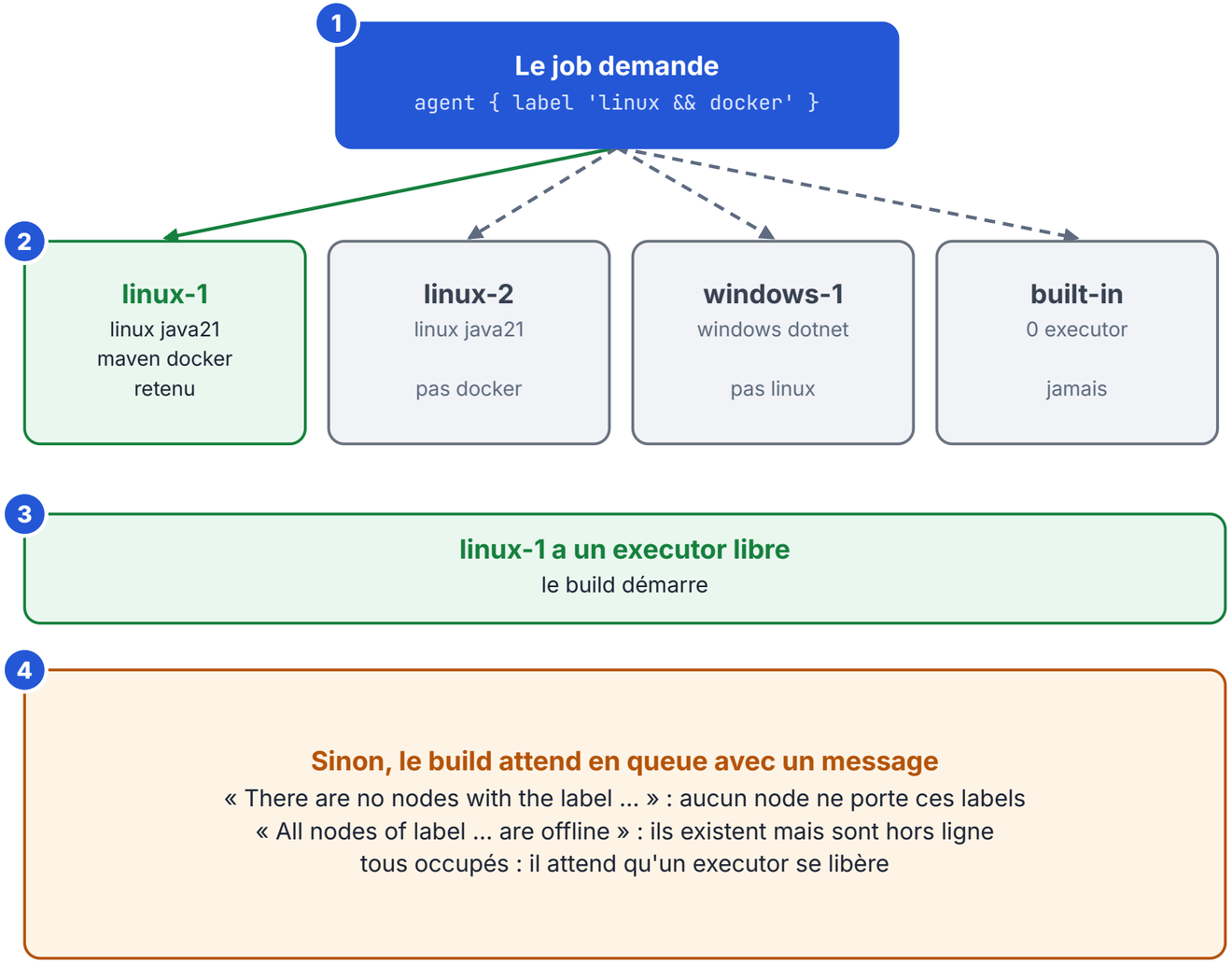
Agent hors ligne

Si tous les nodes `windows` sont hors ligne : « All nodes of label 'windows' are offline ». Le build démarre dès qu'un node revient.

Tous les executors occupés

File d'attente FIFO (avec priorités possibles via plugin Priority Sorter) ; en Freestyle, l'attente en queue n'entre pas dans la durée du build ; en Pipeline, l'attente d'un agent pendant le build y entre.

Schéma : comment Jenkins choisit un node



Comment le lire

1. Le job exprime un **besoin** sous forme d'expression de labels, pas un nom de machine.
2. Jenkins écarte les nodes qui ne satisfont pas l'expression : ici, seul `linux-1` porte `linux` **et** `docker`.
3. Parmi les nodes retenus, il prend un executor libre et lance le build.
4. S'il n'y en a pas, le build **attend** en queue sans échouer ; le message de la queue dit pourquoi.

À retenir

Mode *Only build jobs with label expressions matching this node* sur les nodes spécialisés : un job `agent any` n'y atterrit pas par hasard.

Répartition des jobs entre agents (2/3) : pipelines multi-agents et parallélisme

Un agent par stage

```
pipeline {
  agent none
  stages {
    stage('Build Linux')   { agent { label 'linux && maven' }
                             steps { sh 'mvn -B package'; stash name: 'jar', includes: 'target/*.jar' } }
    stage('Test Windows')  { agent { label 'windows' }
                             steps { unstash 'jar'; bat 'java -jar target\\library-api.jar --smoke' } }
    stage('Deploy')        { agent { label 'deploy && prod-access' }
                             steps { unstash 'jar'; sh './deploy.sh prod' } }
  }
}
```

- Chaque stage obtient son propre workspace sur son node : `stash` / `unstash` ou dépôt d'artefacts pour transférer
- Entre deux stages, le pipeline ne tient aucun executor

Matrice multi-plateformes

```
stage('Tests multi-OS') {
  matrix {
    axes {
      axis { name 'PLATFORM'; values 'linux', 'windows' }
      axis { name 'JDK';      values 'jdk17', 'jdk21' }
    }
    excludes { exclude { axis { name 'PLATFORM'; values 'windows' } axis { name 'JDK'; values 'jdk17' } } }
    agent { label "${PLATFORM}" }
    tools { jdk "${JDK}" }
    stages {
      stage('Test') {
        steps { script { isUnix() ? sh('mvn -B test') : bat('mvn -B test') } }
        post { always { junit 'target/surefire-reports/*.xml' } }
      }
    }
  }
}
```

Parallélisme sur plusieurs agents

```
stage('Tests') {
  parallel {
    stage('Unit')        { agent { label 'linux' }; steps { sh 'mvn -B test' } }
    stage('Integration') { agent { label 'linux && docker' }; steps { sh 'mvn -B verify -P it' } }
    stage('Windows')     { agent { label 'windows' }; steps { bat 'mvn -B test' } }
  }
}
```

- Les trois branches d'un même build s'exécutent simultanément si trois executors sont libres ; sinon elles attendent chacune leur tour
- `failFast true` : interrompre les autres branches dès qu'une échoue

Contrôler la concurrence

Besoin	Solution
Un seul build de ce job à la fois	`options { disableConcurrentBuilds() }`
Un seul déploiement par environnement, tous jobs confondus	plugin Lockable Resources : `lock('env-recette')`
Limiter N builds d'une catégorie sur le cluster	plugin Throttle Concurrent Builds (`throttle(['heavy'])`)
Abandonner les builds dépassés	`milestone`
Rester sur le même node entre deux stages	Un seul `agent` au niveau pipeline, ou `reuseNode true` pour `agent { docker }`

Ordonnanceur

Le load balancer par défaut (*consistent hash* sur le nom du job) tend à renvoyer un job vers les mêmes nodes (workspace chaud, caches) ; plugin Least Load pour répartir par charge. Avec des agents éphémères, elle est sans objet : c'est le cache partagé (volume, PVC, Nexus) qui compte.

Répartition des jobs entre agents (3/3) : diagnostiquer la file d'attente

Lire la Build Queue

- Survoler un élément de la queue : la raison d'attente est affichée
- Messages typiques :
  - *Waiting for next available executor on 'linux'* → saturation
  - *There are no nodes with the label 'x'* → faute de frappe ou node supprimé
  - *'linux-1' is offline* → agent déconnecté
  - *'linux-1' is reserved for jobs with matching label expression* → job sans label et node en mode « Only build jobs with label… »
  - *Build #42 is already in progress (ETA: 3 min)* → `disableConcurrentBuilds`
  - `lock()` en Pipeline : l'attente est dans la console du build et sur la page *Lockable Resources*, pas dans la queue
- API : `GET /queue/api/json` → champ `why` de chaque item

Pourquoi un agent est hors ligne

- Page du node → bandeau rouge avec la cause : déconnexion réseau, **Node monitors** (disque plein, espace temp, temps de réponse) qui mettent le node *offline temporairement* ; *Free Swap Space*, *Clock Difference* et *Architecture* ne sont qu'informatifs
- *Manage Jenkins → Nodes* : colonnes Free Disk Space, Free Temp Space, Clock Difference, Response Time – seuils configurables (*Preventive Node Monitoring*)
- Log du node pour les erreurs de connexion ; `journalctl -u jenkins-agent` côté agent

Statistiques de charge

- *Manage Jenkins → Load Statistics* (global) et onglet *Load Statistics* par node / par label : executors totaux, occupés, longueur de la queue, sur trois échelles : Short (pas de 10 s, ~6 h), Medium (1 min, ~2 j), Long (1 h, ~8 semaines)
- Une queue chroniquement > 0 sur un label = sous-capacité → ajouter des agents ou passer en éphémère élastique
- Des executors chroniquement inactifs = sur-capacité → réduire (coût cloud)

Script Console : état du cluster en un coup d'œil

```
// Manage Jenkins → Script Console
Jenkins.get().computers.each { c →
  println String.format("%-20s online=%-5s exec=%d/%d labels=%s",
    c.name ?: 'built-in', c.online, c.countBusy(), c.numExecutors,
    c.node?.labelString)
  if (!c.online) println "   → " + c.offlineCauseReason
}
println "Queue : " + Jenkins.get().queue.items.collect { it.task.fullDisplayName + ' (' + it.why + ')' }
```

Actions correctives courantes

- Mettre un node **temporairement hors ligne** avant une maintenance (bouton *Mark this node temporarily offline* + raison) : les builds en cours finissent, aucun nouveau n'arrive
- Augmenter les executors d'un node si CPU/RAM le permettent (règle : 1 executor par 2 vCPU et 2–4 Go pour des builds Maven)
- Déconnecter / reconnecter (*Disconnect → Launch agent*) après un changement de configuration

Bonnes pratiques de déploiement (1/3) : topologie de la plateforme

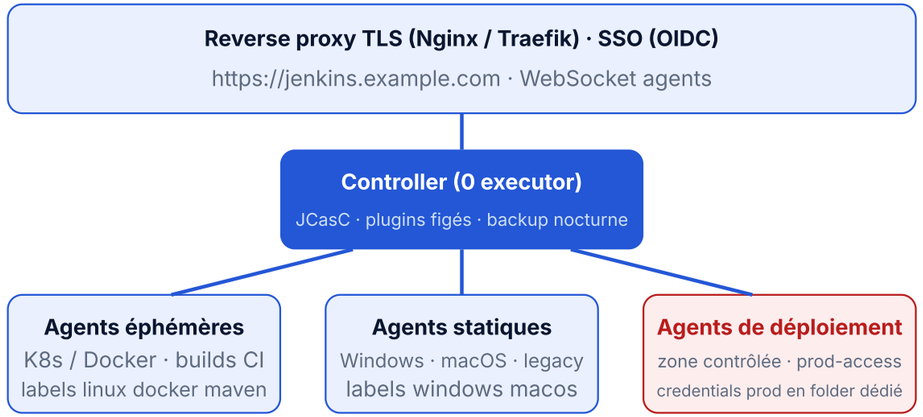
Dimensionner le controller

- **0 executor** sur le built-in node ; labels vides ; usage « Only build jobs with label… »
- JVM : `-Xmx` = 4 à 8 Go pour quelques centaines de jobs ; G1 par défaut (Java 21) ; `-XX:+UseStringDeduplication` ; surveiller le heap (partie 6)
- Disque `JENKINS_HOME` : SSD, volume dédié, 100 Go+, surveillé
- Un controller par **domaine de confiance** ou par grande organisation plutôt qu'un monolithe de 5 000 jobs (CloudBees CI propose la gestion multi-controllers ; en OSS : plusieurs instances + JCasC commun)

Dimensionner les agents

Profil	Recommandation
Build Maven/Gradle Java	2 vCPU, 4 Go par executor ; cache `~/.m2` local ou PVC
Front npm	2 vCPU, 4 Go ; cache npm ; attention aux inodes
Images Docker	Hôte dédié avec `docker system prune` planifié, ou Buildah / BuildKit rootless
Tests E2E navigateur	4 vCPU, 8 Go, `/dev/shm` suffisant
Déploiement	Agent dédié `deploy`, peu de ressources, accès réseau aux environnements, outils (kubectl, ansible) à jour

Architecture cible



- Séparer physiquement les agents qui construisent du code tiers (PR de forks, dépendances) de ceux qui touchent la production
- Les credentials de production ne sont visibles que des jobs du folder `prod/`, exécutés uniquement sur les agents `prod-access`

Bonnes pratiques de déploiement (2/3) : sécurité du cluster

Jamais de build sur le controller

Un `sh 'cat $JENKINS_HOME/credentials.xml'` dans un Jenkinsfile de PR suffit à exfiltrer tous les secrets (chiffrés avec une clé qui est… dans `secrets/`). 0 executor, point.

Agent → Controller Security

Toujours actif et non désactivable depuis Jenkins 2.326. Empêche un agent compromis de lire/écrire des fichiers arbitraires du controller.

Comptes et droits minimaux sur les agents

Utilisateur dédié sans sudo général ; pas de clé SSH personnelle ; pas de kubeconfig admin ; les secrets arrivent via les credentials Jenkins au moment du build, jamais stockés sur l'agent.

Isolation des builds non fiables

Les PR venant de forks externes (*Discover pull requests from forks → Trust : Nobody*) : le Jenkinsfile utilisé est celui de la branche cible, pas celui de la PR. Mais le code de la PR (pom.xml, scripts, tests) s'exécute quand même : ce Jenkinsfile ne doit injecter aucun credential pour les PR de forks (condition sur `CHANGE_FORK`), et ces builds tournent sur des agents éphémères sans secret.

Réseau

Agents inbound en WebSocket via le reverse proxy TLS : un seul port exposé (443), pas de 50000 ouvert. Si TCP : restreindre 50000 aux IP des agents.

Secrets des agents inbound

Le secret du node permet de se faire passer pour l'agent : fichier `600`, jamais dans une image Docker, jamais dans un dépôt. Le secret est dérivé du nom du node : un node recréé sous le même nom retrouve le même secret. En cas de fuite : renommer le node, ou changer la clé `secrets/jenkins.slaves.JnlpSlaveAgentProtocol.secret` (change le secret de tous les agents inbound).

Images d'agents maîtrisées

Construites en interne depuis `jenkins/inbound-agent`, scannées (Trivy), poussées dans le registry interne, taguées par version. Pas de `latest` en production.

Audit

Plugin **Audit Trail** (qui a lancé quoi, modifié quelle config) ; logs des agents centralisés ; revue trimestrielle des nodes et de leurs labels.

Docker socket

Monter `/var/run/docker.sock` dans un agent = root sur l'hôte. Acceptable sur un hôte dédié aux builds CI non sensibles ; interdit sur les agents de déploiement. Préférer Kaniko / Buildah / BuildKit rootless.

Bonnes pratiques de déploiement (3/3) : industrialiser agents et controller

Agents statiques : provisionnés par IaC

- Terraform / Ansible créent la VM, installent Java/Git/Docker, déposent le service systemd ou WinSW ; le **secret** du node est fourni par un coffre (Vault, SOPS)
- Le node Jenkins correspondant est créé par **JCasC** ou par l'API, jamais à la main

```
# jenkins.yaml (JCasC) – déclaration des nodes
jenkins:
  numExecutors: 0
  mode: EXCLUSIVE
  nodes:
    - permanent:
        name: linux-1
        remoteFS: /home/jenkins/agent
        labelString: "linux java21 maven docker"
        numExecutors: 2
        mode: EXCLUSIVE
        launcher:
          ssh:
            host: linux-1.example.com
            port: 22
            credentialsId: agent-ssh-key
            sshHostKeyVerificationStrategy:
              knownHostsFileKeyVerificationStrategy: {}
        retentionStrategy: always
    - permanent:
        name: windows-1
        remoteFS: "C:\\jenkins"
        labelString: "windows dotnet"
        numExecutors: 1
        mode: EXCLUSIVE
        launcher:
          inbound:
            webSocket: true
        retentionStrategy: always
```

Clouds en JCasC

```
jenkins:
  clouds:
    - kubernetes:
        name: k8s
        namespace: jenkins-agents
        jenkinsUrl: http://jenkins.jenkins.svc:8080
        webSocket: true
        containerCapStr: "20"
        templates:
          - name: maven
            label: docker-maven
            containers:
              - name: maven
                image: maven:3.9-eclipse-temurin-21
                command: sleep
                args: infinity
                resourceRequestCpu: "1"
                resourceRequestMemory: 2Gi
```

Cycle de vie

- **Mises à jour** : controller LTS mis à jour en premier, puis les images d'agents (le `agent.jar` embarqué doit rester compatible) ; `ExecStartPre` re-télécharge `agent.jar` au redémarrage du service
- **Rotation** : agents statiques reconstruits régulièrement depuis l'IaC plutôt que corrigés en place (évite la dérive)
- **Nettoyage** : `cleanWs()` en fin de build, tâche du core *Workspace clean-up* (quotidienne), `docker system prune -af --volumes` hebdomadaire sur les hôtes Docker
- **Test** : un job `cluster-smoke` planifié (`H/30 * * * *`) qui exécute `echo ok` sur chaque label et alerte si un label n'est plus servi

Critère de réussite
Perdre un agent, ou le controller, ne doit coûter qu'un redéploiement automatisé depuis Git : aucune configuration n'existe uniquement « dans l'UI ».

06

Administration de Jenkins

Configuration des vues · Considérations multi plates-formes · Visite guidée de la JENKINS_HOME · Monitorer et sauvegarder · Scripts Groovy · Ligne de commande d'administration

Configuration des vues (1/2) : organiser le dashboard

Qu'est-ce qu'une vue ?

- Un **filtre d'affichage** sur les jobs : onglets en haut du dashboard. La vue *All* est créée par défaut
- Une vue ne contient pas les jobs, elle les *montre* ; un job peut apparaître dans plusieurs vues
- Vues **globales** (*New View* sur le dashboard, permission View/Create) et **personnelles** (*My Views*, propres à chaque utilisateur)
- Un **Folder** a ses propres vues : combinaison Folder (regroupement + droits) et vues (présentation)

Types de vues

Type	Contenu
List View	Sélection manuelle de jobs et/ou regex sur le nom ; colonnes configurables
My View	Jobs de premier niveau que l'utilisateur peut configurer (`Job/Configure`)
Dashboard View (plugin)	Portlets : statistiques de jobs, tests, derniers builds, graphes
Build Monitor View (plugin)	Grandes tuiles colorées pour écran mural (radiateur d'information)
Nested View (plugin)	Vues imbriquées (onglets d'onglets) – alternative aux Folders
Pipeline / Delivery Pipeline View (plugins)	Chaîne de jobs visualisée par étapes
Build Pipeline View (plugin)	Enchaînements upstream/downstream Freestyle – déconseillé : peu maintenu, incompatible avec la CSP de Jenkins ; les versions 1.x ont une XSS (SECURITY-879)

Créer une List View « Library »

1. Dashboard → **+** (New View) → nom `Library` , type *List View*
2. *Job Filters* : *Use a regular expression to include jobs into the view* : `library-.*` (et/ou cocher les jobs un par un)
3. *Recurse in subfolders* : pour voir les jobs des Folders
4. *Columns* : Status, Weather, Name, Last Success, Last Failure, Last Duration, Build Button – ajouter *Last Stable*, *Test Result* (plugin), *Description*
5. Plugin **View Job Filters** : filtres avancés (par statut, par SCM, par paramètre, jobs non construits depuis N jours…)

Vue par défaut et vues par équipe

- *Manage Jenkins → System → Default view* : la vue affichée à l'arrivée (ex. `Library` plutôt que *All*)
- Une vue par équipe/produit + une vue `Nightly` + une vue `Déploiements` ; *All* reste pour les administrateurs
- Colonnes utiles pour l'exploitation : *Last Duration* (dérive des temps de build) et *Last Failure* (jobs cassés depuis longtemps)

Vues ≠ sécurité

Une vue ne cache rien : un utilisateur sans permission *Job/Read* ne verra pas le job, quelle que soit la vue ; un utilisateur avec les droits y accède même si aucune vue ne le liste (URL directe). Les droits se gèrent par Folder et par matrice/rôles.

Configuration des vues (2/2) : radiateur d'équipe et vues en code

Build Monitor View (écran mural)

- Plugin **Build Monitor View** : une tuile par job, verte/rouge/jaune, nom du build en cours, « who broke it » (committers du build cassé), temps restant estimé
- Configuration : regex de jobs, ordre (par statut, par nom), affichage des badges, thème
- URL de la vue en plein écran sur un écran de l'open space : `https://jenkins.example.com/view/Radiateur/` ; un utilisateur en lecture seule dédié (`wallboard`) avec session longue

Dashboard View

- Portlets configurables : *Jobs Grid*, *Test Statistics Grid*, *Latest Builds*, *Unstable Jobs*, *Build Statistics*, graphes de tendance tests/couverture (avec plugins JUnit/Coverage)
- Utile pour un responsable : vue d'ensemble de la santé des builds d'un produit

Vues et Multibranch

- Un Multibranch Pipeline est un conteneur : la vue liste le job parent ; *Recurse in subfolders* fait apparaître chaque branche (`library-api » main` , `» PR-57`) – souvent trop verbeux : filtrer sur `.*/main$`

Déclarer les vues en code

```
# JCasC – jenkins.yaml
jenkins:
  primaryView:
    list:
      name: Library
  views:
    - all:
        name: all
    - list:
        name: Library
        includeRegex: "library-.*"
        recurse: true
        columns:
          - status
          - weather
          - jobName
          - lastSuccess
          - lastFailure
          - lastDuration
          - buildButton
    - list:
        name: Nightly
        includeRegex: "nightly-.*"
        columns: [status, weather, jobName, lastDuration]
```

```
// Job DSL – views.groovy (seed job)
listView('Déploiements') {
  description('Jobs de déploiement, tous environnements')
  jobs { regex(/.*\/deploy.*/) }
  recurse(true)
  columns { status(); weather(); name(); lastSuccess(); lastDuration(); buildButton() }
}
buildMonitorView('Radiateur') {
  jobs { regex(/library-.*\/main/) }
  recurse(true)
}
```

Règle

Comme les jobs et les nodes : les vues font partie de la configuration versionnée. Une vue créée à la main disparaît à la prochaine reconstruction du controller.

Considérations multi plates-formes (1/2) : Linux, Windows, macOS

Où tourne quoi ?

Composant	Linux	Windows	macOS
Controller	**Recommandé** (paquet, Docker, K8s)	Possible (service .msi), rare en prod	Dev / test seulement
Agent	SSH ou inbound	Inbound (service WinSW)	Inbound (launchd) ou SSH ; nécessaire pour Xcode/signature iOS
Chemins	`/home/jenkins/agent`	`C:\jenkins`	`/Users/jenkins/agent`
Shell par défaut	`sh` (bash)	`bat` / `powershell`	`sh` (zsh/bash)
Séparateur PATH	`:`	`;`	`:`
Fin de ligne	LF	CRLF	LF
Sensibilité à la casse	Oui	Non	Non (par défaut)

Agent macOS (launchd)

```
<!— ~/Library/LaunchAgents/io.jenkins.agent.plist —>
<plist version="1.0"><dict>
  <key>Label</key><string>io.jenkins.agent</string>
  <key>ProgramArguments</key><array>
    <string>/usr/bin/java</string><string>—jar</string><string>/Users/jenkins/agent/agent.jar</string>
    <string>—url</string><string>https://jenkins.example.com/</string>
    <string>—name</string><string>macos-1</string>
    <string>—secret</string><string>@/Users/jenkins/agent/secret-file</string>
    <string>—webSocket</string><string>—workDir</string><string>/Users/jenkins/agent</string>
  </array>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
</dict></plist>
```

`launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/io.jenkins.agent.plist` – LaunchAgent (session utilisateur, nécessaire pour le trousseau et la signature) plutôt que LaunchDaemon.

Écrire des Jenkinsfile portables

```
// Détection de la plate-forme
def run(String cmd) { isUnix() ? sh(cmd) : bat(cmd) }

pipeline {
  agent { label 'linux || windows' }
  tools { jdk 'jdk21'; maven 'maven-3.9' }      // résolus via Tool Locations par node
  stages {
    stage('Build') {
      steps {
        script {
          run('mvn -B -ntp clean verify')               // Maven est portable
          def sep = isUnix() ? '/' : '\\'
          echo "Workspace : ${env.WORKSPACE}${sep}target"
        }
      }
    }
    stage('Package OS') {
      steps {
        script {
          if (isUnix()) {
            sh 'tar czf dist/app.tar.gz -C target app.jar'
          } else {
            powershell 'Compress-Archive -Path target\\app.jar -DestinationPath dist\\app.zip -Force'
          }
        }
      }
    }
  }
}
```

- Privilégier les outils **eux-mêmes portables** (Maven, Gradle, npm, Python, Docker) et leurs wrappers (`./mvnw` / `mvnw.cmd`) au shell
- Variables : `$VAR` en sh, `%VAR%` en bat, `$env:VAR` en PowerShell – ou injecter via `environment { }` et laisser l'outil lire son environnement
- Chemins relatifs au workspace, jamais absolus
- Tester chaque Jenkinsfile sur chaque plate-forme cible avec une `matrix` (partie 5)

Considérations multi plates-formes (2/2) : outils, encodage, exécution du controller

Tools par plate-forme

- *Manage Jenkins → Tools* : les **installateurs automatiques** (Maven, JDK Adoptium, NodeJS) choisissent le binaire selon l'OS de l'agent : un seul nom `maven-3.9` fonctionne partout
- Outils installés à la main : *Node → Configure → Tool Locations* pour indiquer le chemin spécifique ; le Jenkinsfile reste identique
- Variables d'environnement par node (*Node Properties → Environment variables*) : `JAVA_TOOL_OPTIONS`, `MAVEN_OPTS`, `TEMP` Windows sur un disque rapide

Encodage et locale

- Encodage : depuis Java 18, UTF-8 est le charset par défaut (`file.encoding`) ; les accents cassés viennent de la console (`stdout.encoding`) et de la locale de l'OS, surtout sous Windows (`chcp 65001`, `-Dstdout.encoding=UTF-8`)
- Fuseau horaire : `-Duser.timezone=Europe/Paris` sur le controller (affichage des dates) ; les agents en UTC ou local, mais cohérents (Node monitor *Clock Difference*)
- Locale des messages Maven/Gradle : `-Duser.language=en` évite des logs mi-français mi-anglais

Git

- `core.autocrlf=false` sur les agents Windows + `.gitattributes` (`* text=auto eol=lf`, `*.bat text eol=crlf`) dans les dépôts
- `core.longpaths=true` sur Windows
- Même version majeure de Git sur tous les agents (comportements de `git clean`, `sparse-checkout`)

Le controller selon l'OS

Élément	Linux (paquet)	Windows (service)	Docker
JENKINS_HOME	`/var/lib/jenkins`	Compte dédié : `%LocalAppData%\Jenkins\.jenkins` ; LocalSystem : `%ProgramData%\Jenkins\.jenkins` (vérifier `JENKINS_HOME` dans `jenkins.xml`)	`/var/jenkins_home` (volume)
Options JVM	`systemctl edit jenkins` → `Environment="JAVA_OPTS=…"`	`jenkins.xml` (balise `<arguments>`)	variable `JAVA_OPTS`
Port HTTP	`JENKINS_PORT`	`--httpPort` dans `jenkins.xml`	`-p 8080:8080`
Logs	`journalctl -u jenkins`	`jenkins.out.log`, Event Viewer	`docker logs`
Utilisateur	`jenkins`	Compte de service dédié (recommandé par l'installeur ; LocalSystem déconseillé)	uid 1000
Redémarrage	`systemctl restart jenkins`	`Restart-Service jenkins`	`docker restart`

Controller Windows

Fonctionne, mais : chemins longs dans `jobs/…/builds`, verrous de fichiers lors des mises à jour de plugins, moins d'outillage de sauvegarde. Si l'organisation est Windows-only, un controller Linux en VM ou conteneur + agents Windows est le compromis habituel.

Référence : Jenkins.io → *Installing Jenkins* par plate-forme ; *Administering Jenkins → System properties* pour la liste des `-D`.

Visite guidée de la JENKINS_HOME (1/3) : arborescence

```
/var/jenkins_home/
├── config.xml                    # configuration globale : security realm, authorization,
│                                 #   executors, labels, vues, clouds, nodes « built-in »
├── credentials.xml               # credentials globaux (chiffrés)
├── secrets/                      # *** clés de chiffrement *** - sans ce dossier, credentials.xml est illisible
│   ├── master.key
│   ├── hudson.util.Secret
│   └── initialAdminPassword
├── hudson.model.UpdateCenter.xml # Update Center
├── hudson.plugins.*.xml          # configuration de chaque plugin (ex. hudson.plugins.emailext.ExtendedEmailPublisher.xml)
├── jenkins.model.JenkinsLocationConfiguration.xml   # Jenkins URL, admin e-mail
├── org.jenkinsci.plugins.*.xml   # idem (SonarQube, GitHub…)
├── nodeMonitors.xml              # seuils des node monitors
├── queue.xml                     # file d'attente persistée (survit au redémarrage)
├── identity.key.enc              # identité de l'instance
├── jobs/                         # UN DOSSIER PAR JOB (ou folder)
│   └── library-api/
│       ├── config.xml            # configuration du job (XML du type d'item)
│       ├── nextBuildNumber
│       ├── branches/             # (Multibranch) un sous-job par branche
│       └── builds/
│           ├── 41/
│           ├── 42/
│           │   ├── build.xml     # résultat, durée, causes, actions (tests, fingerprints…)
│           │   ├── log           # console output
│           │   ├── changelog.xml # commits
│           │   ├── archive/      # artefacts archivés
│           │   ├── junitResult.xml
│           │   └── workflow/     # (Pipeline) état des steps, pour la reprise
│           ├── permalinks                # cache lastSuccessfulBuild, lastStableBuild… (plus de symlinks depuis 2.185)
│           └── permalinks
├── nodes/                        # un dossier par agent : config.xml
├── plugins/                      # *.jpi (+ .jpi.disabled, .bak, dossiers dézippés)
├── users/                        # un dossier par utilisateur : config.xml (hash mdp, tokens API)
│   └── users.xml                 # index
├── fingerprints/                 # base des fingerprints (MD5 → jobs)
├── logs/                         # logs des tâches planifiées (Workspace cleanup, Fingerprint cleanup…)
├── updates/                      # cache de l'Update Center
├── workspace/                    # workspaces des builds exécutés SUR LE CONTROLLER (doit rester vide)
├── war/                          # jenkins.war dézippé (regénéré à chaque démarrage)
├── caches/                       # caches Git des Multibranch (scan des branches)
├── init.groovy.d/                # scripts Groovy exécutés au démarrage (partie Scripts)
├── casc_configs/ ou jenkins.yaml # Configuration as Code (si JCasC)
└── userContent/                  # fichiers statiques servis sur /userContent/
```

Trois catégories

Irremplaçable

`secrets/`, `credentials.xml`, `config.xml`, `jobs/*/config.xml`, `nodes/`, `users/`, `*.xml` à la racine, `plugins/*.jpi`. Sans eux, l'instance ne se reconstruit pas. **Sauvegarde obligatoire.**

Historique

`jobs/*/builds/`, `fingerprints/`. Précieux (traçabilité) mais volumineux ; sauvegarde selon la politique de rétention.

Régénérable

`war/`, `updates/`, `caches/`, `workspace/`, `plugins/*/` (dossiers dézippés), `logs/`. À exclure des sauvegardes.

Explorer sans risque

```
du -sh /var/jenkins_home/* | sort -h | tail
du -sh /var/jenkins_home/jobs/* | sort -h | tail -5
find /var/jenkins_home/jobs -maxdepth 3 -name builds -exec sh -c 'echo "$(ls {} | wc -l) {}"' \; | sort -n | tail
```

Visite guidée de la JENKINS_HOME (2/3) : les fichiers clés

`config.xml` (extrait)

```
<hudson>
  <version>2.580.2</version>
  <numExecutors>0</numExecutors>
  <mode>EXCLUSIVE</mode>
  <useSecurity>true</useSecurity>
  <authorizationStrategy class="hudson.security.ProjectMatrixAuthorizationStrategy">
    <permission>GROUP:hudson.model.Hudson.Administer:jenkins-admins</permission>
    <permission>GROUP:hudson.model.Hudson.Read:authenticated</permission>
  </authorizationStrategy>
  <securityRealm class="hudson.security.HudsonPrivateSecurityRealm">
    <disableSignup>true</disableSignup>
  </securityRealm>
  <quietPeriod>5</quietPeriod>
  <scmCheckoutRetryCount>2</scmCheckoutRetryCount>
  <views>…</views>
  <primaryView>Library</primaryView>
  <clouds>…</clouds>
  <label></label>
</hudson>
```

Se dé-verrouiller

Administrateur perdu : arrêter Jenkins, passer `<useSecurity>false</useSecurity>`, redémarrer, recréer les droits, réactiver. À faire uniquement sur un controller isolé du réseau pendant l'opération.

`jobs/library-api/config.xml` (Pipeline from SCM)

```
<flow-definition plugin="workflow-job">
  <description>API REST Library</description>
  <keepDependencies>false</keepDependencies>
  <properties>
    <jenkins.model.BuildDiscarderProperty>
      <strategy class="hudson.tasks.LogRotator"><numToKeep>30</numToKeep></strategy>
    </jenkins.model.BuildDiscarderProperty>
  </properties>
  <definition class="org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition">
    <scm class="hudson.plugins.git.GitSCM">
      <userRemoteConfigs><hudson.plugins.git.UserRemoteConfig>
        <url>git@git.example.com:utopios/library-api.git</url>
        <credentialsId>git-ssh-key</credentialsId>
      </hudson.plugins.git.UserRemoteConfig></userRemoteConfigs>
      <branches><hudson.plugins.git.BranchSpec><name>*/main</name></hudson.plugins.git.BranchSpec></branches>
    </scm>
    <scriptPath>Jenkinsfile</scriptPath>
    <lightweight>true</lightweight>
  </definition>
  <disabled>false</disabled>
</flow-definition>
```

Lire et écrire ces fichiers

- Lecture : `GET /job/library-api/config.xml` (API) – ce que fait la CLI `get-job`
- Écriture : `POST /job/library-api/config.xml` ou CLI `update-job` / `create-job` ; modification directe sur disque suivie de *Reload Configuration from Disk* (en dernier recours)
- `credentials.xml` : valeurs chiffrées en AES avec `secrets/hudson.util.Secret`, lui-même chiffré par `master.key` ; copier les deux ensemble ou rien
- `users/*/config.xml` : hash bcrypt du mot de passe (`#jbcrypt:`), tokens API hachés

Visite guidée de la JENKINS_HOME (3/3) : maîtriser la taille

D'où vient le volume

Source	Croissance	Levier
`builds/*/log`	Linéaire, logs verbeux (Maven download, tests)	`buildDiscarder`, `-ntp`, `-q`
`builds/*/archive/`	Forte si on archive des JAR/WAR/images	`artifactNumToKeepStr`, publier dans Nexus plutôt qu'archiver
`builds/*/junitResult.xml`, stdout des tests	Forte avec `stdioRetention: 'ALL'`	Garder le défaut (sortie tronquée pour tous les tests) ; au plus `stdioRetention: 'FAILED'` (sortie complète des seuls tests en échec ; remplace `keepLongStdio`)
`workspace/` (controller)	Ne devrait pas exister	0 executor
`caches/` (Multibranch)	Un clone Git par dépôt scanné	Normal ; régénérable
`fingerprints/`	Lente	Tâche de nettoyage automatique (quotidienne)
`plugins/`	Stable (~1 Go)	Désinstaller l'inutile

Rétention par défaut à imposer

- Plugin **Build Discarder** (global) : applique `numToKeep` à tous les jobs sans politique propre
- Dans les Shared Libraries : `buildDiscarder(logRotator(numToKeepStr: '30', artifactNumToKeepStr: '5'))` systématique
- Branches et PR Multibranch : *Orphaned Item Strategy* → supprimer les branches disparues dont le dernier build a plus de 7 jours

Mesurer

- Plugin **Disk Usage** : taille par job / par build, graphe de tendance, dans *Manage Jenkins*
- Plugin **Prometheus metrics** + **CloudBees Disk Usage Simple** : `default_jenkins_disk_usage_bytes`, `default_jenkins_file_store_available_bytes` (partie Monitorer)
- Alerte système sur le volume (> 80 %) : un controller à disque plein se corrompt (fichiers `build.xml` tronqués)

Nettoyer

```
// Script Console : supprimer les builds de plus de 90 jours de tous les jobs,
// en gardant les builds marqués "keep forever"
import jenkins.model.Jenkins
def limit = System.currentTimeMillis() - 90L*24*3600*1000
int removed = 0
Jenkins.get().getAllItems(Job).each { job →
  job.builds.findAll { !it.isKeepLog() && it.timeInMillis < limit && !it.isBuilding() }
            .each { it.delete(); removed++ }
}
println "Builds supprimés : ${removed}"
```

```
# Nettoyage des dossiers régénérables (Jenkins arrêté)
rm -rf /var/jenkins_home/war /var/jenkins_home/updates
rm -rf /var/jenkins_home/plugins/*/        # garder les .jpi
# Workspaces résiduels sur un agent (depuis l'agent)
find /home/jenkins/agent/workspace -maxdepth 1 -mtime +30 -type d -exec rm -rf {} +
```

Tâches planifiées intégrées

Tâche du core *Workspace clean-up* : supprime chaque jour les workspaces inutilisés depuis 30 jours (réglage par system properties `hudson.model.WorkspaceCleanupThread.retainForDays`, `.disabled` ; pas d'écran ; le plugin ws-cleanup, lui, fournit `cleanWs()`) ; nettoyage des fingerprints et des `logs/tasks` automatiques.

Monitorer Jenkins (1/3) : ce qu'il faut surveiller

Quatre niveaux

Niveau	Indicateurs	Outils
Système	CPU, RAM, disque `JENKINS_HOME`, inodes, I/O	node_exporter, agent de l'hyperviseur, `df`, `iostat`
JVM	Heap used/max, GC pauses, threads, file descriptors	Metrics plugin, JMX, `jcmd`, Prometheus plugin
Jenkins	Longueur de la queue, executors occupés, nodes offline, durée des builds, taux d'échec, temps de réponse HTTP	Metrics plugin, Prometheus plugin, Load Statistics
Métier	Lead time commit → prod, fréquence de déploiement, MTTR (DORA)	Plugins DORA / Pipeline Metrics, export vers un entrepôt

Seuils d'alerte raisonnables

- Disque `JENKINS_HOME` > 80 % → avertissement ; > 90 % → critique
- Heap après GC > 80 % de `-Xmx` pendant 10 min → fuite ou sous-dimensionnement
- Queue > 10 pendant 15 min sur un label → capacité
- Un node `offline` > 5 min hors maintenance planifiée
- Temps de réponse de `/login` > 2 s → controller saturé
- Job `cluster-smoke` en échec → alerte plateforme

Outils intégrés

- *Manage Jenkins → System Information* : JVM, mémoire, variables, plugins avec versions
- *Manage Jenkins → System Log* : loggers par package (`hudson.model.Run`, `org.jenkinsci.plugins.workflow`, `com.cloudbees.jenkins.plugins.sshcredentials`) avec niveau FINE pour diagnostiquer ; *All Jenkins Logs*
- *Manage Jenkins → Load Statistics* : queue / executors (échelles Short, Medium, Long)
- *Manage Jenkins → Nodes* : monitors disque / horloge / temps de réponse
- `/manage/about`, `/threadDump` (thread dump du controller à chaud) ; `/computer/<node>/systemInfo`
- **Administrative monitors** : bandeaux d'alerte (plugins vulnérables, nouvelle version, config dépréciée, builds sur le controller, trop de builds en mémoire…)

Plugins de monitoring

Plugin	Apporte
Metrics	Métriques Dropwizard : `/metrics/<key>/metrics` (JSON), health checks `/metrics/<key>/healthcheck`
Prometheus metrics	Endpoint `/prometheus/` : queue, executors, builds par job (durée, résultat), JVM
Monitoring (JavaMelody)	Tableaux de bord intégrés : HTTP, JVM, threads, erreurs ; sans infra externe
Support Core	*Support bundle* : zip de diagnostic (logs, config anonymisée, thread dumps) pour analyse
Audit Trail	Qui a fait quoi (config, lancement, suppression)
Build Failure Analyzer	Catégorise les causes d'échec par regex sur les logs

Monitorer Jenkins (2/3) : Prometheus, Grafana et réglage de la JVM

Exposer les métriques

1. Installer **Prometheus metrics plugin**
2. *Manage Jenkins → System → Prometheus* : path `prometheus`, *Collecting metrics period* 120 s, cocher *Count duration of successful builds*, *Fetch test results* ; *Authentication* : désactivée + restriction réseau, ou token
3. Vérifier : `curl -s https://jenkins.example.com/prometheus/ | head`

```
# prometheus.yml
scrape_configs:
  - job_name: jenkins
    metrics_path: /prometheus/
    static_configs: [{ targets: ['jenkins.example.com:443'] }]
    scheme: https
```

Métriques utiles

Métrique	Sens
`jenkins_queue_size_value`	Longueur de la queue
`jenkins_executor_in_use_value` / `_count_value`	Executors occupés / total
`jenkins_node_offline_value`	Nodes hors ligne
`default_jenkins_builds_duration_milliseconds_summary`	Durée par job
`default_jenkins_builds_last_build_result`	1 si le dernier build est SUCCESS ou UNSTABLE, 0 sinon (détail : `…_last_build_result_ordinal`, 0 = SUCCESS … 4 = ABORTED)
`vm_memory_heap_usage`, `vm_gc_*`	JVM
`http_requests`, `http_responseCodes_*`	Santé HTTP

Dashboards Grafana communautaires : « Jenkins: Performance and Health Overview » (ID 9964) comme point de départ.

Régler la JVM du controller

```
# systemctl edit jenkins  (ou JAVA_OPTS du conteneur)
[Service]
Environment="JAVA_OPTS=-Xms2g -Xmx6g \
  -XX:+UseG1GC -XX:+UseStringDeduplication \
  -XX:+ParallelRefProcEnabled -XX:+DisableExplicitGC \
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/lib/jenkins/heapdumps \
  -Xlog:gc*:file=/var/log/jenkins/gc.log:time,uptime:filecount=5,filesize=20m \
  -Duser.timezone=Europe/Paris \
  -Djenkins.install.runSetupWizard=false"
```

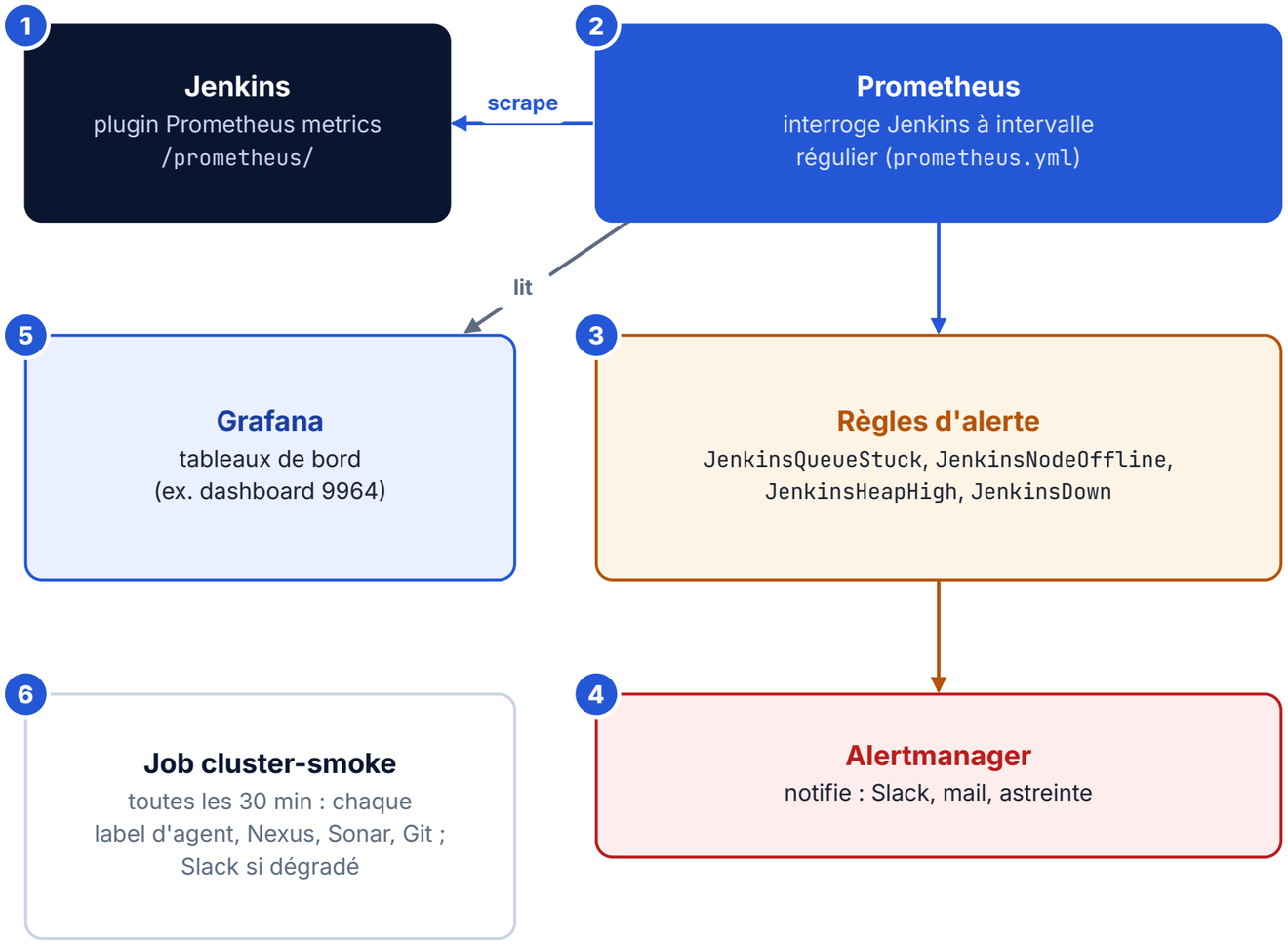
- `-Xmx` : 50–70 % de la RAM de la machine ; au-delà de 16 Go, préférer plusieurs controllers
- Logs GC : des pauses > 1 s répétées = heap trop petit ou trop de builds/plugins en mémoire
- `-XX:MaxRAMPercentage=70` plutôt que `-Xmx` fixe dans un conteneur avec limite mémoire

Diagnostiquer un controller lent

```
# Thread dump (3 à 10 s d'intervalle, 3 fois)
jcmd $(pgrep -f jenkins.war) Thread.print > /tmp/td-$(date +%s).txt
# ou via l'UI : https://jenkins.example.com/threadDump
# Heap histogram
jcmd $(pgrep -f jenkins.war) GC.class_histogram | head -30
```

- Causes classiques : plugin en boucle (*Build Failure Analyzer* sur de gros logs), trop de builds Pipeline chargés en mémoire (*lazy loading* cassé par un plugin), scan Multibranch de centaines de dépôts, `junit` avec `keepLongStdio` sur des milliers de tests
- Plugin **Support Core** → *Generate bundle* pour analyse hors ligne

Schéma : la supervision de Jenkins



Comment le lire

1. Le plugin expose les métriques (queue, executors, nodes, JVM, durées de build) sur `/prometheus/`.
2. Prometheus vient les lire régulièrement : c'est lui qui stocke l'historique.
3. Des règles transforment les métriques en alertes : queue bloquée, node hors ligne, heap saturé, Jenkins injoignable.
4. Alertmanager envoie l'alerte au bon canal.
5. Grafana affiche les tendances pour l'exploitation.
6. Un job de supervision interne vérifie ce que les métriques ne voient pas : chaque type d'agent et les dépendances externes.

À retenir

Les métriques disent que Jenkins va mal ; le job de supervision dit si on peut encore **builder**.

Monitorer Jenkins (3/3) : logs, alerting et job de supervision

Logs du controller

- Journal système (`journalctl -u jenkins` , `docker logs`) : démarrage, erreurs graves, stack traces
- *System Log* : log recorders ciblés ; leur définition est sauvegardée dans `JENKINS_HOME/log/` , mais les enregistrements restent en mémoire (perdus au redémarrage) ; ne pas laisser un logger en FINE indéfiniment
- Centralisation : Loki/Promtail, Elastic (Filebeat), ou plugin **Logstash** pour envoyer aussi les logs de *builds*
- `-Djava.util.logging.config.file=logging.properties` pour une configuration fine et des fichiers tournants

Health checks

```
# Metrics plugin : état agrégé (disk space, plugins, temporary space, thread deadlock)
curl -s https://jenkins.example.com/metrics/<API_KEY>/healthcheck | jq
# Sonde simple pour un load balancer / Kubernetes
curl -sf https://jenkins.example.com/login -o /dev/null
# API : nombre de nodes offline
curl -sg -u user:token "https://jenkins.example.com/computer/api/json?tree=computer[displayName,offline]" \
  | jq '[.computer[] | select(.offline)] | length'
```

Alerting (règles d'alerte Prometheus, notifiées par Alertmanager)

```
- alert: JenkinsQueueStuck
  expr: jenkins_queue_size_value > 10
  for: 15m
- alert: JenkinsNodeOffline
  expr: jenkins_node_offline_value > 0
  for: 5m
- alert: JenkinsHeapHigh
  expr: vm_memory_heap_usage > 0.85
  for: 10m
- alert: JenkinsDown
  expr: up{job="jenkins"} == 0
  for: 2m
```

Un job de supervision interne

```
// admin/cluster-smoke - toutes les 30 min
pipeline {
  agent none
  triggers { cron('H/30 * * * *') }
  options { timeout(time: 10, unit: 'MINUTES'); buildDiscarder(logRotator(numToKeepStr: '48')) }
  stages {
    stage('Labels') {
      parallel {
        stage('linux')        { agent { label 'linux' };        steps { sh 'echo ok' } }
        stage('docker-maven') { agent { label 'docker-maven' }; steps { sh 'echo ok' } }
        stage('windows')      { agent { label 'windows' };      steps { bat 'echo ok' } }
        stage('deploy')       { agent { label 'deploy' };       steps { sh 'kubectl version --client' } }
      }
    }
    stage('Dépendances externes') {
      agent { label 'linux' }
      steps {
        sh 'curl -sf http://nexus:8081/service/rest/v1/status'
        sh 'curl -sf http://sonarqube:9000/api/system/status | grep -q UP'
        sh 'git ls-remote git@git.example.com:utopios/library-api.git HEAD'
      }
    }
  }
  post {
    unsuccessful { slackSend channel: '#jenkins-ops', color: 'danger',     // tout résultat ≠ SUCCESS (dont ABORTED)
                   message: "Plateforme CI dégradée : ${env.BUILD_URL}" }
    success { script { if (currentBuild.previousCompletedBuild?.result ≠ 'SUCCESS')
                slackSend channel: '#jenkins-ops', color: 'good', message: 'Plateforme CI rétablie' } }
  }
}
```

- Avec `agent none` , le `timeout` global couvre l'attente des agents : si un label n'a plus d'agent, le build est interrompu en **ABORTED** (pas FAILURE), d'où `post { unsuccessful }` . `fixed` ne se déclenche qu'après un FAILURE ou un UNSTABLE : après un ABORTED, on teste le build précédent dans `success`
- Tester aussi les dépendances **externes** : la plupart des « pannes Jenkins » sont des pannes de Nexus, Git ou DNS

Revue hebdomadaire
Jobs en échec depuis > 7 jours (vue filtrée), durées en hausse, plugins à mettre à jour, espace disque, nodes inutilisés.

Sauvegarder Jenkins (1/3) : quoi, quand, où

Périmètre de sauvegarde

Contenu	Inclure	Fréquence
`secrets/`, `credentials.xml`, `*.xml` racine, `config.xml`	**Toujours**	Quotidienne + avant chaque changement
`jobs/*/config.xml`, `nodes/`, `users/`, `init.groovy.d/`, `jenkins.yaml`	**Toujours**	Quotidienne
`plugins/*.jpi`	Oui (ou `plugins.txt` avec versions)	Quotidienne
`jobs/*/builds/`	Selon politique (traçabilité)	Hebdomadaire / selon rétention
`fingerprints/`, `userContent/`	Optionnel	Hebdomadaire
`war/`, `updates/`, `caches/`, `workspace/`, `plugins/*/`, `logs/`	**Exclure**	—

Quand

- Nocturne automatique (faible activité)
- **Avant** toute mise à jour (controller, plugins), toute migration, toute opération en Script Console
- Les sauvegardes sont copiées **hors** de la machine (stockage objet, NAS) et chiffrées (elles contiennent les clés de chiffrement des secrets)

Quatre approches

1. Snapshot du volume

VM (snapshot disque), volume Docker, PVC Kubernetes (VolumeSnapshot), EBS. Cohérent si Jenkins est en *quiet mode* ou arrêté ; rapide ; restauration intégrale.

2. Archive de fichiers (`tar` / `rsync`)

Sélective (exclusions), portable, versionnable. Jenkins peut tourner pendant la copie si on accepte un léger risque d'incohérence sur les builds en cours ; sinon *Prepare for Shutdown* avant.

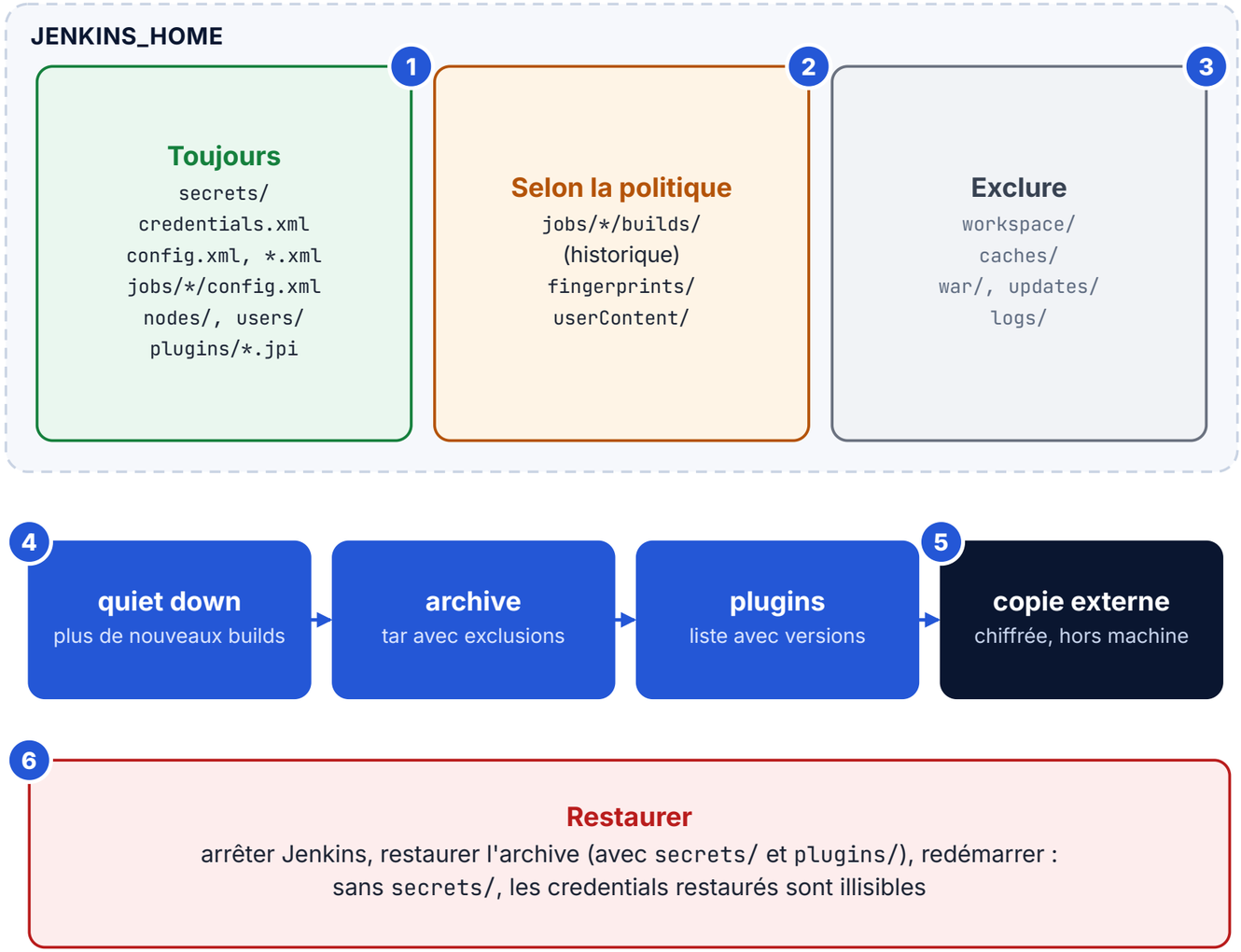
3. Plugins de sauvegarde

ThinBackup : planification dans l'UI, sauvegarde différentielle, exclut les builds au choix, restauration par l'UI. **Backup** (ancien), **PeriodicBackup**. Pratiques mais dépendants du controller : à compléter par une copie externe.

4. Configuration as Code = sauvegarde par construction

JCasC (`jenkins.yaml`) + Job DSL / Jenkinsfile dans Git + `plugins.txt` + Dockerfile : tout sauf l'historique et les secrets est reconstructible depuis Git. Il ne reste à sauvegarder que `secrets/`, `users/` (tokens API), `credentials.xml` s'il n'est pas géré par JCasC, et `builds/` selon la rétention.

Schéma : que sauvegarder, et comment



Comment le lire

1. **Toujours** : la configuration et les secrets. Sans eux, on ne reconstruit rien.
2. **Selon la politique** : l'historique des builds, utile pour la traçabilité mais volumineux.
3. **Exclure** : ce qui se régénère (workspaces, caches, war décompressé, logs).
4. Le script met Jenkins en *quiet down*, archive, enregistre la liste des plugins…
5. … puis copie l'archive hors de la machine, chiffrée : elle contient les clés de déchiffrement des secrets.
6. La restauration remet `secrets/` avec le reste : sans ce dossier, les credentials restaurés ne peuvent pas être déchiffrés.

À retenir

Une sauvegarde n'existe que si sa **restauration** a été testée.

Sauvegarder Jenkins (2/3) : script de sauvegarde et restauration

Script de sauvegarde (à planifier en cron système, pas dans Jenkins)

```
#!/usr/bin/env bash
# /usr/local/sbin/jenkins-backup.sh
set -euo pipefail
JENKINS_HOME=/var/lib/jenkins
JENKINS_URL=https://jenkins.example.com
AUTH="backup:$(cat /etc/jenkins-backup.token)"       # user technique, droits Administer
# toujours sortir du quiet mode, même si une étape échoue (sauf KEEP_QUIET=1)
[ "${KEEP_QUIET:-0}" = 1 ] || trap 'curl -sf -X POST -u "$AUTH" "$JENKINS_URL/cancelQuietDown"' EXIT
DEST=/backup/jenkins
TS=$(date +%Y%m%d-%H%M)

# 1. Quiet mode : plus de nouveaux builds, les builds en cours se terminent
curl -sf -X POST -u "$AUTH" "$JENKINS_URL/quietDown"
for i in $(seq 1 60); do
  busy=$(curl -s -u "$AUTH" "$JENKINS_URL/computer/api/json?tree=busyExecutors" | jq .busyExecutors)
  [ "$busy" = "0" ] && break; sleep 10
done

# 2. Archive sélective
# code 1 de GNU tar = « fichier modifié pendant la lecture » : avertissement
tar --exclude='./war' --exclude='./updates' --exclude='./caches' \
    --exclude='./workspace' --exclude='./plugins/*/*' --exclude='./logs' \
    --exclude='./jobs/*/builds/*/archive' \
    -czf "$DEST/jenkins-home-$TS.tgz" -C "$JENKINS_HOME" . || [ $? -eq 1 ]

# 3. Liste des plugins avec versions (reconstruction)
curl -sg -u "$AUTH" "$JENKINS_URL/pluginManager/api/json?depth=1&tree=plugins[shortName,version]" \
  | jq -r '.plugins[] | "\(.shortName):\(.version)"' | sort > "$DEST/plugins-$TS.txt"

# 4. Sortie du quiet mode : assurée par le trap EXIT

# 5. Copie hors site + rétention 30 jours
aws s3 cp "$DEST/jenkins-home-$TS.tgz" s3://backups-ci/jenkins/ --sse aws:kms
find "$DEST" -name 'jenkins-home-*.tgz' -mtime +30 -delete
echo "Backup OK $TS"
```

Procédure de restauration

1. Provisionner une machine/conteneur avec la **même version** de Jenkins (ou plus récente d'un cran LTS) et la même version de Java
2. Jenkins **arrêté** ; `JENKINS_HOME` vide
3. `tar -xzf jenkins-home-YYYYMMDD.tgz -C /var/lib/jenkins` ; `chown -R jenkins:jenkins`
4. Si les plugins ne sont pas dans l'archive : `jenkins-plugin-cli --plugin-file plugins.txt` (image Docker) ou copie des `.jpi`
5. Démarrer ; vérifier : connexion admin, un credential utilisable (lancer un job qui s'en sert), nodes en ligne, dernier build visible
6. Mettre à jour l'URL si l'hôte a changé (*Jenkins Location*), les webhooks côté Git, et le DNS

Le test qui compte

Une sauvegarde jamais restaurée n'existe pas. Restaurer **trimestriellement** sur une instance jetable (Docker) et vérifier que les credentials se déchiffrent : si `secrets/` manque ou est désynchronisé de `credentials.xml`, tous les secrets sont perdus.

Restauration partielle

- Un job supprimé par erreur : extraire `jobs/<job>/` de l'archive dans `JENKINS_HOME/jobs/` puis *Manage Jenkins → Reload Configuration from Disk*
- Un `config.xml` de job cassé : plugin **Job Configuration History** conserve chaque version avec diff et restauration en un clic – à installer dès le départ

Sauvegarder Jenkins (3/3) : mise à jour et plan de reprise

Procédure de mise à jour LTS

1. Lire la **changelog** LTS et les *Upgrade Guides* (jenkins.io) : versions de Java requises, plugins incompatibles, comportements changés
2. **Sauvegarde** complète (slide précédente)
3. Tester sur une **copie** : `docker run -v copie_home:/var/jenkins_home jenkins/jenkins:<nouvelle LTS>` → vérifier le démarrage, les administrative monitors, un pipeline représentatif
4. *Prepare for Shutdown* → attendre la fin des builds
5. Mettre à jour le controller : `apt-get install jenkins` / nouvelle image Docker / nouveau `jenkins.war`
6. Démarrer ; *Manage Jenkins* → *Plugins* → *Updates* : mettre à jour les plugins (d'abord ceux marqués incompatibles avec la nouvelle version)
7. Vérifier : bandeaux d'alerte, un build sur chaque type d'agent, `cluster-smoke` vert
8. Mettre à jour les images d'agents (et les `agent.jar` installés à la main) à chaque mise à jour du controller

Rythme
Une LTS tous les 3 mois : suivre à N ou N-1 ; les correctifs de sécurité LTS (x.y.z) dès parution. Un controller à 2 ans de retard ne se met plus à jour en une étape (sauts de version Java, plugins).

Rollback

- Jenkins ne rétrograde pas proprement (formats de données migrés) : le rollback = **restauration de la sauvegarde** + ancienne version du binaire
- Plugins : *Installed plugins* → bouton *Downgrade to x.y* disponible tant que l'ancien `.jpi.bak` existe

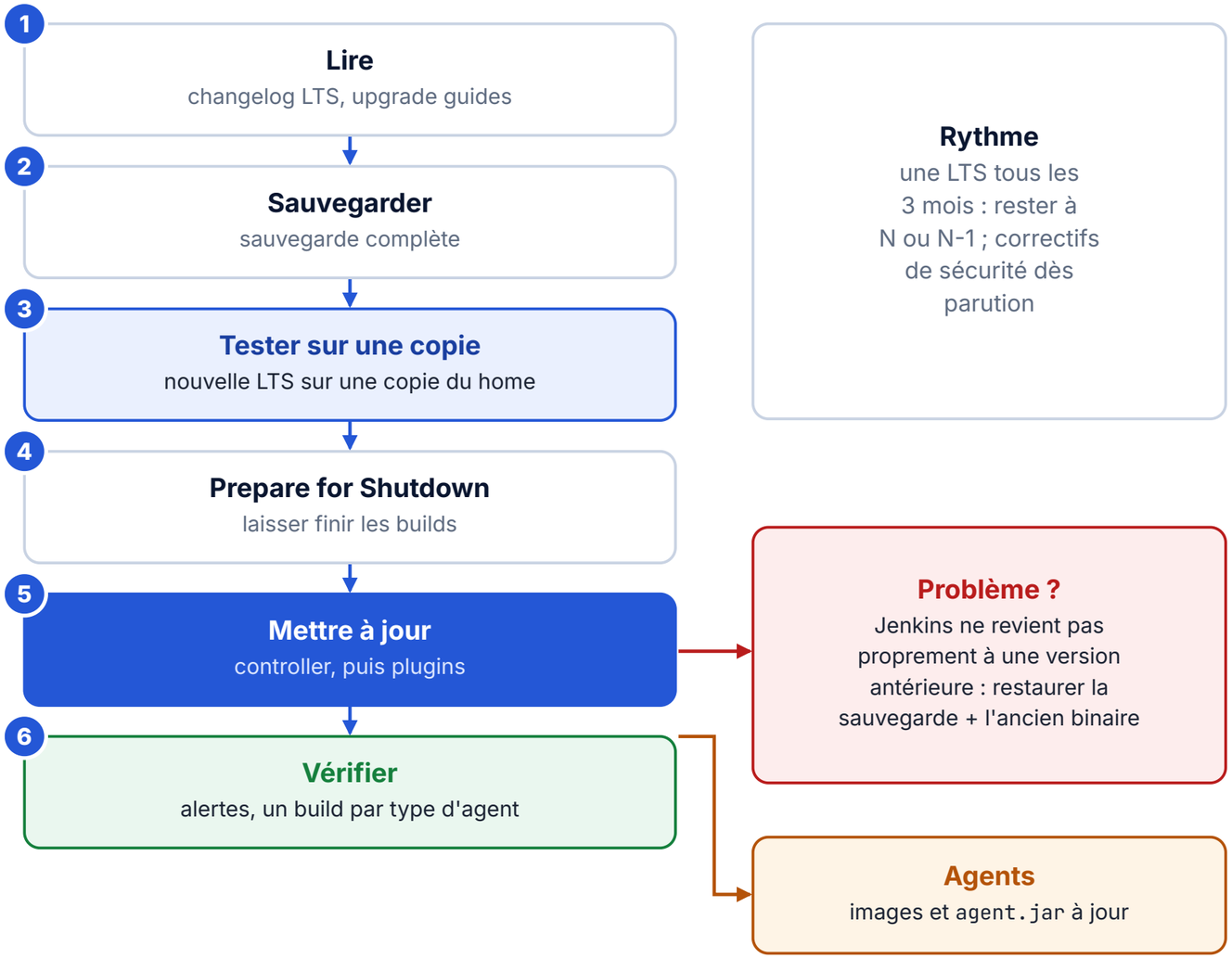
Plan de reprise d'activité (PRA)

Scénario	Réponse	RTO cible
Controller planté (process)	Redémarrage systemd / restart policy Docker / Kubernetes	Minutes
Disque corrompu / machine perdue	Nouvelle machine + restauration de la dernière sauvegarde	1–2 h
Configuration cassée (plugin, Script Console)	Restauration de `config.xml` / `jobs/*/config.xml` ; Job Config History	30 min
Agent perdu	IaC → nouvelle VM ; agents éphémères : rien à faire	Minutes à 1 h
Site indisponible	Reconstruction depuis Git (JCasC, Dockerfile) + secrets restaurés sur un autre site	Demi-journée

Haute disponibilité

- Jenkins OSS = **un seul controller actif** à la fois (pas de cluster actif/actif) ; HA = **actif/passif** : `JENKINS_HOME` sur stockage partagé (NFS/EFS, PVC RWO re-attaché) + bascule par orchestrateur (Kubernetes redémarre le pod ailleurs)
- CloudBees CI propose l'HA active/active ; sinon, réduire le RTO par l'automatisation plutôt que viser le zéro interruption

Schéma : mettre à jour Jenkins sans casser



Comment le lire

1. Lire ce qui change : version de Java exigée, plugins incompatibles, comportements modifiés.
2. Sauvegarder entièrement : c'est le seul chemin de retour.
3. Démarrer la nouvelle version sur une **copie** du home et y faire tourner un pipeline représentatif.
4. *Prepare for Shutdown* : plus de nouveaux builds, ceux en cours se terminent.
5. Mettre à jour le controller, puis les plugins.
6. Vérifier : bandeaux d'alerte, un build sur chaque type d'agent. En cas d'échec : restaurer, pas rétrograder.

À retenir
Un controller qui a deux ans de retard ne se met plus à jour en une étape : suivre les LTS régulièrement coûte moins cher.

Scripts Groovy (1/4) : la Script Console

Qu'est-ce que c'est

- *Manage Jenkins → Script Console* (`/manage/script`) : exécute du **Groovy** directement dans la JVM du controller, avec accès à l'API Java de Jenkins (`jenkins.model.Jenkins` , `hudson.model.*`)
- Permission `Overall/Administer` requise – contrôle total du controller : lecture des secrets déchiffrés, exécution de commandes système avec les droits du processus Jenkins
- Disponible aussi par node : `/computer/<node>/script` exécute sur l'agent
- À tracer (plugin Audit Trail) ; sauvegarder avant toute opération d'écriture

Objets de départ

Expression	Donne
`Jenkins.get()`	Le controller (singleton)
`.items` / `.getAllItems(Job)`	Jobs de premier niveau / tous (récursif, Folders inclus)
`.getItemByFullName('library/library-api')`	Un item par chemin
`.computers` / `.nodes`	Agents (état runtime) / définitions
`.queue`	File d'attente
`.pluginManager.plugins`	Plugins installés
`.securityRealm` , `.authorizationStrategy`	Sécurité
`job.builds` , `job.lastBuild` , `build.result` , `build.duration`	Historique

Javadoc : `javadoc.jenkins.io` ; auto-complétion absente : construire les scripts pas à pas avec `println` .

Scripts d'inventaire (lecture seule)

```
// Jobs non construits depuis 90 jours
def limit = System.currentTimeMillis() - 90L*24*3600*1000
Jenkins.get().getAllItems(Job).findAll { j →
  j.lastBuild == null || j.lastBuild.timeInMillis < limit
}.each { println "${it.fullName} - dernier build : ${it.lastBuild?.timestampString ?: 'jamais'}" }
```

```
// Jobs sans politique de rétention
Jenkins.get().getAllItems(Job).findAll { it.buildDiscarder == null }
              .each { println it.fullName }
```

```
// Plugins installés avec version (format plugins.txt)
Jenkins.get().pluginManager.plugins.sort(false) { it.shortName }   // liste non modifiable : copie triée
      .each { println "${it.shortName}:${it.version}" }
```

```
// Qui a quels droits (Matrix / Project Matrix)
def s = Jenkins.get().authorizationStrategy
s.grantedPermissionEntries.each { perm, entries →
  println "${perm.group.title}/${perm.name} : ${entries*.sid.join(', ')}"
}
```

```
// Credentials : IDs et types (jamais les valeurs dans un log)
// (store global uniquement : les stores de folders se parcourent à part)
import com.cloudbees.plugins.credentials.CredentialsProvider
import com.cloudbees.plugins.credentials.common.StandardCredentials
import hudson.security.ACL
CredentialsProvider.lookupCredentialsInItemGroup(StandardCredentials, Jenkins.get(), ACL.SYSTEM2, [])
  .each { println "${it.id}  (${it.class.simpleName})" }
```

Scripts Groovy (2/4) : opérations d'administration courantes

Agir sur les jobs

```
// Désactiver tous les jobs d'un folder (gel avant migration)
Jenkins.get().getItemByFullName('legacy').getAllItems(Job)
       .each { it.disabled = true; it.save(); println "désactivé : ${it.fullName}" }
```

```
// Imposer une rétention par défaut aux jobs qui n'en ont pas
import hudson.tasks.LogRotator
Jenkins.get().getAllItems(Job).findAll { it.buildDiscarder == null }.each { j →
  j.buildDiscarder = new LogRotator(-1, 30, -1, 5)   // jours, builds, jours artefacts, artefacts
  j.save(); println "rétention posée : ${j.fullName}"
}
```

```
// Interrompre tous les builds en cours depuis plus de 2 h
Jenkins.get().computers.each { c →
  c.executors.findAll { it.busy && it.elapsedTime > 2*3600*1000 }.each { e →
    println "Arrêt : ${e.currentExecutable}"; e.interrupt()
  }
}
```

```
// Vider la file d'attente
Jenkins.get().queue.clear()
```

Agir sur les nodes

```
// Mettre hors ligne tous les agents 'windows' avec une raison
import hudson.slaves.OfflineCause
Jenkins.get().computers.findAll { it.node?.labelString?.contains('windows') }.each {
  it.setTemporaryOfflineCause(new OfflineCause.UserCause(User.current(), 'Maintenance Windows Update'))
  println "offline : ${it.name}"
}
```

```
// Relancer la connexion des agents hors ligne (hors maintenance volontaire)
Jenkins.get().computers.findAll { it.offline && !it.temporarilyOffline && it.name }.each {
  println "reconnexion : ${it.name}"; it.connect(true)
}
```

Sécurité et plugins

```
// Révoquer tous les tokens API d'un utilisateur
def u = hudson.model.User.getById('ancien-collegue', false)
def p = u?.getProperty(jenkins.security.ApiTokenProperty)
p?.revokeAllTokens()        // révoque et sauvegarde
```

```
// Lister les approbations de scripts en attente (Script Security)
org.jenkinsci.plugins.scriptsecurity.scripts.ScriptApproval.get()
   .pendingScripts.each { println it.script.take(80) }
```

Règles

1) Lire avant d'écrire : d'abord le `findAll` + `println`, puis seulement l'action. 2) `save()` après chaque modification d'objet. 3) Conserver les scripts dans un dépôt Git `jenkins-admin-scripts` avec leur description. 4) Jamais de `println` d'un secret.

Scripts Groovy (3/4) : scripts de démarrage `init.groovy.d`

Principe

- Tout fichier `JENKINS_HOME/init.groovy.d/*.groovy` est exécuté **au démarrage** du controller, par ordre alphabétique, avec les droits système
- Usage : provisionner un Jenkins neuf (admin, sécurité, tools, credentials) sans passer par l'UI – historiquement *la* méthode avant JCasC, toujours utile pour ce que JCasC ne couvre pas
- Idempotence obligatoire : le script tourne à chaque redémarrage
- Les secrets viennent de variables d'environnement ou de fichiers montés, jamais en dur

`01-security.groovy`

```
import jenkins.model.Jenkins
import hudson.security.*
import org.jenkinsci.plugins.matrixauth.PermissionEntry

def j = Jenkins.get()
def adminUser = System.getenv('JENKINS_ADMIN_USER') ?: 'admin'
def adminPass = System.getenv('JENKINS_ADMIN_PASSWORD')
if (!adminPass) { println '[init] JENKINS_ADMIN_PASSWORD absent, sécurité non modifiée'; return }

if (!(j.securityRealm instanceof HudsonPrivateSecurityRealm)) {
  def realm = new HudsonPrivateSecurityRealm(false)       // pas d'inscription libre
  realm.createAccount(adminUser, adminPass)
  j.securityRealm = realm
}
if (!(j.authorizationStrategy instanceof ProjectMatrixAuthorizationStrategy)) {
  def strategy = new ProjectMatrixAuthorizationStrategy()
  strategy.add(Jenkins.ADMINISTER, PermissionEntry.user(adminUser))
  strategy.add(Jenkins.READ, PermissionEntry.group('authenticated'))
  j.authorizationStrategy = strategy
}
j.numExecutors = 0
j.save()
println '[init] sécurité configurée'
```

`02-tools.groovy`

```
import jenkins.model.Jenkins
import hudson.tasks.Maven
import hudson.tools.*

def desc = Jenkins.get().getDescriptor(Maven.MavenInstallation)
if (!desc.installations.find { it.name == 'maven-3.9' }) {
  def installer = new Maven.MavenInstaller('3.9.9')
  def props = [new InstallSourceProperty([installer])]
  desc.installations = (desc.installations as List) +
                       new Maven.MavenInstallation('maven-3.9', null, props)
  desc.save(); println '[init] Maven 3.9 déclaré'
}
```

`03-credentials.groovy`

```
import jenkins.model.Jenkins   // init.groovy.d : pas d'imports automatiques
import com.cloudbees.plugins.credentials.*
import com.cloudbees.plugins.credentials.domains.Domain
import com.cloudbees.plugins.credentials.impl.UsernamePasswordCredentialsImpl

def store = Jenkins.get().getExtensionList(SystemCredentialsProvider)[0].store
def existing = store.getCredentials(Domain.global()).find { it.id == 'nexus-deploy' }
def user = System.getenv('NEXUS_USER'); def pass = System.getenv('NEXUS_PASSWORD')
if (!existing && user && pass) {
  store.addCredentials(Domain.global(),
    new UsernamePasswordCredentialsImpl(CredentialsScope.GLOBAL, 'nexus-deploy',
                                        'Compte technique Nexus', user, pass))
  println '[init] credential nexus-deploy créé'
}
```

init.groovy.d vs JCasC

Aujourd'hui : **JCasC** pour tout ce qu'il sait décrire (sécurité, tools, credentials, nodes, clouds, vues, plugins config) ; `init.groovy.d` pour les cas résiduels (plugin sans support JCasC, logique conditionnelle). Les deux se versionnent et se montent dans l'image Docker.

Scripts Groovy (4/4) : Job DSL et génération de jobs

Job DSL

- Plugin **Job DSL** : décrire les **jobs**, folders et vues en Groovy, dans un dépôt Git, et les faire générer par un **seed job**
- Complémentaire : JCasC décrit le *serveur*, Job DSL décrit les *jobs*, le Jenkinsfile décrit le *pipeline*
- Les jobs générés sont marqués et peuvent être supprimés automatiquement s'ils disparaissent du DSL (`removedJobAction: DELETE`)
- Référence des méthodes : `<JENKINS_URL>/plugin/job-dsl/api-viewer/`

`jobs/library.groovy`

```
folder('library') {
  displayName('Suite Library')
  description('Jobs de la suite Library — générés par Job DSL, ne pas éditer à la main')
}

['library-core', 'library-api', 'library-portal'].each { repo →
  multibranchPipelineJob("library/${repo}") {
    branchSources {
      github {
        id(repo)
        repoOwner('utopios'); repository(repo)
        scanCredentialsId('github-app')
        buildOriginBranch(true); buildOriginPRMerge(true)
      }
    }
    factory { workflowBranchProjectFactory { scriptPath('Jenkinsfile') } }
    orphanedItemStrategy { discardOldItems { daysToKeep(7) } }
    triggers { periodicFolderTrigger { interval('1h') } }
  }
}

pipelineJob('library/deploy') {
  parameters {
    choiceParam('APP', ['library-api', 'library-portal'])
    stringParam('VERSION', '', 'Version release')
    choiceParam('TARGET_ENV', ['dev', 'recette', 'preprod', 'prod'])
  }
  definition {
    cpsScm {
      scm { git { remote { url('git@git.example.com:utopios/ci-pipelines.git'); credentials('git-ssh-key') }
                  branch('main') } }
      scriptPath('deploy/Jenkinsfile')
    }
  }
}
```

Le seed job

1. Créer un job Pipeline `admin/seed` dont le Jenkinsfile est :

```
pipeline {
  agent { label 'linux' }
  triggers { pollSCM('H/15 * * * *') }     // ou webhook du dépôt ci-jobs
  stages {
    stage('Checkout') { steps { git url: 'git@git.example.com:utopios/ci-jobs.git', credentialsId: 'git-ssh-key' } }
    stage('Generate') {
      steps {
        jobDsl targets: 'jobs/**/*.groovy',
               removedJobAction: 'DELETE',
               removedViewAction: 'DELETE',
               lookupStrategy: 'JENKINS_ROOT',
               sandbox: true
      }
    }
  }
}
```

2. Premier lancement : le script DSL doit être **approuvé** (*In-process Script Approval*) s'il n'est pas en sandbox ; avec `sandbox: true`, seules les méthodes du DSL sont permises et le seed doit tourner sous un utilisateur précis (plugin Authorize Project, *Run as the specified user* pour un seed planifié ou pollé)
3. Chaque push sur `ci-jobs` régénère les jobs ; un job supprimé du DSL est supprimé de Jenkins

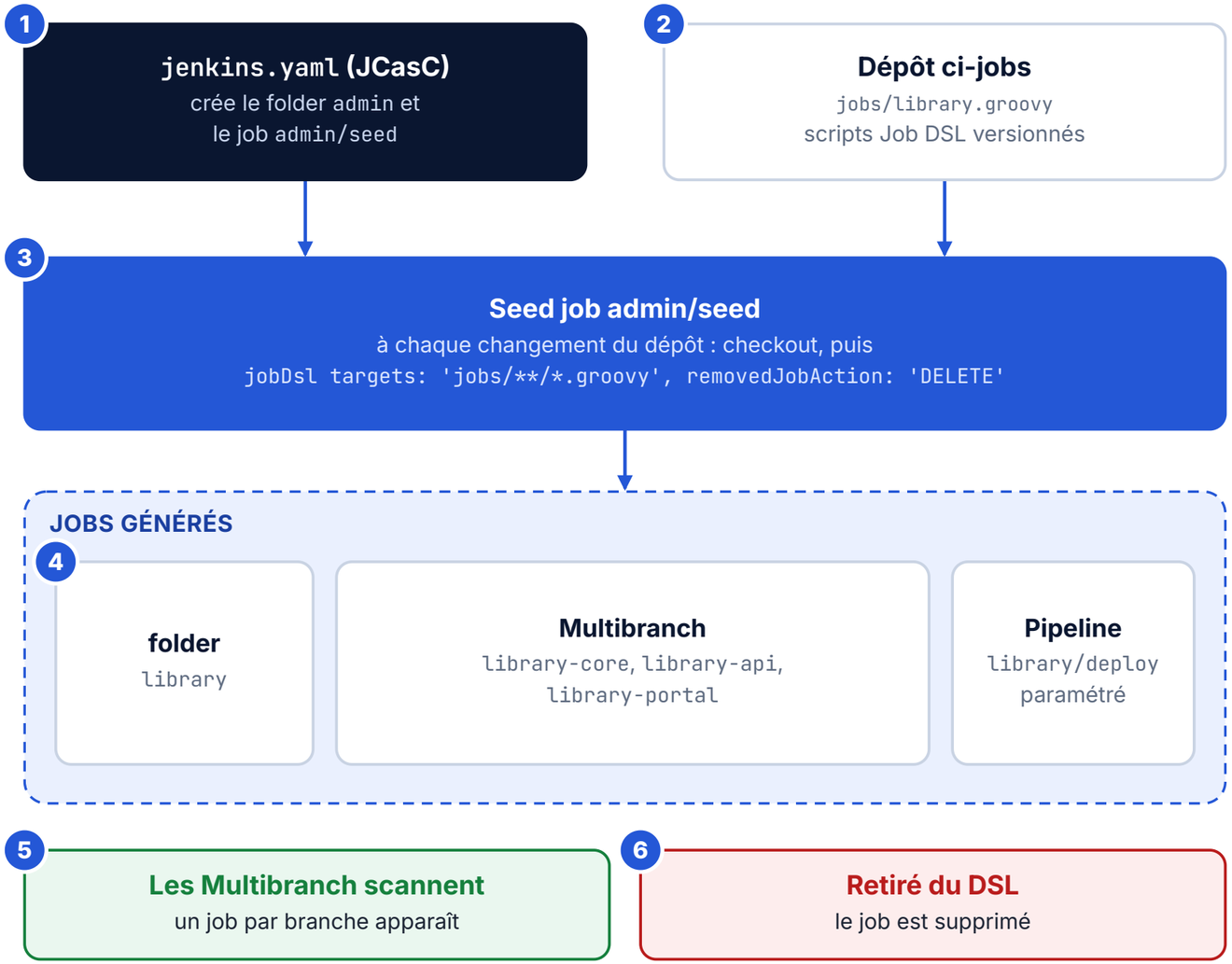
JCasC peut appeler Job DSL

```
# jenkins.yaml
jobs:
  - script: |
      folder('admin')
      pipelineJob('admin/seed') {
        definition { cpsScm {
          scm { git { remote { url('git@git.example.com:utopios/ci-jobs.git')
                               credentials('git-ssh-key') }
                      branch('main') } }
          scriptPath('seed/Jenkinsfile') } }
      }
```

Résultat

Un controller neuf + `jenkins.yaml` → le seed job existe → il génère tous les jobs → les Multibranch scannent → tout est reconstruit sans un clic.

Schéma : du dépôt Git aux jobs, avec Job DSL



Comment le lire

1. Au démarrage, JCasC crée le seul job écrit à la main : le **seed job**.
2. La description de tous les autres jobs vit dans un dépôt Git, en Groovy (Job DSL).
3. Le seed job lit ce dépôt et exécute les scripts : il crée ou met à jour les jobs décrits.
4. On obtient folders, Multibranch et pipelines paramétrés, sans un clic.
5. Les Multibranch scannent leurs dépôts et créent les jobs de branches.
6. Un job retiré du DSL est supprimé au passage suivant (`removedJobAction: 'DELETE'`) : modifier un job généré dans l'UI est inutile, il sera écrasé.

À retenir

Un controller neuf + `jenkins.yaml` + le dépôt `ci-jobs` = tous les jobs reconstruits.

Ligne de commande d'administration (1/3) : la CLI Jenkins

Installation

```
# Le jar est servi par le controller (même version que lui)
curl -sSLo jenkins-cli.jar https://jenkins.example.com/jnlpJars/jenkins-cli.jar
java -jar jenkins-cli.jar -s https://jenkins.example.com/ help
```

Authentification

Mode	Commande	Remarque
HTTP + token (défaut)	`-auth user:apitoken` ou `-auth @/path/fichier`	Token créé dans *User → Security → API Token* ; le fichier évite le token dans l'historique shell
SSH	`ssh -l user -p 2222 jenkins.example.com help`	Clé publique dans le profil utilisateur ; port SSH activé dans *Security → SSH Server*
WebSocket	`-webSocket`	Traverse les reverse proxies sans configuration spéciale

```
# Variables d'environnement pour alléger les commandes
export JENKINS_URL=https://jenkins.example.com/
export JENKINS_USER_ID=admin
export JENKINS_API_TOKEN=11xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
alias jcli='java -jar ~/jenkins-cli.jar -webSocket'
jcli who-am-i
```

Droits

La CLI respecte exactement les permissions de l'utilisateur : un développeur ne peut lancer que les jobs qu'il peut lancer dans l'UI.

Commandes essentielles

Commande	Rôle
`help` , `version` , `who-am-i`	Découverte
`list-jobs [folder]`	Lister les jobs
`build <job> [-p K=V] [-s] [-f] [-v]`	Lancer (-s : attendre, -f : suivre, -v : afficher la console)
`console <job> [#]`	Afficher la console d'un build
`stop-builds <job>`	Interrompre les builds en cours
`get-job` / `create-job` / `update-job` / `delete-job` / `copy-job`	Gérer les jobs via leur `config.xml`
`enable-job` / `disable-job`	Activer / désactiver
`get-node` / `create-node` / `update-node` / `delete-node`	Gérer les agents
`online-node` / `offline-node -m "raison"` / `connect-node` / `disconnect-node` / `wait-node-online`	État des agents
`install-plugin <id[:version min]|url⟹ [-deploy] [-restart]` (`=` : fichier lu sur stdin), `list-plugins` , `enable-plugin` , `disable-plugin`	Plugins
`get-credentials-as-xml` , `create-credentials-by-xml` , `list-credentials`	Credentials (par store/domaine)
`get-view` / `create-view` / `update-view`	Vues
`quiet-down [-block]` / `cancel-quiet-down` / `safe-restart` / `restart` / `safe-shutdown` / `shutdown`	Cycle de vie
`reload-configuration` , `reload-job`	Recharger depuis le disque
`groovy =` / `groovysh`	Exécuter du Groovy sur le controller (comme la Script Console, mais **sans ses imports automatiques**)
`declarative-linter`	Valider un Jenkinsfile
`clear-queue` , `keep-build` , `delete-builds <job> 1-50` , `set-build-description`	Builds

Ligne de commande d'administration (2/3) : scénarios d'exploitation

Lancer et suivre un build

```
jcli build library/deploy -s -v \
     -p APP=library-api -p VERSION=2.3.0 -p TARGET_ENV=recette
echo "code retour : $?"      # 0 = SUCCESS, autre = échec/instable (-s)
```

Exporter / importer / dupliquer des jobs

```
# Sauvegarde texte de toutes les configurations de jobs
mkdir -p jobs-export
for j in $(jcli list-jobs library); do
  jcli get-job "library/$j" > "jobs-export/$j.xml"
done

# Copier un job (non généré par Job DSL) et modifier la branche
jcli get-job library/library-legacy \
  | sed 's#\*/main#*/release/2.x#' \
  | jcli create-job library/library-legacy-release

# Mettre à jour un job existant (un job généré serait écrasé par le seed)
jcli update-job library/library-legacy < jobs-export/library-legacy.xml
```

Gérer les agents

```
# Maintenance planifiée d'un agent
jcli offline-node linux-1 -m "Patching kernel - $(date +%F)"
# … attendre la fin des builds en cours (voir Build Executor Status) …
jcli disconnect-node linux-1 -m "reboot"
# après redémarrage
jcli connect-node linux-1 && jcli online-node linux-1 && jcli wait-node-online linux-1

# Créer un agent depuis un modèle XML
jcli get-node linux-1 | sed 's/linux-1/linux-3/g; s/linux-1.example.com/linux-3.example.com/' \
  | jcli create-node linux-3
```

Plugins

```
# Installer des plugins puis redémarrer proprement
jcli install-plugin role-strategy configuration-as-code job-dsl -deploy
jcli safe-restart

# Lister avec versions installées (comparer à plugins.txt) ; list-plugins
# affiche « x.y (z.w) » quand une mise à jour existe : passer par l'API
curl -sg -u "$JENKINS_USER_ID:$JENKINS_API_TOKEN" \
  "${JENKINS_URL}pluginManager/api/json?tree=plugins[shortName,version]" \
  | jq -r '.plugins[] | "\(.shortName):\(.version)"' | sort
```

Fenêtre de maintenance complète

```
#!/usr/bin/env bash
set -e
jcli quiet-down -block                # attend la fin de tous les builds
KEEP_QUIET=1 /usr/local/sbin/jenkins-backup.sh   # sauvegarde, sans lever le quiet mode
sudo apt-get install -y jenkins       # mise à jour LTS (redémarre le service)
until curl -sf "${JENKINS_URL}login" >/dev/null; do sleep 5; done   # controller revenu
jcli cancel-quiet-down
jcli build admin/cluster-smoke -s     # test de non-régression
```

Groovy depuis le shell

```
# Exécuter un script versionné sans ouvrir l'UI
# (la CLI n'ajoute pas les imports de la Script Console : les écrire en tête)
jcli groovy = < scripts/jobs-sans-retention.groovy

# One-liner
echo 'import jenkins.model.Jenkins; import hudson.model.Job
println Jenkins.get().getAllItems(Job).size()' | jcli groovy =
```

Scriptabilité

Tout ce qui précède s'intègre dans Ansible, des runbooks ou des pipelines d'administration. La CLI est la voie recommandée pour les opérations *d'exploitation* ; JCasC/Job DSL pour la *configuration*.

Ligne de commande d'administration (3/3) : API REST

Principe

- Chaque page de Jenkins a une représentation API : ajouter `/api/json` (ou `/api/xml`, `/api/python`) à l'URL
- Paramètre `tree=` pour ne demander que les champs utiles (performance) ; `depth=` pour la profondeur
- Authentification : **Basic** avec user + **API token** ; les requêtes `POST` nécessitent un **crumb** CSRF *sauf* avec un token API (le token exempte du crumb)
- Documentation : `<JENKINS_URL>/api/` et `<URL de n'importe quelle page>/api/`

Lecture

```
J=https://jenkins.example.com; A="admin:$JENKINS_API_TOKEN"

# Jobs et couleur du dernier build
curl -sg -u "$A" "$J/api/json?tree=jobs[name,color]" | jq

# Détail d'un build
curl -sg -u "$A" "$J/job/library/job/library-api/job/main/42/api/json?
tree=result,duration,timestamp,building,actions[parameters[name,value]]"

# Dernier build réussi : numéro
curl -s -u "$A" "$J/job/library/job/library-api/job/main/lastSuccessfulBuild/buildNumber"

# Console
curl -s -u "$A" "$J/job/library/job/library-api/job/main/42/consoleText" | tail -50

# Résultats de tests
curl -s -u "$A" "$J/job/library/job/library-api/job/main/42/testReport/api/json?
tree=failCount,passCount,skipCount"

# Nodes
curl -sg -u "$A" "$J/computer/api/json?tree=computer[displayName,offline,idle,numExecutors]"

# Queue
curl -sg -u "$A" "$J/queue/api/json?tree=items[task[name],why,inQueueSince]"
```

Écriture

```
# Lancer un build (sans paramètres) - 201 Created, header Location = item de queue
curl -s -i -u "$A" -X POST "$J/job/library/job/library-core/job/main/build"

# Avec paramètres
curl -s -u "$A" -X POST "$J/job/library/job/deploy/buildWithParameters" \
     --data-urlencode APP=library-api --data-urlencode VERSION=2.3.0 \
     --data-urlencode TARGET_ENV=recette

# Suivre : queue item → numéro de build
QURL=$(curl -s -i -u "$A" -X POST "$J/job/library/job/deploy/buildWithParameters?APP=library-api&VERSION=2.3.0&TARGET_ENV=dev" \
       | awk '/^Location:/ {print $2}' | tr -d '\r')
sleep 6; curl -s -u "$A" "${QURL}api/json" | jq '.executable.number'

# Interrompre un build
curl -s -u "$A" -X POST "$J/job/library/job/deploy/57/stop"

# Activer / désactiver un job
curl -s -u "$A" -X POST "$J/job/legacy/job/old-job/disable"

# Lire / écrire la configuration
curl -s -u "$A" "$J/job/library/job/deploy/config.xml" > deploy.xml
curl -s -u "$A" -X POST "$J/job/library/job/deploy/config.xml" \
     -H 'Content-Type: application/xml' --data-binary @deploy.xml

# Quiet mode / redémarrage
curl -s -u "$A" -X POST "$J/quietDown"
curl -s -u "$A" -X POST "$J/safeRestart"

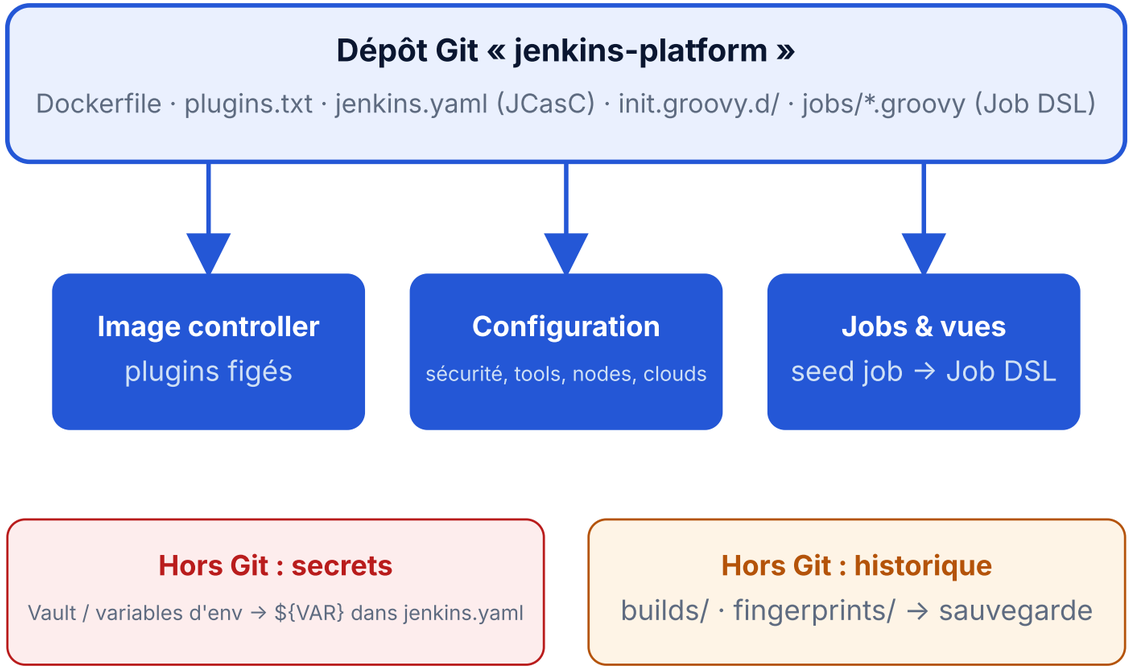
# Script Groovy
curl -s -u "$A" -X POST "$J/scriptText" \
     --data-urlencode "script=println Jenkins.get().version"
```

Avec un mot de passe (sans token) : crumb obligatoire

`CRUMB=$(curl -s -c cj -b cj -u user:pass "$J/crumbIssuer/api/json" | jq -r '.crumbRequestField+":"+.crumb')` puis `-c cj -b cj -H "$CRUMB"` sur chaque POST (le crumb est lié à la session : garder le cookie). Les tokens API évitent cette étape : toujours les préférer pour l'automatisation.

Clients : `python-jenkins`, `jenkinsapi` (Python), `jenkins-rest` (Java), module Ansible `community.general.jenkins_job`, Terraform provider `taiidani/jenkins`.

Jenkins « as Code » : vue d'ensemble



Dockerfile de la plateforme

```
FROM jenkins/jenkins:2.580.2-lts-jdk21
ENV JAVA_OPTS="-Djenkins.install.runSetupWizard=false"
# hors du volume /var/jenkins_home : une nouvelle image met bien le YAML à jour
ENV CASC_JENKINS_CONFIG=/usr/share/jenkins/casc
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
COPY casc/*.yaml /usr/share/jenkins/casc/
# ref/ n'est recopié dans le volume que si le fichier est absent (suffixe .override sinon)
COPY init.groovy.d/ /usr/share/jenkins/ref/init.groovy.d/
```

Secrets dans JCasC

```
credentials:
  system:
    domainCredentials:
      - credentials:
          - usernamePassword:
              id: nexus-deploy
              username: "${NEXUS_USER}"          # résolu depuis l'environnement,
              password: "${NEXUS_PASSWORD}"      # un fichier (/run/secrets) ou Vault
          - basicSSHUserPrivateKey:
              id: agent-ssh-key
              username: jenkins
              privateKeySource:
                directEntry: { privateKey: "${readFile:/run/secrets/agent_key}" }
```

- *Manage Jenkins → Configuration as Code* : **View Configuration** (exporter l'état actuel en YAML : point de départ), **Reload existing configuration**, **Apply new configuration**
- Au rechargement, les éléments déclarés dans le YAML sont réappliqués (les listes déclarées en entier, comme les nodes ou les credentials, sont remplacées) ; un réglage absent du YAML reste en l'état : sans discipline, la configuration dérive

Synthèse

4 – Projets complexes

- Enchaîner : `build job:` depuis un orchestrateur, `upstream`, déclenchement par SNAPSHOT ; ou mono-dépôt multi-modules (reactor)
- Jakarta EE : EJB + WAR + EAR par Maven, tests avec Testcontainers, déploiement WildFly par plugin Maven
- Ant : tool `ant`, rapports JUnit XML, publication via Nexus Uploader ; migration progressive
- Multi-outils : un conteneur par stage, `parallel`, `stash` ; Shared Libraries pour factoriser
- Référentiels : snapshots vs releases, CI-friendly versions, tag Git = release ; Nexus/Artifactory
- Déploiement : pipeline CD paramétré, promotion, `input`, `lock`, `milestone`, stratégies rolling / blue-green / canary

5 – Cluster

- Controller à 0 executor ; agents = capacité, hétérogénéité, sécurité
- Unix : SSH (controller → agent) ou inbound en systemd ; Windows : inbound en service WinSW ; macOS : launchd
- Éphémères : Docker plugin, Kubernetes plugin (pods, build d'images sans démon Docker)
- Répartition : labels et expressions, mode « Only build jobs with label », agent par stage, matrix, locks ; lire la queue et les node monitors
- Déploiement : topologie par zones, agents de déploiement isolés, IaC + JCasC pour nodes et clouds, job de smoke

6 – Administration

- Vues : List/Dashboard/Build Monitor, déclarées en JCasC ou Job DSL ; vues ≠ droits
- Multi-OS : `isUnix()`, `sh` / `bat` / `powershell`, Tool Locations, UTF-8, Git CRLF
- `JENKINS_HOME` : irremplaçable (`secrets/`, `*.xml`, `jobs/*/config.xml`) vs historique vs régénérable
- Monitorer : Prometheus/Metrics, JVM, queue, nodes, disque ; job `cluster-smoke`
- Sauvegarder : quiet mode + tar sélectif hors site ; tester la restauration ; LTS tous les 3 mois
- Groovy : Script Console (lire puis écrire), `init.groovy.d`, Job DSL ; CLI et API REST avec tokens

Fil conducteur des six parties

Jenkins décrit en code (image + JCasC + Job DSL + Jenkinsfile + Shared Libraries), des agents jetables, des artefacts immuables promus d'environnement en environnement, et une plateforme supervisée et sauvegardée. Chaque clic dans l'UI est une dette à convertir en fichier versionné.

Ressources et pour aller plus loin

Documentation

- `jenkins.io/doc/book/managing` – administration (nodes, sécurité, plugins, system properties)
- `jenkins.io/doc/book/scaling` – architecture et montée en charge
- `jenkins.io/doc/book/managing/cli` et `jenkins.io/doc/book/using/remote-access-api`
- `github.com/jenkinsci/configuration-as-code-plugin` – démos JCasC par plugin
- `jenkinsci.github.io/job-dsl-plugin` – référence Job DSL
- `jenkins.io/doc/book/pipeline/shared-libraries`
- `javadoc.jenkins.io` – API pour les scripts Groovy
- `github.com/jenkinsci/docker` – image officielle, `jenkins-plugin-cli`, `plugins.txt`
- `github.com/jenkinsci/helm-charts` – Jenkins sur Kubernetes
- Maven : `maven.apache.org/maven-ci-friendly.html`, `maven.apache.org/plugins/maven-ear-plugin` ; Ant : `ant.apache.org/manual`

Lectures

- Jez Humble, David Farley – *Continuous Delivery* (promotion, pipelines de déploiement)
- Gene Kim et al. – *The DevOps Handbook*
- CloudBees – *Jenkins Best Practices* (controller/agents, sécurité)

Travaux pratiques proposés

1. Mono-dépôt `library-suite` multi-modules + enchaînement vers `library-portal` par SNAPSHOT trigger
2. Construire et déployer l'EAR sur un WildFly Docker, tests Testcontainers
3. Pipeline multi-outils (Maven + npm + Docker) avec `parallel` et une Shared Library à 3 steps
4. Release par tag Git vers `maven-releases`, puis promotion recette → préprod → prod avec `input` et `milestone`
5. Cluster : agent Linux SSH, agent Linux inbound systemd, agent Windows WinSW, cloud Docker ; job `cluster-smoke` ; mettre un agent hors ligne et observer la queue
6. Exporter la configuration en JCasC, reconstruire le controller depuis l'image + `jenkins.yaml` + seed Job DSL, restaurer `secrets/` et vérifier les credentials
7. Prometheus + Grafana sur le controller ; alerte sur la queue ; script de sauvegarde en cron et restauration sur une instance jetable

Certification

- **Certified Jenkins Engineer** (CJE) : couvre l'ensemble des six parties (CI/CD, pipelines, agents, administration, sécurité)

Utopios – MA-IT Consulting · Jenkins – Projets complexes, cluster, administration · Support v1.5